A BEGINNER'S PROGRAMMING TEXTBOOK

Java from Zero

Learn Java step by step — from your first program to real, working projects

```java
import java.util.Scanner;

public class Greeter {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Your name: ");
        String name = in.nextLine();
        System.out.println("Hello, " + name + "!");
    }
}
```

Nine chapters of small, friendly lessons take you from a blank editor to a complete grade-book program — with checks, quizzes, challenges, and mini-projects to make every idea stick.

AUDIENCE	LEVEL	EDITION
Students & Self-Learners	**Beginner — No Experience**	**First**

9 CHAPTERS · 3 MINI-PROJECTS · FINAL PROJECT

About This Book

Licence, support, and feedback

Free to read

Java from Zero is a free book. You never need to pay to read it, print it, or hand it to a friend, a classroom, or a study group.

Licence

© 2026 Mohamed Ahmed Fayad. This work is licensed under the Creative Commons Attribution-NonCommercial 4.0 International licence (CC BY-NC 4.0). You are welcome to share and adapt it for any non-commercial purpose, as long as you give credit and say if you made changes. Full licence text: creativecommons.org/licenses/by-nc/4.0

Support the next book

If this book helped you learn, a small donation helps keep it updated and pays for more free lessons. Donating is always optional.

Donate at: **ko-fi.com/1to10**

Found a mistake?

No book is perfect. If you spot a typo, a program that does not run, or a question with the wrong answer, please tell us so the next version can fix it.

Report it here: **moh.fayad14@gmail.com**

Edition

First Edition · Version 1.0 · October 2026. A newer version may be available where you downloaded this book.

The programs in this book are written for a modern Java Development Kit (JDK) and run in any editor or from the command line. Java and OpenJDK are trademarks or registered trademarks of Oracle and/or its affiliates. This book is an independent educational resource and is not affiliated with or endorsed by Oracle or any other company named in it.

Table of Contents

How to Use This Book 5

Chapter 1 — Getting to Know Java 7

Lesson 1.1: What Is Java? 7

Lesson 1.2: How Java Runs: From Text to Screen 8

Lesson 1.3: Your First Program 9

Lesson 1.4: Comments and Error Messages 11

Chapter 1 Practice Questions 12

Chapter 1 Mixed Quiz 13

Chapter 1 Challenges 15

Chapter 1 Recap 16

Chapter 1 Common Mistakes 17

Chapter 2 — Variables and Data Types 18

Lesson 2.1: Variables and Assignment 18

Lesson 2.2: int and double 20

Lesson 2.3: Strings and chars 22

Lesson 2.4: booleans and Reading Input 24

Chapter 2 Practice Questions 27

Chapter 2 Mixed Quiz 28

Chapter 2 Challenges 30

Chapter 2 Recap 31

Chapter 2 Common Mistakes 32

Chapter 3 — Operators and Expressions 33

Lesson 3.1: Arithmetic Operators 33

Lesson 3.2: Integer Division and Remainder 35

Lesson 3.3: Precedence, Parentheses, and Casting 37

Lesson 3.4: Comparison and Logical Operators 39

Chapter 3 Practice Questions 42

Chapter 3 Mixed Quiz 43

Chapter 3 Challenges 45

Chapter 3 Recap 46

Chapter 3 Common Mistakes . . . 46

Mini-Project 1: Cafe Order Calculator . . . 48

Step 1 — The Skeleton and the Input . . . 48

Step 2 — The Arithmetic . . . 48

Step 3 — The Receipt . . . 49

Chapter 4 — Making Decisions . . . 50

Lesson 4.1: The if Statement . . . 50

Lesson 4.2: else and else-if Chains . . . 52

Lesson 4.3: Nested ifs and Common Patterns . . . 54

Lesson 4.4: The switch Statement . . . 56

Chapter 4 Practice Questions . . . 59

Chapter 4 Mixed Quiz . . . 61

Chapter 4 Challenges . . . 63

Chapter 4 Recap . . . 64

Chapter 4 Common Mistakes . . . 64

Chapter 5 — Loops . . . 66

Lesson 5.1: while Loops . . . 66

Lesson 5.2: for Loops . . . 68

Lesson 5.3: do-while and Input Validation . . . 70

Lesson 5.4: break, continue, and Nested Loops . . . 72

Chapter 5 Practice Questions . . . 74

Chapter 5 Mixed Quiz . . . 76

Chapter 5 Challenges . . . 77

Chapter 5 Recap . . . 78

Chapter 5 Common Mistakes . . . 78

Chapter 6 — Methods . . . 80

Lesson 6.1: Your First Methods . . . 80

Lesson 6.2: Parameters . . . 82

Lesson 6.3: Return Values . . . 84

Lesson 6.4: Overloading, Scope, and Javadoc . . . 86

Chapter 6 Practice Questions . . . 88

Chapter 6 Mixed Quiz . . . 90

Chapter 6 Challenges . . . 92
Chapter 6 Recap . . . 93
Chapter 6 Common Mistakes . . . 93

Mini-Project 2: Number Guessing Game . . . 95
Step 1 — A Secret Number Helper . . . 95
Step 2 — One Round of Guessing . . . 95
Step 3 — main Brings It Together . . . 96

Chapter 7 — Arrays . . . 98
Lesson 7.1: Creating Arrays . . . 98
Lesson 7.2: Arrays and Loops . . . 100
Lesson 7.3: Array Algorithms . . . 102
Lesson 7.4: 2D Arrays and java.util.Arrays . . . 104
Chapter 7 Practice Questions . . . 106
Chapter 7 Mixed Quiz . . . 107
Chapter 7 Challenges . . . 109
Chapter 7 Recap . . . 110
Chapter 7 Common Mistakes . . . 111

Chapter 8 — Working with Strings . . . 112
Lesson 8.1: Useful String Methods . . . 112
Lesson 8.2: Comparing Strings and Immutability . . . 114
Lesson 8.3: printf and Formatting . . . 116
Lesson 8.4: StringBuilder . . . 118
Chapter 8 Practice Questions . . . 120
Chapter 8 Mixed Quiz . . . 121
Chapter 8 Challenges . . . 123
Chapter 8 Recap . . . 124
Chapter 8 Common Mistakes . . . 125

Chapter 9 — Files and Exceptions . . . 127
Lesson 9.1: Exceptions: try and catch . . . 127
Lesson 9.2: finally and throw . . . 129
Lesson 9.3: Reading Files . . . 131
Lesson 9.4: Writing and Appending Files . . . 134

Chapter 9 Practice Questions . . . 136
Chapter 9 Mixed Quiz . . . 138
Chapter 9 Challenges . . . 140
Chapter 9 Recap . . . 141
Chapter 9 Common Mistakes . . . 141

Mini-Project 3: Personal Journal (Diary) App . . . 143
Step 1 — Writing an Entry (Append Mode) . . . 143
Step 2 — Reading the Journal Back . . . 144
Step 3 — The Menu Loop in main . . . 144

Final Project : Student Grade Book . . . 147
Step 1 — The Data: Parallel Arrays . . . 147
Step 2 — The Brain: Helper Methods . . . 147
Step 3 — The Report: Loops and printf . . . 148
Step 4 — Saving and Loading the File . . . 149
Step 5 — The Menu: do-while and switch . . . 151
Step 6 — Combine Into One File . . . 152

Glossary . . . 153

Answer Key . . . 157
Part 1 — Lesson Check Answers . . . 157
Part 2 — Practice and Mixed Quiz Answers . . . 179
Part 3 — Challenge Solutions . . . 207

Index . . . 237

START HERE

How to Use This Book

This book teaches Java from absolute zero. You do not need any programming experience, and you do not need to be a math genius. You only need a computer, a little patience, and the willingness to type out the examples yourself instead of just reading them. Typing code by hand forces your brain to notice the small details that make programs work, and those details are exactly where beginners usually stumble.

The book has **nine chapters**. Each chapter contains four **lessons** that introduce one idea at a time, with plenty of runnable code examples in shaded boxes. Every lesson ends with a **Lesson Check** of four multiple-choice questions that verify you really understood what you just read. Take these checks honestly: if you miss a question, re-read the relevant part of the lesson before moving on, because later lessons build on earlier ones.

After the four lessons, each chapter gives you four practice sections in a fixed order. **Practice Questions** are ten problems ordered from easiest to hardest: five multiple choice, one true/false, one code-output prediction, one find-the-bug puzzle, one short written answer, and one small program to write. The **Mixed Quiz** throws fifteen questions at you in random style, mixing everything from the chapter. The five **Challenges** are genuinely hard thinking problems; each has a hint, and full step-by-step solutions live in the Answer Key.

Every chapter closes with a **Chapter Recap** you can use for fast revision and a **Common Mistakes** section that shows the classic beginner errors for that topic, why they happen, and how to fix them. After Chapters 3, 6, and 9 you will build a **Mini-Project**: a complete, runnable program with a line-by-line explanation and extra ideas to try on your own. The book ends with a **Final Project**, the Student Grade Book, which combines every concept from all nine chapters into one program that you build step by step.

The back of the book is your safety net. The **Answer Key** explains every answer in detail: the correct choice appears in green with the reasoning behind it, plus an explanation of why each wrong option is wrong, so a wrong guess still teaches you something. The **Glossary** defines more than fifty terms in plain English, and the **Index** tells you exactly which chapter and lesson introduced every keyword, method, and topic.

If you are studying alone, a sustainable rhythm is one lesson per day and one full chapter per week: read, type the examples, take the Lesson Check, and stop while you still have energy. If you are using this book in a classroom, the Lesson Checks work as warm-up questions, the Mixed Quiz as a weekly test, and the Challenges as homework or pair work. However you use it, the golden rule never changes: **run every program yourself**. Java is a skill, and skills grow through practice, not reading.

Feature	What it is for
Lesson	New idea explained step by step, with runnable code.
Lesson Check	Four multiple-choice questions right after each lesson.
Practice Questions	Ten graded problems at the end of each chapter.
Mixed Quiz	Fifteen mixed questions per chapter.
Challenges	Five hard problems with hints and full solutions.

Feature	What it is for
Mini-Project	A complete program after Chapters 3, 6, and 9.
Final Project	The Student Grade Book, built step by step.
Glossary / Index	Definitions of every term and where to find it.

CHAPTER 1

Getting to Know Java

Every programmer's journey starts with a single program that prints a few words on the screen. In this chapter you will learn what Java is, how a Java program travels from the text you type to the results you see, and how to write, run, and fix your very first program. By the end of the chapter you will be able to read simple Java code out loud and explain what every line is doing.

In this chapter you will learn to:

- explain what Java is and why it is worth learning
- describe how Java source code becomes a running program
- write and run a complete Java program with **main** and **System.out.println**
- use comments to leave notes inside your code
- recognize and fix the most common first-week errors

Lesson 1.1 — What Is Java?

Java is a programming language: a precise language for writing instructions that a computer can follow. It was created in 1995 by a team led by James Gosling at Sun Microsystems, and it is now maintained by Oracle. Thirty years later it is still one of the most used languages on Earth, powering bank systems, Android apps, Minecraft, scientific tools, and the software behind many websites you visit every day.

Why start with Java instead of some easier-looking language? First, Java is strict. It checks your spelling, your punctuation, and many of your mistakes **before** the program runs, which means you get clear error messages at the exact spot where something went wrong. Second, Java is everywhere in industry, so the habits you build here transfer directly to real jobs. Third, Java teaches you the structure that almost every other language copied, so your second language will be far easier than your first.

A Java program is just a list of instructions, like a recipe. The computer follows the recipe exactly, in order, doing precisely what the words say. That last sentence is worth reading twice: the computer does what you **typed**, not what you **meant**. Most beginner frustration comes from the small gap between those two things, and most beginner growth comes from learning to close that gap.

> **NOTE**
> Java and JavaScript are two completely different languages with confusingly similar names. JavaScript runs inside web browsers; Java runs on servers, phones, and almost everywhere else. Knowing one does not mean you know the other.

> **TIP**
> From this chapter onward, type every example into a real editor and run it. Reading code feels like understanding it, but only typing it builds the reflexes you need.

Lesson Check 1.1

Choose the one best answer for each question. Answers are in the Answer Key.

1. Who led the team that originally created Java?

A. Guido van Rossum at CWI
B. James Gosling at Sun Microsystems
C. Bill Gates at Microsoft
D. Dennis Ritchie at Bell Labs

2. A Java program is best described as...

A. a list of instructions the computer follows in order
B. a description of what you want, however phrased
C. a picture that the computer displays
D. a list of goals the computer tries to guess

3. Which sentence about Java's strictness is true?

A. Java only reports errors after weeks of use
B. Java silently fixes all your mistakes
C. Java refuses to check anything you write
D. Java catches many mistakes before the program runs

4. Which statement about Java and JavaScript is correct?

A. Java programs run only inside web browsers
B. One is the newer release of the other
C. They are two different languages with similar names
D. JavaScript is the browser version of Java

Lesson 1.2 — How Java Runs: From Text to Screen

Java code that you write is called **source code**, and it lives in a plain text file whose name ends in **.java**. Computers cannot execute text directly, so a program called the **compiler** (a tool named **javac**) translates your source code into **bytecode**, a compact set of instructions stored in a file whose name ends in **.class**. Translation happens once; the bytecode can then run many times.

Bytecode is not for any particular chip or operating system. Instead it runs on the **Java Virtual Machine** (JVM), a program whose whole job is to execute bytecode. You start the JVM with the **java** command. This is the famous idea behind the slogan “write once, run anywhere”: the same .class file normally runs on Windows, macOS, and Linux, because each of those systems has its own JVM that understands the same bytecode.

Stage	What happens	Tool or file
1. Write	You type source code into a text editor	Hello.java
2. Compile	The compiler checks the code and translates it	javac Hello.java
3. Result	Bytecode is produced for the JVM	Hello.class
4. Run	The JVM executes the bytecode	java Hello

To follow along at home you need the **JDK** (Java Development Kit), which contains the compiler javac, the JVM, and the standard libraries. Many beginners instead use an IDE such as IntelliJ IDEA, Eclipse, or VS Code with a Java extension; an IDE hides the two commands behind buttons but performs exactly the same steps. Knowing what the buttons really do will save you hours of confusion later, so keep the table above in mind even if you never open a terminal yourself.

```
javac Hello.java      // compile: produces Hello.class
java Hello            // run: JVM executes the bytecode
```

Terminal commands for compiling and running (not part of a Java file)

> **WARNING**
> The public class name and the file name must match exactly: a class called **Hello** must live in a file called **Hello.java**. A mismatch is one of the first errors almost every beginner meets.

> **NOTE**
> When an error appears it is just the compiler protecting you. Red text is not punishment; it is a map that points at the line that confused it.

Lesson Check 1.2

Choose the one best answer for each question. Answers are in the Answer Key.

1. What does the javac compiler produce from Hello.java?

A. Hello.class, a bytecode file
B. Hello.txt, a plain text copy
C. Hello.exe, a Windows program
D. Hello.jar, a compressed archive

2. Which program actually executes Java bytecode?

A. The javac compiler
B. The operating system shell directly
C. The Java Virtual Machine (JVM)
D. The text editor

3. A public class is named Greeting. What must the source file be called?

A. Greeting.class
B. Greeting.java
C. greeting.java
D. Any name is fine

4. Why does the same .class file normally run on Windows, macOS, and Linux?

A. Java programs contain the source code of all systems
B. Bytecode is written separately for each system
C. The compiler secretly produces three files
D. Each system has its own JVM that understands the same bytecode

Lesson 1.3 — Your First Program

Here it is, the program every Java programmer on the planet started with. Read it slowly, then type it exactly as shown, including every brace, dot, and semicolon. The file must be saved as **Hello.java**.

```
public class Hello {

    public static void main(String[] args) {
        // This line prints text to the screen
        System.out.println("Hello, world!");
    }
}
```

Hello.java - your first complete program

Line by line: **public class Hello** starts a container named Hello that holds your program; the opening brace **{** begins its body. Inside the class, **public static void main(String[] args)** defines the **main method**, the exact place where every Java program starts running. For now treat that whole line as a magic spell you write the same way every time; Chapter 6 will pull it apart and explain each word.

Inside main, the statement **System.out.println("Hello, world!");** does the visible work. **System.out** is Java's connection to your screen, **println** is an instruction meaning “print this text and then move to a new line”, and the text in double quotes is a **string literal**: the exact characters to display. The semicolon ends the statement, the way a period ends a sentence. Note that **System.out.println** is a method you are calling, not a keyword of the language.

Java also has **print**, which works exactly like println but does **not** move to a new line afterward. The two together let you control exactly how output is arranged, as the next example shows. Run it and predict the shape of the output before you press Enter.

```
public class PrintDemo {

    public static void main(String[] args) {
        System.out.print("A");
        System.out.print("B");
        System.out.println("C");   // then newline
        System.out.println("D");
    }
}
```

print stays on the same line; println starts a new one. Output: ABC, then D on the next line

TIP

Curly braces always come in pairs: one **{** opens, one **}** closes. Indent every line inside a pair by four spaces. The indentation does not change what the program does, but it lets your eyes see the structure instantly.

WARNING

Java is **case-sensitive**. **System**, **system**, and **SYSTEM** are three different spellings, and only the first one works. The same applies to **String** and **main**: write them exactly as shown.

Lesson Check 1.3

Choose the one best answer for each question. Answers are in the Answer Key.

1. Where does a Java program start running?

A. At the first line of the file
B. In the main method
C. In the class named after the file
D. In the last method written

2. What is the difference between print and println?

A. print shows text; println shows numbers
B. They are two spellings of the same method
C. println moves to a new line afterward; print does not
D. println is faster than print

3. Which line correctly prints Java is fun?

   A. System.out.println("Java is fun");
   B. system.out.println("Java is fun");
   C. System.Out.println("Java is fun");
   D. System.out.println(Java is fun);

4. What ends most Java statements?

   A. A closing brace }
   B. A period .
   C. A newline
   D. A semicolon ;

Lesson 1.4 — Comments and Error Messages

A **comment** is a note written inside your code for humans to read. The compiler completely ignores comments, so they never change what the program does. Java has two everyday forms: **//** comments out everything from the slashes to the end of the line, and **/* ... */** comments out everything between the markers, even across several lines. Good comments explain **why** the code does something, not just what it obviously does.

```
public class CommentDemo {

    // A single-line comment: the compiler skips this.
    /* A block comment can stretch
        across several lines. */
    public static void main(String[] args) {
        System.out.println("Comments are ignored");
        // System.out.println("I never run");
    }
}
```

Only the uncommented println actually executes

Sooner or later you will meet an **error message**. When the compiler cannot understand your code, it stops and prints a message such as **error: ';' expected**, usually with the file name and a line number. This is called a **compile-time error**, and it means the program never even started. The winning habit is to read the **first** error message only, go to the reported line, fix that one problem, and compile again, because one missing semicolon can make the compiler confused about everything after it.

Not all problems are caught by the compiler. A **runtime error** happens while the program is running and normally stops it with a red exception report. A **logic error** is sneakier: the program runs happily to the end but produces the wrong result, for example printing the same line twice by using print where you meant println. Comments, careful typing, and small tests are your main defenses against all three kinds of trouble.

> **TIP**
> When output looks wrong, trace the program on paper: read one statement at a time and write down exactly what it prints. This five-minute habit finds most logic errors without any tools.

> **NOTE**
> Java also has a special comment style, **/** ... */**, used to generate official documentation. You will meet it properly in Chapter 6, in time to use it in the final project.

Lesson Check 1.4

Choose the one best answer for each question. Answers are in the Answer Key.

1. What does the compiler do with comments?

A. Runs them before the main program
B. Ignores them completely
C. Translates them into bytecode
D. Shows them on the screen at runtime

2. Which comment can legally span several lines?

A. # like this #
B. // like this //
C. /* like this */
D. ** like this **

3. The compiler reports: error: ';' expected on line 4. What does this mean?

A. A semicolon is probably missing at or before line 4
B. The program ran but printed the wrong result
C. The computer is missing a file named ';'
D. Line 4 must be deleted

4. A program compiles, runs, and finishes, but prints the wrong total. This is best described as a...

A. comment error
B. runtime error
C. compile-time error
D. logic error

Practice Questions

Work through all ten questions. They are ordered from easiest to hardest. Write your answers on paper before checking the Answer Key.

1. What exactly does System.out.println("Hi"); do?

A. Prints Hi and stays on the same line
B. Saves Hi into a file
C. Prints Hi and then moves to a new line
D. Speaks the word Hi out loud

2. After you compile Hello.java with javac, which file appears?

A. Hello.java.bak
B. Hello.txt
C. hello.class
D. Hello.class

3. Which one is the correct first line of the method where every Java program starts?

A. public static void main(String[] args)
B. static public void Main(String[] args)
C. public static int main(String[] args)
D. public void static main(String args)

4. What is the role of the JVM?

A. It draws windows and buttons on the screen
B. It connects your computer to the internet
C. It converts your typing into source files
D. It executes Java bytecode

5. Which of these would cause a compile-time error?

A. Using println instead of print
B. A missing semicolon at the end of a statement
C. Printing the same line twice
D. A comment that misspells a word

6. The compiler turns your comments into bytecode together with the rest of the code.

[ ] True [ ] False

7. What is the exact output of this program?

```
public class WarmUp {
    public static void main(String[] args) {
        System.out.print("Ja");
        System.out.println("va");
        System.out.println("Rocks!");
    }
}
```

Output:

8. This program does not compile. Find the bug.

```
public class BugHunt {
    public static void main(String[] args) {
        System.out.println("Line one")
        System.out.println("Line two");
    }
}
```

Bug:

9. In two or three sentences, explain the difference between a compile-time error and a logic error.

10. Write a complete program called MyInfo that prints your name on the first line and the text I am learning Java on the second line. Include one comment.

Mixed Quiz

Fifteen questions mixing everything from this chapter. Score 12 or more before moving on to the challenges.

1. What does System.out.println("3 + 4"); print?

A. 34 **B.** 7

C. 3 + 4 **D.** Nothing; it is a comment

2. A public class is named StarBox. What must the source file be called?

A. StarBox.java **B.** starbox.java

C. StarBox.class **D.** Star_Box.java

3. Which command starts the JVM so your bytecode runs?

A. javac Hello **B.** execute Hello.class

C. run Hello **D.** java Hello

4. The lines System.out.print("A"); print("B"); and println("C"); run in order. What appears on the screen?

A. ABC on one line
B. CBA on one line
C. Only C
D. A, then B, then C on three lines

5. Why do programmers write comments?

A. To print extra text on the screen
B. To hide code from the compiler forever
C. To leave notes that humans read and computers ignore
D. To make the program run faster

6. Which line contains a spelling mistake that will not compile?

A. System.out.println("Bye");
B. System.out.Println("Hi");
C. System.out.println("Hi");
D. System.out.print("Hi");

7. What is bytecode?

A. The text you type into the editor
B. The intermediate instructions the compiler produces for the JVM
C. A secret password for Java programs
D. The sound a computer error makes

8. What does the “write once, run anywhere” slogan describe?

A. Downloading Java from any website
B. Typing code once and never testing it
C. Writing programs without a keyboard
D. One .class file running on any system that has a JVM

9. Java source code files always end with the extension .java.

[ ] True [ ] False

10. The JVM starts running your program from the very first line of the file, whatever that line is.

[ ] True [ ] False

11. A block comment between /* and */ can cover three separate lines.

[ ] True [ ] False

12. What is the exact output of this program?

```
public class QuizPrint {
    public static void main(String[] args) {
        System.out.println("Java");
        System.out.print("Is ");
        System.out.print("Fun ");
        System.out.println("Today");
    }
}
```

Output:

13. This program does not compile. Find the bug.

```
public class Oops {
    public static void main(String[] args) {
        System.out.println("Where is my brace?");

}
```

Bug:

14. Explain in two or three sentences why the same .class file can normally run on Windows, macOS, and Linux.

15. Write a complete program called Banner that prints JAVA on the first line and ROCKS on the second line. Add a comment above each println saying what it prints.

Challenges

Five harder thinking problems. Each includes a hint; full step-by-step solutions are in the Answer Key. Try the problem seriously before reading the hint.

Challenge 1.1 — Fix the Broken Program

The program below has five separate mistakes. Without running it first, find all five on paper, then fix them and make the program compile and run. The intended output is exactly two lines: I am here! and Learning Java.

```
public class broken {
    public static void main(String[] args) {
        system.out.PrintLn("I am here!")
        System.out.println("Learning Java);
    }
}
```

Hint: Check five things: the file name versus the class name, the spelling of System, the spelling of println, the ending semicolons, and the closing quote of the second string.

Challenge 1.2 — Star Staircase

Using only println statements (no loops yet!), write a complete program called Staircase that prints exactly this shape of four lines:

*
**
***
****

Hint: One println per line of the staircase. Count the stars on each line carefully; the fourth line has four of them.

Challenge 1.3 — Error Detective

Match each compiler message below to its most likely cause, then write the fix:
1. error: class Greet is public, should be declared in a file named Greet.java
2. error: ';' expected
3. error: cannot find symbol: variable system

Hint: Message 1 is about names, message 2 is about punctuation, and message 3 is about case-sensitivity.

Challenge 1.4 — Name Tag

Write a program called NameTag that prints a framed name tag exactly like this, where the middle line contains your own name:

```
+------------------+
| AMAZING CODER |
+------------------+
```

Hint: Each border line is one println. Count the dashes and spaces so both borders match, and remember that spaces inside quotes are printed too.

Challenge 1.5 — Human Compiler

Act like the JVM. Read the program below and write down, on paper, the exact output it produces. Then run it and check your prediction. Explain any difference in one sentence.

```
public class HumanCompiler {
    public static void main(String[] args) {
        System.out.print("Java ");
        System.out.print("is ");
        System.out.println("fun.");
        System.out.println("Are you ready?");
    }
}
```

Hint: Track the invisible cursor. print leaves it right after the last character; println leaves it at the start of the next line.

Chapter Recap

- Java is a strict, widely used language created in 1995 by James Gosling's team at Sun Microsystems and now maintained by Oracle.
- You write source code in .java files; the compiler javac turns it into bytecode in .class files; the JVM (started with the java command) executes that bytecode.
- The same .class file normally runs on any operating system that has a JVM: write once, run anywhere.
- A public class name and its file name must match exactly, including capitalization.
- Every program starts running in the main method: public static void main(String[] args).
- System.out.println prints text and ends the line; print prints and stays on the same line; statements end with semicolons; Java is case-sensitive.
- Comments (// to end of line, /* ... */ across lines) are ignored by the compiler and exist for human readers.
- Errors come in three flavors: compile-time (caught before running), runtime (crash mid-run), and logic (wrong result, no crash). Fix the first reported error, then recompile.

Common Mistakes

Beginners run into these again and again. Read them now to avoid the pain later.

WARNING

Forgetting the semicolon

Every statement ends with a semicolon, but the compiler often reports the error on the line AFTER the mistake, which makes beginners look in the wrong place and get confused.

Fix: When the compiler says ';' expected, check the end of the reported line AND the line just above it.

WARNING

Lowercasing System or println

Java is case-sensitive, so system.out.println and System.out.PrintLn are both unknown names and produce cannot find symbol errors.

Fix: Copy exact spellings: System.out.println. Watch the capital S and the lowercase l in println.

WARNING

File name and class name do not match

A public class Hello saved in hello.java or HelloWorld.java triggers an error before compilation even begins, and beginners rarely connect the message to the file name.

Fix: Make the file name identical to the public class name: Hello.java for class Hello.

WARNING

Mixing up print and println

Using print when you meant println glues output together onto one line, which looks like a program bug even though the code is perfectly legal.

Fix: Use println whenever a line should end after the text; keep print only when the next output should continue on the same line.

WARNING

Editing the .class file or expecting edits to appear without recompiling

The .class file is generated bytecode. Changing your .java file does nothing to the running program until you compile again, so beginners think their fix “does not work”.

Fix: Always edit the .java source, then compile again with javac, then run with java.

CHAPTER 2

Variables and Data Types

Programs would be useless if they could not remember things. A variable is a named storage box inside the computer's memory, and this chapter teaches you how to create boxes, fill them, and read them back. You will meet Java's everyday data types: int and double for numbers, String and char for text, and boolean for true-and-false answers. You will also write your first interactive programs, ones that ask the user questions and use the answers. These skills are the raw material for every chapter that follows.

In this chapter you will learn to:

- declare variables, assign values with **=**, and follow Java's naming rules
- choose between **int** and **double** and predict what Java stores
- join text and numbers with concatenation and use escape sequences
- use **boolean** values and the shortcuts **+=**, **-=**, ***=**, **/=**, and **%=**
- write interactive programs that read input with **Scanner**

Lesson 2.1 — Variables and Assignment

Think of a variable as a labeled box in the computer's memory. You choose the label (the name), Java chooses the box (a small piece of memory), and you decide what goes inside (the value). Once a value is in the box you can look at it as many times as you like, replace it with a new value, and use it in calculations. Every interesting program you will ever write depends on remembering values this way.

Creating a variable takes two pieces of information: the type of value it will hold, and the name you want to give it. The line **int score;** declares a variable named score that will hold whole numbers. The line **score = 50;** is an **assignment**: it stores the value 50 into the box, and programmers read it in their heads as “score gets 50”. The equals sign here does not mean “is equal to” the way it does in mathematics; it is an instruction that means **store**.

```
public class VariableDemo {

    public static void main(String[] args) {
        int score;              // 1. declare: make the box
        score = 50;             // 2. assign: put 50 in it
        System.out.println(score);

        score = 75;             // 3. assign again: replace
        System.out.println(score);

        int lives = 3;          // declare and assign at once
        System.out.println(lives);
    }
}
```

Declaring, assigning, and reassigning. Output: 50, then 75, then 3

The program above shows the three basic moves. First you declare the variable, then you assign it, and later you may assign it again. Reassignment simply replaces the old value: after **score = 75;** the 50 is gone forever, because a box holds exactly one thing at a time. Declaring and assigning can also be combined, as **int lives = 3;** does, and that one-step form is the style you will see most often. A first assignment like this one is called an **initialization**.

Java has firm rules for variable names, which programmers call **identifiers**. A name may contain letters, digits, underscores, and dollar signs, but it may not start with a digit, may not contain spaces, and may not be a reserved word such as **int** or **class**. Names are case-sensitive, so **score**, **Score**, and **SCORE** are three different variables. By convention a variable name starts lowercase, and multi-word names use camelCase, with each new word capitalized: **playerScore**, **topSpeed**, **roomTemperature**.

```
public class NamesDemo {

    public static void main(String[] args) {
        int playerScore = 120;   // legal and descriptive
        double roomArea = 18.5;   // legal: camelCase
        int _temp = 0;            // legal: may start with _
        // int 2fast = 9;         // NO: cannot start with 2
        // int top speed = 9;     // NO: no spaces allowed
        // int int = 9;           // NO: reserved word

        System.out.println(playerScore);
        System.out.println(roomArea);
        System.out.println(_temp);
    }
}
```

Legal and illegal identifiers (the illegal ones are commented out so the file compiles)

WARNING
A local variable must be given a value before you read it. The declaration **int total;** creates the box but leaves it empty, and any attempt to print or use **total** before you assign it stops compilation with the error **variable total might not have been initialized**. Assign first, use second.

One more subtlety: reading a variable never changes it. Printing **score** a hundred times still leaves 50 (or whatever it currently holds) in the box; only an assignment puts a new value inside. This matters because expressions such as **score + 10** compute a brand-new value from the box's contents without touching the box itself. If you want the sum kept, you must store it: **score = score + 10;**. That statement is worth rereading: it takes the current value, adds 10, and stores the result back into the same box.

TIP
Choose names that say what the value means: temperature, not t; secondsLeft, not s. Future you, reading the code next month, is effectively a different person with no memory of what t meant.

Lesson Check 2.1

Choose the one best answer for each question. Answers are in the Answer Key.

1. Which line creates a variable that holds whole numbers and stores 40 in it?

A. int apples == 40;
B. int apples = 40;
C. 40 = int apples;
D. apples = 40;

2. In Java, the = between a variable and a value means...

A. compare the two sides and print the result
B. the two sides are mathematically equal
C. swap the two sides
D. store the right-hand value into the left-hand variable

3. After int lives = 3; runs, then lives = lives - 1; runs, what does lives hold?

A. 2
B. 3
C. 4
D. 1

4. Which of these is NOT a legal variable name?

A. topScore
B. score2
C. 2fast
D. _hidden

Lesson 2.2 — int and double

Whole numbers and decimal numbers get two different types in Java. **int** holds whole numbers such as 7, 0, and -34, while **double** holds numbers with a decimal point such as 3.5, 0.25, and -12.0. The type matters because it decides which operations make sense and what Java does when values are combined. As a rule of thumb, use **int** for counting things (students, lives, points) and **double** for measuring things (temperature, distance, price).

```
public class NumberTypes {

    public static void main(String[] args) {
        int students = 24;
        double temperature = 36.6;
        double price = 9.99;

        System.out.println(students);    // 24
        System.out.println(temperature); // 36.6
        System.out.println(price);       // 9.99
        System.out.println(5);           // int literal
        System.out.println(5.0);         // double literal
    }
}
```

int and double variables, and the difference between the literals 5 and 5.0

What happens when one int and one double meet in the same expression? Java quietly converts the int to a double and computes in double arithmetic, so **3 + 2.0** is the double **5.0**. This one-way conversion (int into double) is called **widening**, and it never loses information. The reverse trip is not automatic: **int x = 4.5;** does not compile, because squeezing a decimal value into a whole-number box could throw part of it away. Chapter 3 introduces casting, the tool that performs this narrowing on purpose.

```
public class MixedMath {

    public static void main(String[] args) {
        int count = 3;
        double factor = 1.5;
        double total = count + factor; // int + double

        System.out.println(total);  // 4.5
        // int x = 4.5;  // does not compile: 4.5 is not int
    }
}
```

Mixing an int with a double widens the result to double

Typing **score = score + 10;** is so common that Java has a shorthand for it: **score += 10;**. The compound assignment operators are **+=**, **-=**, ***=**, **/=**, and **%=**, and each one reads as "combine and store back". So **tokens -= 4** subtracts 4 from tokens and stores the result, and **coins *= 2** doubles coins. These shortcuts work with every numeric type; the example below sticks to int so the values stay easy to check. The last one uses %, the remainder operator that Chapter 3 explains in full.

```
public class ShortcutDemo {

    public static void main(String[] args) {
        int tokens = 10;
        tokens += 5;    // same as: tokens = tokens + 5;
        System.out.println(tokens);   // 15
        tokens *= 2;    // same as: tokens = tokens * 2;
        System.out.println(tokens);   // 30
        tokens -= 4;    // same as: tokens = tokens - 4;
        System.out.println(tokens);   // 26
        tokens /= 2;    // same as: tokens = tokens / 2;
        System.out.println(tokens);   // 13
        tokens %= 5;    // remainder, see Chapter 3
        System.out.println(tokens);   // 3
    }
}
```

The five compound assignment shortcuts, traced step by step

Two facts about int deserve a gentle introduction now. First, when you divide one int by another with **/**, Java drops the fraction part instead of giving you a decimal: **7 / 2** evaluates to **3**, not 3.5. Chapter 3 treats integer division and its partner, the remainder operator, in full, so a short preview is enough here. Second, an int is not infinitely large: it holds whole numbers between about -2.1 billion and +2.1 billion, which covers almost every count a beginner program needs. Java has larger whole-number types for rare bigger needs, but they can wait.

WARNING

Assignment always flows right to left: the expression on the right is computed first, then the result is stored into the variable on the left. Writing **50 = score;** is a compile error, because 50 is a value, not a box, and values cannot be stored into.

NOTE

Why does printing 5.0 show 5.0 while printing 5 shows 5? Because a double keeps its decimal point even when the fraction is zero. The printed form is a clue to the type of the value, which makes output a handy debugging tool.

TIP
When a calculation looks wrong, look for a missing .0 in the output: it almost always means integer division happened or an int variable crept in somewhere.

Lesson Check 2.2

Choose the one best answer for each question. Answers are in the Answer Key.

1. Your program stores a temperature of 36.6 degrees. Which declaration fits?

A. double temperature = 36;
B. double temperature = 36.6;
C. int temperature = 36.6;
D. temperature = 36.6;

2. What does int result = 7 / 2; store in result?

A. 3.5
B. The line does not compile
C. 4
D. 3

3. int n = 5; then n += 3; runs. What is n now?

A. 5
B. 53
C. 8
D. The line does not compile

4. The expression 3 + 2.0 evaluates to...

A. 5.0, a double
B. nothing; mixing types is an error
C. 3 2.0, a String
D. 5, an int

Lesson 2.3 — Strings and chars

Not every value is a number. The **String** type holds text: zero or more characters wrapped in double quotes, such as “Java” or “Hi, Mom!”. A String variable is declared and assigned exactly like the numeric ones: **String name = "Layla";**. Notice the capital S; String is a class name, and class names start uppercase. Strings are in fact objects, a deep idea saved for Chapter 6, and their built-in text methods arrive in Chapter 8; for now, treat a String as an ordinary value.

Java's favorite text trick is **concatenation**: the + operator glues strings together end to end. “Hello, ” + “world” builds the single string “Hello, world”. More useful still, when one side of + is a String, Java converts the other side to text and glues it on, so **"Score: " + 95** produces “Score: 95”. This is how you label your output. The gluing works left to right, so **"Count: " + 1 + 2** becomes “Count: 12”, not “Count: 3”: after the first +, the whole thing is already one big string.

```
public class ConcatDemo {

    public static void main(String[] args) {
        String name = "Layla";
        int score = 95;

        System.out.println("Hello, " + name);
        System.out.println("Score: " + score);
        System.out.println("2" + 3);     // 23, not 5!
        System.out.println("Count: " + 1 + 2);
    }
}
```

Output: Hello, Layla / Score: 95 / 23 / Count: 12

Look hard at the third line of the example: **"2" + 3** prints 23. The quotes turn 2 from a number into the character pair “2”, and once a String is involved, + means glue, not add. Beginners trip over this constantly, so fix the rule in memory: quotes make text, and text plus anything is longer text. If you truly want 5, keep both sides numeric and glue the label separately.

Some characters are awkward to type directly inside a string: a double quote would end the string, and a line break will not fit on one line of code. Java solves this with **escape sequences**: short codes starting with a backslash that stand for one special character. The everyday set is **\n** (new line), **\t** (tab), **\"** (a double quote), and \\ (a single backslash). This is the book's designated home for escape sequences, and the next program prints one of each.

```
public class EscapeDemo {

    public static void main(String[] args) {
        System.out.println("Line one\nLine two");
        System.out.println("Name:\tLayla");
        System.out.println("She said \"wow\"");
        System.out.println("Path: C:\\Temp");
    }
}
```

Output: two lines (Line one, Line two); Name: Layla with a tab gap; She said "wow"; Path: C:\Temp

For a single character Java offers the **char** type, whose literals use **single** quotes: **char grade = 'A';**. The distinction is strict: 'A' (single quotes) is a char, while “A” (double quotes) is a String one character long, and the two are not interchangeable. A char can also hold an escape sequence such as '\n', which counts as one character. You will use char most often for single letters, digits, and symbols. One more special String: the **empty string**, written as two double quotes with nothing between them, contains zero characters but is a perfectly real value; you will meet it for real in the next lesson.

```
public class CharDemo {

    public static void main(String[] args) {
        char grade = 'A';
        char initial = 'L';

        System.out.println(grade);
        System.out.println(initial);
        System.out.print('!');
    }
}
```

Three lines of output: A, then L, then ! (print leaves the ! without a newline)

> **WARNING**
> 'A' and "A" are different values with different types, and swapping their quotes is a compile error. Single quotes hold exactly one character for a char; double quotes hold any number of characters for a String. Also remember that escape sequences only work inside quotes: typing \n outside a string does nothing useful.

Lesson Check 2.3

Choose the one best answer for each question. Answers are in the Answer Key.

1. What does System.out.println("2" + 3); print?

A. The line does not compile **B.** 2 3
C. 5 **D.** 23

2. Which line stores a single character correctly?

A. char letter = Q; **B.** char letter = 'Q';
C. String letter = 'Q'; **D.** char letter = "Q";

3. How many lines of output does System.out.println("A\nB"); produce?

A. Zero: \n starts a comment **B.** Three lines
C. Two lines: A, then B **D.** One line: A\nB

4. Which escape sequence prints a single backslash?

A. \\ **B.** \"
C. \t **D.** \n

Lesson 2.4 — booleans and Reading Input

The last everyday type answers yes-or-no questions. The **boolean** type holds exactly two values, written in lowercase: **true** and **false**. Declare and assign one like any other variable: **boolean gameOver = false;**. Printing a boolean shows the word true or the word false. Right now you can only store and print booleans; Chapter 3 teaches you how to compute them from comparisons such as age >= 13, which is where they become powerful.

```
public class BooleanDemo {

    public static void main(String[] args) {
        boolean gameOver = false;
        boolean isRaining = true;

        System.out.println(gameOver);   // false
        System.out.println(isRaining);  // true
    }
}
```

boolean variables print as the lowercase words true and false

Every program so far decided everything by itself. Interactive programs ask the **user** for values, and Java's standard tool for reading keyboard input is the **Scanner** class. Scanner does not live in the core that Java loads automatically; it lives in a library drawer called **java.util**. To use it, you must tell the compiler where to look with an **import statement** placed above your class: **import java.util.Scanner;**. The import ends with a semicolon and sits between any comments and the public class line.

Inside main, the line **Scanner in = new Scanner(System.in);** creates a Scanner and stores it in a variable named in. **System.in** is Java's name for the keyboard, and **new** builds a fresh Scanner object wired to it; Chapter 6 explains objects fully, so for now treat that line as a fixed recipe. Once in exists, four methods do the everyday reading: **in.nextInt()** reads a whole number, **in.nextDouble()** reads a decimal, **in.next()** reads one word up to the next space, and **in.nextLine()** reads everything up to the user pressing Enter, spaces included. Each call waits until the user types a value and presses Enter.

```
import java.util.Scanner;

public class Greet {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("What is your name? ");
        String name = in.next();
        System.out.println("Welcome, " + name + "!");
    }
}
```

Answering the prompt with Omar prints: Welcome, Omar!

Notice the pairing pattern in the example: a prompt, then a read. Use **print** (not println) for the prompt so the user's typing lands on the same line, and follow it immediately with the matching read. Every read should have its own prompt, or the program will silently sit there waiting while the user stares at a blank screen. The method you choose must match the value you expect: nextInt for a whole number, nextDouble for a decimal, next for one word, nextLine for a whole line.

```
import java.util.Scanner;

public class SumTwo {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("First number: ");
        int a = in.nextInt();
        System.out.print("Second number: ");
        int b = in.nextInt();

        System.out.println("Sum: " + (a + b));
    }
}
```

The parentheses make the addition happen before the concatenation; Chapter 3 explains exactly why

WARNING

Here is the classic Scanner trap. nextInt (and nextDouble) read the number but leave the invisible Enter keystroke, the newline, still waiting in the input. A nextLine called right afterward reads that leftover newline and hands you an empty string, so the program appears to skip its question. The standard fix is one extra call, **in.nextLine();**, placed immediately after the nextInt to swallow the leftover newline; the next real nextLine then works normally.

```
import java.util.Scanner;

public class InputFlow {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Age in years: ");
        int age = in.nextInt();
        in.nextLine();   // swallow the leftover newline!
        System.out.print("Full name: ");
        String fullName = in.nextLine();

        System.out.println(age + " - " + fullName);
    }
}
```

The extra in.nextLine() makes the full-name read work as expected

TIP

When a program seems frozen, it is usually waiting at a read with no prompt printed. Print a prompt (with print, not println) before every read, and the conversation stays clear for both of you.

NOTE

The import line belongs above the class, outside it, and is needed once per file no matter how many Scanners you create. Forgetting it is one of the most common compile errors of Chapter 2: cannot find symbol: class Scanner.

Lesson Check 2.4

Choose the one best answer for each question. Answers are in the Answer Key.

1. Which values can a boolean variable hold?

A. 0 and 1 only
B. True and False, capitalized
C. true and false, lowercase
D. yes and no

2. What does the line import java.util.Scanner; do?

A. Tells the compiler where to find the Scanner class
B. Creates a Scanner object ready to read
C. Prints the Scanner documentation
D. Reads one line from the keyboard

3. The user should type a full street address, spaces included. Which method reads it?

A. in.nextDouble()
B. in.nextInt()
C. in.next()
D. in.nextLine()

4. int x = in.nextInt(); reads the number 9. What does an immediately following String s = in.nextLine(); store in s?

A. The text 9
B. An empty string
C. It waits forever for another line
D. The number 9

Practice Questions

Work through all ten questions. They are ordered from easiest to hardest. Write your answers on paper before checking the Answer Key.

1. int a = 1; int b = 2; then a = b; then b = a; run. What are the final values?

A. a is 2 and b is 1
B. a is 1 and b is 1
C. a is 2 and b is 2
D. a is 1 and b is 2

2. What does System.out.println(2 + 3 + "!"); print?

A. 5
B. 5!
C. 23!
D. 2 3!

3. Which line correctly stores 7.5 into a new variable?

A. int rate = 7.5;
B. double rate = '7.5';
C. rate = double 7.5;
D. double rate = 7.5;

4. int stars = 6; then stars *= 3; then stars -= 8; run. What is stars?

A. 18
B. 10
C. 2
D. -2

5. A program must read a shoe size like 9.5. Which call fits?

A. in.nextDouble()
B. in.nextInt()
C. in.nextLine()
D. in.next()

6. A variable's value can be replaced later in the program by assigning a new value to it.

[ ] True [ ] False

7. What is the exact output of this program?

```
public class TraceIt {
    public static void main(String[] args) {
        int x = 5;
        x += 3;
        x *= 2;
        System.out.println(x);
        System.out.println("x holds " + x);
    }
}
```

Output:

8. This program does not compile. Find the bug.

```
public class BugTotal {
    public static void main(String[] args) {
        int total;
        System.out.println("Total: " + total);
    }
}
```

Bug:

9. In two or three sentences, explain why descriptive variable names (such as secondsLeft) are worth the extra typing compared with short ones (such as s).

10. Write a complete program called TwoNumbers that asks the user for two whole numbers, reads them with a Scanner, and prints Sum: followed by their total.

Mixed Quiz

Fifteen questions mixing everything from this chapter. Score 12 or more before moving on to the challenges.

1. Which name is NOT a legal variable name?

A. my score
B. myScore2
C. myScore
D. my_score

2. About how large a whole number can an int hold?

A. About 2.1 billion
B. About 21 thousand
C. Any size at all
D. About 2.1 trillion

3. Which line compiles correctly?

A. char letter = "A";
B. string letter = 'A';
C. Char letter = 'A';
D. char letter = 'A';

4. What does System.out.println("Score: " + 7 + 3); print?

A. Score: 7 3
B. Score: 7+3
C. Score: 10
D. Score: 73

5. Which method reads exactly one word, stopping at the first space?

A. in.nextLine()
B. in.next()
C. in.nextWord()
D. in.nextInt()

6. Which prompt statement fits the line int age = in.nextInt(); that follows it?

A. No prompt is needed
B. System.out.println("Your age: ");
C. System.out.print("Your age: ");
D. System.out.print("Your age:\n");

7. boolean isDone = true; then System.out.println(isDone); runs. What appears?

A. isDone
B. true
C. "true"
D. 1

8. int n = 13; then n %= 5; runs. What is n?

A. 0
B. 13
C. 3
D. 2

9. The line String name = 'Kim'; compiles correctly.

[ ] True [ ] False

10. An int can safely hold the value five billion.

[ ] True [ ] False

11. The import for Scanner must appear above the class declaration that uses it.

[ ] True [ ] False

12. What is the exact output of this program?

```
public class QuizTrace {
    public static void main(String[] args) {
        int coins = 10;
        coins += 5;
        coins -= 3;
        System.out.println("Coins: " + coins);
        double share = coins / 4.0;
        System.out.println(share);
    }
}
```

Output:

13. This program does not compile. Find the bug.

```
public class QuizBug {
    public static void main(String[] args) {
        Int score = 10;
        System.out.println(score);
    }
}
```

Bug:

14. A friend writes System.out.println("2" + 2); and expects 4. Explain what actually happens and why.

15. Write a complete program called AskAge that prints the prompt What is your age? (staying on the same line), reads a whole number, and then prints You are N years old. using the typed value.

Challenges

Five harder thinking problems. Each includes a hint; full step-by-step solutions are in the Answer Key. Try the problem seriously before reading the hint.

Challenge 2.1 — Fix the Greeting Machine

The program below should ask for a name and greet the user, but it has three separate bugs. Find them on paper first, then fix them and run the program with your own name.

```
public class GreetFix {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Your name: ")
        string name = in.next();
        System.out.println("Hello, " + name);
    }
}
```

Hint: One bug is at the top of the file, one is at the end of a line, and one is the spelling of a type.

Challenge 2.2 — Swap Machine

Write a program called SwapMachine that creates int variables left = 17 and right = 42, prints them in the order Left: 17 then Right: 42, swaps their values using a third variable, and prints Left: 42 then Right: 17.

Hint: You need a third variable to remember one value while you overwrite it; a direct two-step swap loses a value, as practice question 1 showed.

Challenge 2.3 — Escape Certificate

Write a program called Certificate that prints a three-line award using escape sequences. Line 1: the title Best Beginner wrapped in double quotes. Line 2: the words Earned by, a tab-sized gap, and the name Layla. Line 3: the text Saved to C:\Certs\best.txt with the backslashes visible.

Hint: You need \" twice on line one, \t once on line two, and \\ twice on line three; the line breaks come free from println.

Challenge 2.4 — Profile Card

Write a program called ProfileCard that asks three questions (a one-word name, a whole-number age, and a decimal shoe size), reads each answer with the matching Scanner method, and then prints a card:
Name: N
Age: A years
Shoe size: S

Hint: Match each read to its value: next for the name, nextInt for the age, nextDouble for the shoe size, and prompt with print before every read.

Challenge 2.5 — Human Compiler: Variables

Act like the JVM. Write down the exact output of this program on paper, statement by statement, before running anything. Then type it, run it, and compare with your prediction.

```
public class Trace5 {
    public static void main(String[] args) {
        int a = 4;
        int b = a + 2;
        a += b;
        System.out.println("a: " + a);
        System.out.println("b: " + b);
        System.out.println("a" + a);
    }
}
```

Hint: Track the boxes: b is computed from a's value before a changes; then a += b stores the sum back into a, and the last line glues text onto a number.

Chapter Recap

- A variable is a named box in memory: declare it with a type and a name, assign with = (meaning store), and reassign any time; reading a value never changes it.
- Names (identifiers) may use letters, digits, _, and $, cannot start with a digit, cannot be reserved words, and are case-sensitive; the convention is descriptive camelCase names.
- int holds whole numbers (about -2.1 billion to +2.1 billion) and double holds decimals; mixing an int with a double widens the result to double, and 7 / 2 on two ints drops the fraction (full story in Chapter 3).
- The shortcuts += -= *= /= %= combine an operation with assignment: n += 3 means n = n + 3.
- String values live in double quotes; + concatenates and converts numbers to text ("2" + 3 is "23"); char literals use single quotes for one character; the escape sequences \n \t \" \\ stand for newline, tab, double quote, and backslash.
- boolean holds exactly true or false, lowercase.
- Scanner reads the keyboard: import java.util.Scanner; above the class, Scanner in = new Scanner(System.in); inside main, a prompt printed with print, then a read with nextInt, nextDouble, next,

or nextLine.

- nextInt leaves the Enter keystroke behind: follow it with an extra in.nextLine(); before any real nextLine, or you will read an empty string.

Common Mistakes

Beginners run into these again and again. Read them now to avoid the pain later.

WARNING

Using a variable before giving it a value

A declaration creates the box but stores nothing in it, and reading an empty local variable is a compile error (variable might not have been initialized) that beginners rarely connect to the missing assignment.

Fix: Initialize at the declaration (int total = 0;) or assign a value before the first read.

WARNING

Expecting "2" + 2 to be 4

Quotes turn 2 into text, and a String on either side of + switches the operator into concatenation, so Java glues and prints 22.

Fix: Keep quotes off numbers you intend to add: println(2 + 2) prints 4. Concatenate only the labels: "Sum: " + (2 + 2).

WARNING

Writing Int or string instead of int and String

Type names are spelled exactly, and Java is case-sensitive, so Int and string are unknown names that produce cannot find symbol errors.

Fix: Memorize the exact spellings: int, double, char, and boolean are all lowercase; String starts with a capital S.

WARNING

Forgetting the Scanner import

Scanner lives in the java.util library, which Java does not load automatically, so without the import every use reports cannot find symbol: class Scanner.

Fix: Put import java.util.Scanner; at the top of the file, above the public class line.

WARNING

Reading a line right after nextInt

nextInt consumes the digits but leaves the Enter keystroke, so the following nextLine returns that leftover newline as an empty string and the question seems skipped.

Fix: Add an extra in.nextLine(); immediately after nextInt or nextDouble to swallow the leftover newline.

CHAPTER 3

Operators and Expressions

Numbers are only half the story; the other half is what you do to them. This chapter turns Java into a calculator: you will combine values with operators, discover the surprising behavior of integer division and remainder, and learn the rules that decide which operation happens first. You will also meet casting for converting between types, a small toolbox of Math helpers, and the comparison and logic operators that produce boolean answers. Everything here feeds directly into Chapter 4, where these expressions start making decisions.

In this chapter you will learn to:

- use the arithmetic operators **+ - * / %** with ints, doubles, and Strings
- predict the results of integer division and remainder, including negative numbers
- control evaluation order with precedence rules, parentheses, and casting
- round and transform values with **Math.abs**, **Math.min**, **Math.max**, **Math.sqrt**, **Math.pow**, and **Math.round**
- build boolean answers with **== != > < >= <= && || !** and explain short-circuit evaluation

Lesson 3.1 — Arithmetic Operators

Java's arithmetic operators are the symbols you expect, with two keyboard adaptations: multiplication is written with the asterisk ***** and division with the slash **/**, because keyboards have no times or division signs. An **expression** is a combination of values and operators that Java evaluates down to a single value, so 7 + 3 is an expression whose value is 10. The five workhorse operators are + (add), - (subtract), * (multiply), / (divide), and % (remainder). Wherever a single value could appear, an expression can appear, including inside a println.

```
public class BasicMath {

    public static void main(String[] args) {
        System.out.println(7 + 3);    // 10
        System.out.println(7 - 3);    // 4
        System.out.println(7 * 3);    // 21
        System.out.println(7 / 3);    // 2 (int division)
        System.out.println(7 % 3);    // 1 (remainder)
        System.out.println(7.0 / 2);  // 3.5 (double)
    }
}
```

The five arithmetic operators at work; note the two different divisions

The types of the operands decide the kind of arithmetic. When both sides are ints, Java does integer arithmetic; when **either** side is a double, the other is widened and the whole computation runs in double arithmetic, which is why 7.0 / 2 prints 3.5 while 7 / 3 prints 2. This one rule explains most of the surprises in this chapter: look at the operand types first, and only then at the operator. The % operator, which gives the

remainder after division, works on doubles too (7.5 % 2 is 1.5), though ints are its natural home and the next lesson makes it famous.

The plus sign is the busiest operator in Java because it has two jobs. With two numbers it adds; with a String on either side it concatenates. The operand types decide which job happens, and evaluation runs left to right, so the same symbol can add and glue within one expression. Watch how the parentheses change everything between the first two lines of the next example.

```
public class ConcatMath {

    public static void main(String[] args) {
        System.out.println("Total: " + 2 + 3);
        System.out.println("Total: " + (2 + 3));
        System.out.println(1 + 2 + "3");
    }
}
```

Output: Total: 23 / Total: 5 / 33. Left to right: the first line glues twice, the second adds first, the third adds 1 + 2 before meeting the String

The minus sign also has two jobs. Between two values it subtracts; in front of a single value it is the **unary minus**, which flips the sign: -5, or -balance when balance is a variable. Unary minus computes a new value from the box without changing the box; if you want the flip kept, store it back with balance = -balance. You can even subtract a negative number, which adds its positive twin, as the last line of the example shows.

```
public class SignDemo {

    public static void main(String[] args) {
        int balance = 50;
        System.out.println(-balance);  // -50
        balance = -balance;            // store the flip
        System.out.println(balance);   // -50 now
        System.out.println(3 - -2);     // 5
    }
}
```

Unary minus flips a sign; subtracting a negative adds

TIP

Put spaces around binary operators (a + b, not a+b). The compiler does not care, but human readers parse spaced-out expressions faster and make fewer copying mistakes.

WARNING

There is no implied multiplication in Java. To triple the quantity a + b you must write 3 * (a + b); writing 3(a + b) is a syntax error, because the compiler expects an operator between the 3 and the parenthesis.

Lesson Check 3.1

Choose the one best answer for each question. Answers are in the Answer Key.

1. What does System.out.println(9 % 4); print?

A. 0 **B.** 2

C. 2.25 **D.** 1

2. What is the value and type of 6 / 2.0?

A. 3.0, a double
B. The expression does not compile
C. 6.0, a double
D. 3, an int

3. What does System.out.println("Sum: " + 1 + 2); print?

A. Sum: 1+2
B. Sum: 1 2
C. Sum: 12
D. Sum: 3

4. Which line correctly computes three times the quantity a plus b?

A. 3 * a + b
B. 3 * (a + b)
C. a + b * 3
D. 3(a + b)

Lesson 3.2 — Integer Division and Remainder

Integer division is Java's most famous surprise. When **both operands are ints**, the / operator performs **integer division**: it computes the mathematical quotient and **truncates toward zero**, throwing away the fraction. So 7 / 2 is 3 (not 3.5), 99 / 10 is 9, and 5 / 8 is 0. Nothing is rounded to the nearest value; the fraction is simply dropped, no matter how large. If either operand is a double, ordinary decimal division happens instead, so 7.0 / 2 is 3.5.

WARNING
Do not call integer division floor division. Java truncates toward zero: -7 / 2 is -3, because -3.5 loses only its fraction. A floor operation would give -4 (the largest integer below -3.5), and that is not what Java does here. Truncation toward zero is the guarantee: negative quotients stay closer to zero.

```
public class IntDivDemo {

    public static void main(String[] args) {
        System.out.println(7 / 2);     // 3
        System.out.println(99 / 10);   // 9
        System.out.println(5 / 8);     // 0
        System.out.println(-7 / 2);    // -3 (toward 0)
        System.out.println(7 / -2);    // -3 (same rule)
    }
}
```

Integer division drops the fraction; negative quotients truncate toward zero, not down

The % operator (pronounced mod) answers a different question: after integer division, **what is left over?** 7 % 2 is 1, because 7 is three 2s with 1 left over; 17 % 5 is 2. The sign of the remainder follows the **dividend**, the left operand: -7 % 2 is -1, not 1, and 7 % -2 is 1. In practice you will mostly use % with positive numbers, but the rule explains every result Java prints.

```
public class RemainderDemo {

    public static void main(String[] args) {
        System.out.println(17 % 5);    // 2
        System.out.println(48 % 2);    // 0 (48 is even)
        System.out.println(123 % 10);  // 3 (last digit)
        System.out.println(-7 % 2);    // -1 (of dividend)
        System.out.println(7 % -2);    // 1  (of dividend)
    }
}
```

Remainders, including the sign rule for negatives

Two % patterns appear so often that they are worth memorizing. First, **x % 2** is 0 when x is even and 1 when x is odd (for positive x): the remainder after dividing by 2 can only be 0 or 1. Deciding what to do with that fact needs if-statements, which arrive in Chapter 4; for now you can print the expression and read the 0 or 1 yourself. Second, **x % 10** is the last digit of x, and **x / 10** chops that digit off: 123 % 10 is 3, and 123 / 10 is 12. Dividing and remaining by powers of ten is the standard toolkit for peeling digits off a number.

```
public class ModPatterns {

    public static void main(String[] args) {
        int n = 123;
        System.out.println(n % 2);     // 1 (odd)
        System.out.println(124 % 2);   // 0 (even)
        System.out.println(n % 10);    // 3 (last digit)
        System.out.println(n / 10);    // 12 (digit gone)
    }
}
```

Even/odd and last-digit patterns, printed as 0s, 1s, and digits

Put the pair to work on a classic task: converting a total number of seconds into minutes and leftover seconds. If totalSeconds is 500, then 500 / 60 is 8 (the whole minutes) and 500 % 60 is 20 (the leftover seconds), so 500 seconds is 8 minutes and 20 seconds. The same divide-and-remainder split works for hours and minutes, dollars and cents, feet and inches: any time a bigger unit contains a fixed count of smaller units.

```
public class TimeSplit {

    public static void main(String[] args) {
        int totalSeconds = 500;
        int minutes = totalSeconds / 60;
        int seconds = totalSeconds % 60;

        System.out.println(totalSeconds + " seconds is "
                + minutes + " min " + seconds + " sec");
    }
}
```

Output: 500 seconds is 8 min 20 sec

> **NOTE**
> One boundary to respect: dividing or remaining by zero crashes an int computation at runtime with an ArithmeticException, because division by zero is undefined. 5 / 0 and 5 % 0 both stop the program; keep divisors nonzero until Chapter 9 explains exceptions.

> **TIP**
> Read x % y out loud as x mod y. If you can answer both x / y and x % y for two numbers by hand, integer division holds no more mysteries for you.

Lesson Check 3.2

Choose the one best answer for each question. Answers are in the Answer Key.

1. What does System.out.println(7 / 2); print?

A. 4 **B.** 2
C. 3 **D.** 3.5

2. What does System.out.println(-7 % 2); print?

A. -1 **B.** 1
C. -3 **D.** 0

3. A variable n holds 1234. Which expression gives the last digit, 4?

A. n / 1000 **B.** n / 10
C. n % 100 **D.** n % 10

4. totalSeconds holds 1500. Which pair of expressions gives the minutes and the leftover seconds?

A. 1500 / 100 and 1500 % 100 **B.** 1500 / 60 and 1500 % 60
C. 1500 / 60 and 1500 - 60 **D.** 1500 % 60 and 1500 / 60

Lesson 3.3 — Precedence, Parentheses, and Casting

When an expression mixes operators, Java must decide what to evaluate first, and it always follows fixed **precedence** rules rather than reading blindly left to right. Multiplication, division, and remainder (* / %) bind tighter than addition and subtraction (+ -), so 2 + 3 * 4 is 14, not 20. Operators at the same level group left to right, so 20 - 6 - 3 means (20 - 6) - 3, which is 11. Parentheses outrank everything and force any order you want; good programmers add them even when the rules already agree, because they make the intent visible.

Precedence	Operators	How they group
1 (highest)	( ) parentheses	innermost pair first
2	unary - (sign flip)	applied before * / %
3	* / %	left to right
4 (lowest)	+ -	left to right

```
public class PrecedenceDemo {

    public static void main(String[] args) {
        System.out.println(2 + 3 * 4);     // 14
        System.out.println((2 + 3) * 4);   // 20
        System.out.println(20 - 6 - 3);    // 11
        System.out.println(24 / 2 * 3);    // 36!
        System.out.println(10.0 / 2 / 5);  // 1.0
    }
}
```

*Parentheses override precedence; equal-level operators run left to right (24 / 2 * 3 is 36, not 4)*

The sneakiest precedence bug is assuming every expression reads left to right. In 24 / 2 * 3 the division runs first, because * and / share one level, giving 36; if you wanted 24 / (2 * 3), you must say so with parentheses. A practical habit: when any expression mixes two levels, either add parentheses or run the program and check the printed value against a hand calculation. Both take seconds; shipping the wrong number takes longer.

Sometimes you need to force a conversion by hand, and Java's tool is the **cast**: write the destination type in parentheses in front of the value. The cast **(int)** converts a double to an int by truncating toward zero, exactly like integer division, so (int) 7.9 is 7, not 8, and (int) -7.9 is -7. The cast **(double)** widens an int losslessly: (double) 5 is 5.0. Casts also unlock the classic fix for integer division: (double) a / b casts a to 7.0 before dividing, so (double) 7 / 2 is 3.5.

```
public class CastDemo {

    public static void main(String[] args) {
        double exact = 7.9;
        int whole = (int) exact;   // 7, truncated
        System.out.println(whole);
        System.out.println((int) -7.9);  // -7

        int a = 7;
        int b = 2;
        double half = (double) a / b; // cast first
        System.out.println(half);     // 3.5
    }
}
```

(int) truncates toward zero; (double) before the division keeps the fraction

Casts are only needed for narrowing (double to int). Widening happens automatically in mixed expressions, as Lesson 3.1 showed, so writing (double) there is legal but unnecessary. Because a cast changes only the value it touches, placement matters: (double) a / b casts a before dividing, but (double) (a / b) would perform the integer division first and then widen the already-truncated result, giving 3.0 instead of 3.5. When in doubt, cast the operand, not the whole expression.

Java also ships a library of ready-made number tools in the **Math** class. The everyday ones are **Math.abs(x)** (absolute value), **Math.min(a, b)** and **Math.max(a, b)** (the smaller or larger of two values), **Math.sqrt(x)** (square root), and **Math.pow(a, b)** (a raised to the power b, which always returns a double). Rounding is **Math.round(x)**: the nearest whole number, with halves going up, so Math.round(7.6) is 8 and Math.round(7.4) is 7. One quirk to know: Math.round returns a **long**, a larger whole-number type; printing it shows an ordinary-looking whole number, which is all a beginner needs.

```
public class MathDemo {

    public static void main(String[] args) {
        System.out.println(Math.abs(-9));     // 9
        System.out.println(Math.min(3, 7));   // 3
        System.out.println(Math.max(3, 7));   // 7
        System.out.println(Math.sqrt(25.0)); // 5.0
        System.out.println(Math.pow(2, 10));  // 1024.0
        System.out.println(Math.round(7.6));  // 8
    }
}
```

The everyday Math toolbox; sqrt and pow return doubles, and round's answer prints as a plain whole number

WARNING
(int) truncates; it never rounds. (int) 7.9 is 7, and (int) -7.9 is -7. When you want the nearest whole number, call Math.round instead.

TIP
Precedence works inside method calls too: Math.pow(1 + 1, 2 * 2) evaluates the arguments first, giving Math.pow(2, 4), which is 16.0.

Lesson Check 3.3

Choose the one best answer for each question. Answers are in the Answer Key.

1. What does System.out.println(2 + 3 * 4); print?

A. 9 **B.** 12
C. 14 **D.** 20

2. What is the value of (int) 9.7?

A. 0.7 **B.** 10
C. 9.7 **D.** 9

3. What does System.out.println(Math.round(6.5)); print?

A. 7 **B.** 6.5
C. 6 **D.** 7.0

4. With int a = 7; int b = 2; which expression evaluates to 3.5?

A. a / b **B.** (double) a / b
C. (double) (a / b) **D.** (int) (a / b)

Lesson 3.4 — Comparison and Logical Operators

Expressions can answer questions. The six comparison operators are == (equal to), != (not equal to), **>** (greater than), **<** (less than), **>=** (greater than or equal to), and **<=** (less than or equal to), and each produces a **boolean** value: true or false. For example, 5 > 3 is true and 5 == 3 is false. Comparisons usually run after arithmetic, so x + 1 > 5 parses as (x + 1) > 5: the addition happens first, then the comparison asks its question.

```
public class CompareDemo {

    public static void main(String[] args) {
        int age = 15;
        System.out.println(age >= 13);   // true
        System.out.println(age == 18);   // false
        System.out.println(age != 15);   // false
        System.out.println(age < 100);   // true

        boolean teen = age >= 13;
        System.out.println(teen);        // true
    }
}
```

Each comparison produces a boolean that can be printed or stored

The single most dangerous typo of early Java is writing = when you mean ==. The double equals == **compares** two values and produces a boolean; the single = **stores** a value into a variable. They look alike and do utterly different things. When you read code aloud, say == as “is equal to” and = as “gets”, and the difference will stay loud.

Comparisons combine with three logic operators: **&&** (AND), **||** (OR), and **!** (NOT). A && B is true only when both sides are true; A || B is true when at least one side is true; !A flips true to false and back. These let you phrase range checks and multi-part conditions: age >= 13 && age <= 19 is true exactly when age is in the teenage range. Precedence here runs ! first, then &&, then ||, so when you mix them, add parentheses unless you are sure.

```
public class LogicDemo {

    public static void main(String[] args) {
        int age = 15;
        boolean teen = age >= 13 && age <= 19;
        System.out.println(teen);   // true

        boolean hasTicket = true;
        boolean hasID = false;
        System.out.println(hasTicket && hasID);
        System.out.println(hasTicket || hasID);
        System.out.println(!hasTicket);
        System.out.println(3 > 1 && 5 > 4);
    }
}
```

Output: true / false / true / false / true

Java evaluates && and || **lazily**, a behavior called **short-circuit evaluation**. For &&, if the left side is false the whole expression must be false, so Java never evaluates the right side. For ||, if the left side is true the whole expression must be true, and the right side is skipped again. This is not a random optimization: it lets you put a safety check on the left. When count is 0, the division in count != 0 && total / count > 10 never runs, so no crash occurs.

```
public class ShortCircuit {

    public static void main(String[] args) {
        int count = 0;
        int total = 90;

        // Left side false, so the division never runs.
        boolean high = count != 0 && total / count > 10;
        System.out.println(high);  // false, no crash
    }
}
```

The && safety-check pattern: a false left side protects the right side from ever running

NOTE

Keep two ideas apart. **Short-circuit evaluation** is about skipping the **right side of one expression**: Java decides the answer early and never computes the remaining operand. Choosing between branches, the way if and else-if will work in Chapter 4, is a different mechanism sometimes called first-match branching, and it selects among blocks of statements rather than operands. Both feel like “stopping early”, but one skips an operand and the other picks a block of code.

NOTE

Java also has single & and single |. They compute the same AND and OR answers, but they **always evaluate both sides**, so they cannot protect you the way short-circuit operators do. Everyday code almost always wants && and ||.

WARNING

= compares nothing and == stores nothing. Inside a spot that expects a comparison, a single = is almost always a bug: age = 18 changes age, while age == 18 asks a question. Check for it whenever a boolean expression behaves strangely.

TIP

Name boolean variables like questions: isReady, hasTicket, isEven. Then boolean isReady = temperature > 20; reads like a sentence.

Lesson Check 3.4

Choose the one best answer for each question. Answers are in the Answer Key.

1. What does System.out.println(5 > 3); print?

A. The line does not compile
B. 5
C. false
D. true

2. What does System.out.println(2 == 2 && 3 < 1); print?

A. false
B. true
C. 2
D. It crashes the program

3. Java reads the expression x + 1 > 5. How does it group it?

A. x + 1 with no comparison at all
B. (x + 1) > 5
C. x + (1 > 5)
D. It is a syntax error

4. When is the right side of a || expression evaluated?

A. Always, on every evaluation
B. Never, under any circumstances
C. Only when the left side is false
D. Only when the left side is true

Practice Questions

Work through all ten questions. They are ordered from easiest to hardest. Write your answers on paper before checking the Answer Key.

1. What does System.out.println(17 / 5); print?

A. 3.4
B. 2
C. 3
D. 4

2. What does System.out.println(1 + 2 + "3"); print?

A. 6
B. 33
C. 3
D. 123

3. What is the value of (int) 12.9?

A. 0.9
B. 12
C. 13
D. 12.9

4. score is an int. Which expression is true exactly when score is at least 60 and at most 100?

A. score >= 60 && score < 100
B. score >= 60 || score <= 100
C. 60 <= score <= 100
D. score >= 60 && score <= 100

5. What does Math.sqrt(16.0) return?

A. 4.0, a double
B. 2.0, a double
C. 8.0, a double
D. 4, an int

6. The expression 7 % 2.0 is legal and evaluates to 1.0.

[ ] True [ ] False

7. What is the exact output of this program?

```
public class Trace3 {
    public static void main(String[] args) {
        int total = 47;
        int packs = total / 6;
        int leftover = total % 6;
        System.out.println(packs);
        System.out.println(leftover);
        System.out.println(total + " total");
    }
}
```

Output:

8. This program compiles and runs, but prints the wrong value. Find the bug.

```
public class Bug3 {
    public static void main(String[] args) {
        int result = 7 / 2;
        System.out.println("Half of 7 is " + result);
    }
}
```

Bug:

9. Explain, in two or three sentences, why (a + b) / 2 and a + b / 2 usually compute different things.

10. Write a complete program called SplitTime that asks for a number of seconds (a whole number), reads it with a Scanner, and prints it as minutes and leftover seconds in this exact shape: 1500 seconds is 25 min 0 sec (when the input is 1500).

Mixed Quiz

Fifteen questions mixing everything from this chapter. Score 12 or more before moving on to the challenges.

1. What does System.out.println(10 % 3); print?

A. 0 **B.** 3
C. 1 **D.** 3.33

2. What is the value of 7.0 / 2?

A. 3.0 **B.** 3
C. 4 **D.** 3.5

3. Which line prints Sum: 5?

A. System.out.println("Sum: " + (2 + 3)); **B.** System.out.println("Sum: " + 2 + 3);
C. System.out.println(Sum: + (2 + 3)); **D.** System.out.println("Sum: " + 2 * 3);

4. What is the value of (int) -3.9?

A. 3 **B.** -4
C. -3 **D.** -3.9

5. What does Math.round(2.5) give?

A. 2 **B.** 2.5
C. 0.5 **D.** 3

6. What are the type and value of Math.pow(2, 3)?

A. 6.0, a double **B.** 8.0, a double

C. 8, an int **D.** 9.0, a double

7. What does System.out.println(true && false || true); print?

A. true **B.** Nothing

C. It does not compile **D.** false

8. n is a positive int. The expression n % 2 == 0 is true when...

A. n is odd **B.** n is even

C. n is zero **D.** n is a multiple of 10

9. The ! operator binds tighter than && and ||.

[ ] True [ ] False

10. In an && expression whose left side is false, Java still evaluates the right side.

[ ] True [ ] False

11. Comparison operators such as > bind tighter than the arithmetic operators.

[ ] True [ ] False

12. What is the exact output of this program?

```
public class Quiz3Out {
    public static void main(String[] args) {
        int n = 4567;
        System.out.println(n % 10);
        System.out.println(n / 10 % 10);
        System.out.println(n / 1000);
    }
}
```

Output:

13. This program does not compile. Find the bug.

```
public class Quiz3Bug {
    public static void main(String[] args) {
        int age = 20;
        boolean adult = age = 18;
        System.out.println(adult);
    }
}
```

Bug:

14. In your own words, explain short-circuit evaluation for && and ||, and how it differs from choosing between alternative blocks of code (first-match branching, coming in Chapter 4).

15. Write a complete program called Reverse3 that stores the three-digit number 375 in an int variable, then uses / and % to print its digits reversed: 573.

Challenges

Five harder thinking problems. Each includes a hint; full step-by-step solutions are in the Answer Key. Try the problem seriously before reading the hint.

Challenge 3.1 — Fix the Average Finder

This program compiles and runs, but it prints the wrong average. The true average of 7, 8, and 10 is about 8.33. Trace the types by hand, explain exactly what went wrong, then fix the program so it prints the correct average.

```
public class AverageFix {
    public static void main(String[] args) {
        int a = 7;
        int b = 8;
        int c = 10;
        double average = (a + b + c) / 3;
        System.out.println("Average: " + average);
    }
}
```

Hint: The sum is computed in int arithmetic before the division even starts; look closely at the operands of the / operator.

Challenge 3.2 — Digit Extractor

Write a program called DigitExtractor that stores the four-digit number 4921 in an int variable and prints its digits one per line, in this order: thousands, hundreds, tens, ones (so 4, then 9, then 2, then 1). Use only / and %, with no extra variables.

Hint: Dividing by a power of ten slides digits right; % 10 keeps only the last digit. Combine the two: number / 100 % 10 grabs the hundreds digit.

Challenge 3.3 — Money Split

A vending machine owes a customer 287 cents. Write a program called MoneySplit that prints the change in biggest-first coins, one per line, exactly like this:
Dollars: 2
Quarters: 3
Dimes: 1
Nickels: 0
Pennies: 2

Hint: Take the biggest coin first with /, keep the rest with %, and repeat down the coin ladder: 100, 25, 10, 5.

Challenge 3.4 — Time Machine

Extend the minutes-and-seconds split to a full clock. Write a program called TimeMachine that converts totalSeconds = 7385 into hours, minutes, and seconds, printing exactly: 7385 seconds is 2 h 3 m 5 s

Hint: There are 3600 seconds in an hour. Divide and take the remainder twice: first by 3600 for hours, then by 60 for minutes.

Challenge 3.5 — Digit Sum

Numerologists add the digits of a number: 4921 becomes 4 + 9 + 2 + 1 = 16. Write a program called DigitSum that stores 4921 in an int variable, computes the sum of its four digits with / and %, and prints exactly: Digit sum: 16

Hint: You already know how to pull out each digit (see DigitExtractor); add the four expressions together into one int sum.

Chapter Recap

- Arithmetic operators are + - * / %; the operand types decide the math: int with int gives int results, and any double operand widens the whole expression to double.
- + has two jobs: numeric addition, or concatenation when a String operand appears; evaluation is left to right, so "Total: " + 2 + 3 is Total: 23 but "Total: " + (2 + 3) is Total: 5.
- int / int truncates toward zero (7 / 2 is 3, and -7 / 2 is -3); it drops the fraction and never rounds, so it is not floor division.
- % gives the remainder: 7 % 2 is 1; the sign follows the dividend (-7 % 2 is -1); x % 2 is 0 for even and 1 for odd (positive) values, and x % 10 grabs the last digit.
- Precedence: parentheses first, then * / % (left to right), then + - (left to right); when in doubt, add parentheses.
- (int) casts truncate toward zero ((int) 7.9 is 7); (double) widens losslessly; Math.round gives the nearest whole number with halves up; Math.pow and Math.sqrt return doubles; Math.abs, Math.min, and Math.max round out the toolbox.
- Comparisons == != > < >= <= produce booleans; combine them with && (both), || (either), and ! (flip); ! binds tighter than &&, which binds tighter than ||; arithmetic runs before comparisons.
- && and || short-circuit: a decided left side skips the right operand. That skips an operand, which is different from first-match branching between code blocks in Chapter 4.

Common Mistakes

Beginners run into these again and again. Read them now to avoid the pain later.

> **WARNING**
> **Expecting 7 / 2 to be 3.5**
> With two int operands, / means integer division and truncates toward zero, so the program quietly uses 3 where you expected 3.5 and every later result drifts.
> **Fix:** Make one operand a double: 7.0 / 2, or cast: (double) 7 / 2. Cast the operand, not the whole already-truncated expression.

> **WARNING**
> **Treating (int) as rounding**
> (int) 7.9 is 7 and (int) -7.9 is -7: the cast truncates toward zero, so values that should land on 8 or -8 end up one short.
> **Fix:** Use Math.round when you want the nearest whole number (halves go up), and reserve (int) for when dropping the fraction is truly what you mean.

WARNING

Writing = where == is meant

= stores a value and == asks a question; a stray = inside a comparison overwrites the variable and, in a boolean spot, becomes a type error such as assigning an int to a boolean.

Fix: Read == as “is equal to” and = as “gets” whenever you scan code; check every comparison spot for a doubled equals sign.

WARNING

Forgetting that the % sign follows the dividend

Programmers who expect a nonnegative remainder read -7 % 2 as 1, but Java prints -1, because the remainder takes the sign of the left operand.

Fix: Remember the rule (sign of the dividend) whenever negatives can reach a %, and test those cases on paper first.

WARNING

Writing 2(x + 1) or other implied multiplication

Java needs an explicit operator between a number and a parenthesis, so 2(x + 1) is a syntax error, not twice the quantity.

Fix: Write every operator: 2 * (x + 1). Scan translated math formulas for missing * symbols.

MINI-PROJECT 1

Cafe Order Calculator

Time to build something you could actually use. This program greets a customer, reads how many coffees and muffins they want, works out the bill with sales tax, and prints a tidy receipt. It is a straight-line program: no decisions, no loops, just input, arithmetic, and output, exactly the skills from Chapters 1 to 3.

Concepts used: variables, int and double, Scanner input, arithmetic operators, casting, precedence, string concatenation

Step 1 — The Skeleton and the Input

Start with the class, the main method, and a Scanner for keyboard input. We read quantities first (whole numbers) so nextInt is the right tool for both. The prompt uses print instead of println so the user's typing lands on the same line as the question.

```
import java.util.Scanner;

public class CafeOrder {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);

        System.out.println("Welcome to Bean Corner!");
        System.out.print("How many coffees? ");
        int coffees = in.nextInt();
        System.out.print("How many muffins? ");
        int muffins = in.nextInt();
```

Step 2 — The Arithmetic

Prices are money, so they belong in double variables. The subtotal multiplies each quantity by its price; int * double is a mixed expression, and Java widens the int to a double automatically, so no cast is needed here. The tax rate is stored in its own named constant-style variable so it is easy to find and change later.

```
        double coffeePrice = 2.75;
        double muffinPrice = 1.90;
        double TAX_RATE = 0.08;

        double subtotal = coffees * coffeePrice
                + muffins * muffinPrice;
        double tax = subtotal * TAX_RATE;
        double total = subtotal + tax;
```

Step 3 — The Receipt

Finally the program prints the receipt with string concatenation. Math.round(total * 100) / 100.0 rounds the money to two decimal places the classic way: shift the decimal two places right, round to a whole number, then shift back. Math.round returns a long, and dividing a long by the double 100.0 widens it back to double, so the arithmetic is safe.

```
        double rounded = Math.round(total * 100)
                / 100.0;

        System.out.println("---------- YOUR RECEIPT ----------");
        System.out.println(coffees + " coffees x $" + coffeePrice);
        System.out.println(muffins + " muffins x $" + muffinPrice);
        System.out.println("Subtotal: $" + subtotal);
        System.out.println("Tax: $" + tax);
        System.out.println("TOTAL: $" + rounded);
        System.out.println("Thank you, come again!");
    }
}
```

Sample run: user types 2 then 3, and sees a receipt with subtotal $11.2, tax $0.89..., total $12.09

How It Works

The program follows a strict pipeline: read, compute, print. The two nextInt calls pause the program and wait for the user to type numbers, and those numbers land in int variables because quantities are always whole. Every calculation after that point uses the ordinary operators from Chapter 3, and because prices are doubles, Java widens every int it meets along the way without complaint.

The most delicate line is the rounding of the total. Money should display two decimal places, but raw double arithmetic often produces long tails such as 12.086. Multiplying by 100 moves the cents into the whole-number part, Math.round snaps it to the nearest whole number, and dividing by 100.0 restores the decimal point. It is an old programmer's trick that you will meet again in Chapter 8 in a more elegant form.

Precedence quietly does the right thing throughout: coffees * coffeePrice + muffins * muffinPrice groups the two multiplications before the addition, exactly as math class promised. Where the grouping might be unclear to a human reader, the code splits the expression across two lines so the intent is obvious.

Things to Try

- Add a third product (for example, hot chocolate at $2.25) and extend the subtotal line to include it.
- Give a 10 percent discount when the subtotal is large. You will need Chapter 4's if statement, so come back after it!
- Print how many muffins could be bought with the exact total using integer division and remainder on the cents.
- Make the program ask for the customer's name (nextLine) and put it in the receipt header.
- Change TAX_RATE to your local sales tax and rerun. Notice how one well-named variable updates the whole receipt.

CHAPTER 4

Making Decisions

So far every program you have written has run the same lines in the same order, every single time. Real programs are smarter than that: they look at the situation and choose what to do, the way you choose a jacket based on the weather. In this chapter you will learn the Java tools for making choices: the if statement, the if/else pair, else-if chains, nested decisions, and the switch statement. By the end, your programs will respond differently to different input, which is the first step toward software that feels intelligent.

In this chapter you will learn to:

- write if statements with boolean conditions and brace-enclosed bodies
- build if/else and else-if chains, and predict which branch runs first
- use nested ifs, && ranges, and boolean flags to express rules clearly
- write switch statements with case, break, and default, and explain fall-through
- trace and fix the classic decision bugs: = versus ==, missing braces, and missing break

Lesson 4.1 — The if Statement

Programs become interesting the moment they can react differently to different situations. An if statement is Java's basic tool for this: it asks a yes-or-no question and runs a piece of code only when the answer is true. The question is called the condition, it must be a boolean expression such as score >= 60 or age != 0, and it always sits in parentheses right after the word if. The code that may or may not run is called the body, and it lives inside a pair of curly braces.

```
public class PassCheck {

    public static void main(String[] args) {
        int score = 72;
        if (score >= 60) {
            System.out.println("You passed!");
        }
        if (score == 100) {
            System.out.println("Perfect score!");
        }
        System.out.println("Done checking");
    }
}
```

With score 72 this prints You passed! and then Done checking. Change 72 to 55 and only Done checking prints.

Trace the program above by hand. The first condition, score >= 60, is true because 72 really is at least 60, so Java runs the body and prints You passed!. The second condition, score == 100, is false, so Java skips that whole brace-enclosed body exactly as if it were not there. Execution then continues with the statement after the closing brace, which prints Done checking either way. This skip-when-false behavior is the entire secret of the if statement.

Notice three formatting habits in the example. The condition is wrapped in parentheses, the body is wrapped in braces even though it holds only one statement, and the body is indented by four spaces so your eyes can see what belongs to the if. The braces matter more than the indentation: they mark exactly where the body starts and ends. Indentation is for humans, but it is a habit you should never skip, because code is read far more often than it is written.

WARNING
Without braces, an if owns only the NEXT single statement. Everything after that first statement runs unconditionally, no matter how far it is indented. The compiler ignores indentation completely, so code that looks related may not be related at all.

```
int temp = 15;
if (temp > 30)
    System.out.println("It is hot today");
    System.out.println("Drink water");   // always runs!
```

The second println is NOT inside the if: it runs even when temp is 15.

NOTE
The condition must be a real boolean. A line like if (score = 60) with a single equals sign tries to store 60 into score, and because 60 is a number and not true or false, the compiler rejects it. Java, unlike some older languages, never treats 0 as false or nonzero as true, which quietly protects you from a whole family of bugs.

TIP
Read every if out loud in this rhythm: if the condition is true, do everything between the braces; otherwise skip straight past them. Saying it this way while tracing on paper prepares you for every exercise in this chapter. Tracing small programs by hand is the cheapest debugging tool you will ever own.

Lesson Check 4.1

Choose the one best answer for each question. Answers are in the Answer Key.

1. What happens to the body of an if when the condition is false?

A. It runs once before the program continues
B. The program stops with an error
C. It is skipped and the program continues after the if
D. It runs anyway but prints nothing

2. What kind of value must the expression inside if (...) have?

A. double - positive means true
B. boolean - true or false
C. String - the word yes or no
D. int - zero means false

3. int x = 5; if (x > 10) { System.out.println("big"); } System.out.println("end"); What prints?

A. Nothing at all
B. big
C. big, then end
D. end

4. Which statement about braces around an if body is correct?

A. Without braces, only the first statement after if belongs to it
B. Braces are decoration and can never change behavior
C. An if body must hold at least two statements
D. Java adds missing braces automatically

Lesson 4.2 — else and else-if Chains

One question with two possible actions needs the else keyword. An if/else statement looks like an if with a second brace-enclosed body attached, and exactly one of the two bodies runs: the if body when the condition is true, the else body when it is false. The else keyword takes no condition of its own; it simply means otherwise. This either-or shape is called a two-way branch, and it appears in some form in nearly every program you will ever write.

```
import java.util.Scanner;

public class AgeCheck {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter your age: ");
        int age = in.nextInt();
        if (age >= 18) {
            System.out.println("You are an adult.");
        } else {
            System.out.println("You are a minor.");
        }
    }
}
```

Sample run: the user types 15 and the program prints You are a minor.

Many decisions have more than two outcomes, and stacking else after if builds an else-if chain. Java tests the conditions from top to bottom and, at the FIRST one that is true, runs that branch and skips all the remaining branches. We will call this first-match branching: the first matching condition wins and the chain is finished for this value. The trailing else, when present, runs only when nothing above it matched, like a catch-all drawer. Order matters enormously, as the next two programs show.

```
public class GradeReport {

    public static void main(String[] args) {
        int score = 93;
        if (score >= 90) {
            System.out.println("Grade: A");
        } else if (score >= 80) {
            System.out.println("Grade: B");
        } else if (score >= 70) {
            System.out.println("Grade: C");
        } else if (score >= 60) {
            System.out.println("Grade: D");
        } else {
            System.out.println("Grade: F");
        }
    }
}
```

With score 93 the first branch matches immediately and prints Grade: A.

Now watch what happens when the same conditions are written in the wrong order. Because score >= 60 is tested first, a score of 93 matches it immediately, prints the D-or-better message, and never reaches the A branch at all. That second branch has become unreachable: no score of 90 or more can get past a condition

that already accepts 60 and up. Debugging tip: when a chain gives answers that are too general, read the conditions top to bottom and ask which one fires first.

```
public class WrongOrder {

    public static void main(String[] args) {
        int score = 93;
        if (score >= 60) {
            System.out.println("Grade: D or better");
        } else if (score >= 90) {
            System.out.println("Grade: A");   // never reached
        }
    }
}
```

93 matches the first condition, so the A branch can never run.

The final else is the chain's safety net. If every condition above it fails, the else body runs, guaranteeing the program responds one way or another. Without an else, a chain in which no condition matches simply does nothing, which is sometimes what you want and sometimes a silent bug. When in doubt, end the chain with an else that handles impossible or unexpected values.

NOTE

Chapter 3 taught you that && and || short-circuit: inside one boolean expression, the right operand is skipped when the left operand already settles the answer. First-match branching is a different mechanism one level higher: an if / else if / else chain picks exactly one branch, the first condition that is true, and never even tests the rest. Short-circuit evaluation skips part of a single expression, while first-match branching chooses one alternative out of many. Both involve skipping work, but they act in different places: one inside an expression, one across a chain of statements.

TIP

Order chains from the most specific condition to the most general. For score ranges that means the highest threshold first; for special cases it means the unusual case before the everyday one. The final else then catches whatever is left.

Lesson Check 4.2

Choose the one best answer for each question. Answers are in the Answer Key.

1. In an if/else statement, how many of the two bodies run?

A. Always both
B. Exactly one
C. Neither, unless a condition repeats
D. Zero or two

2. In an else-if chain, which branch runs?

A. The first one, from the top, whose condition is true
B. Every branch whose condition is true
C. The branch closest to the else
D. The last one whose condition is true

3. A chain tests score >= 90 (A), then score >= 80 (B), then score >= 70 (C), then else (D). What prints for score = 85?

A. Grade: D
B. Grade: C
C. Grade: B
D. Grade: A

4. Which sentence describes the difference between first-match branching and short-circuit evaluation?

A. Short-circuit picks branches; first-match skips operands
B. They are two names for the same mechanism
C. They act in different places: chain vs expression
D. First-match picks a branch; short-circuit skips an operand

Lesson 4.3 — Nested ifs and Common Patterns

Sometimes one decision depends on another: first check whether the visitor has a ticket, and only then check whether they are tall enough for the ride. Placing an if inside another if is called nesting, and Java allows it to any depth. The inner if runs only when the outer condition is true; if the outer condition is false, Java skips the outer body, inner if included. Indentation shows the levels: each nested if sits one step further inside its parent's braces.

```
public class RideCheck {

    public static void main(String[] args) {
        boolean hasTicket = true;
        int heightCm = 125;
        if (hasTicket) {
            if (heightCm >= 120) {
                System.out.println("Board the roller coaster!");
            } else {
                System.out.println("Too short for this ride.");
            }
        } else {
            System.out.println("Buy a ticket first.");
        }
    }
}
```

With these values the outer if passes and the inner one checks the height.

Many range checks can be written either as nesting or as a single condition joined with &&. The condition score >= 80 && score <= 89 is true exactly when both parts are true, which is precisely the range 80 through 89. Inside an else-if chain you could drop the upper bound, because reaching that branch already means the earlier conditions failed; in a standalone if, however, both bounds are required. Choose whichever version reads more clearly, but learn to read both.

```
public class RangeCheck {

    public static void main(String[] args) {
        int score = 84;
        if (score >= 80 && score <= 89) {
            System.out.println("Score is in the 80s.");
        } else {
            System.out.println("Score is outside the 80s.");
        }
    }
}
```

Both bounds sit in one condition, so this if works anywhere, chain or not.

A boolean variable that remembers the result of a test is called a flag. Writing boolean passed = score >= 60; stores the yes-or-no answer once, and the flag can then be used in several conditions below. Flags also make code read like English: if (hasTicket), if (!isRaining), while (!done). Because a flag already holds true or false, comparing it to true or false adds nothing, as the table shows.

You write	Same as	Why
if (isRaining == true)	if (isRaining)	the flag already is the answer
if (isRaining == false)	if (!isRaining)	! flips true to false and back
if (isRaining != true)	if (!isRaining)	not equal to true means false
if (isRaining != false)	if (isRaining)	not equal to false means true

WARNING

When braces are missing around a nested if, an else belongs to the NEAREST if that does not already have one, regardless of indentation. The compiler pairs them up by structure, not by how far the lines are indented. Always brace both levels of a nested if so the code shows what you mean.

```
public class DanglingElse {

    public static void main(String[] args) {
        int age = 16;
        boolean withAdult = false;
        if (age >= 18)
            if (withAdult)
                System.out.println("Allowed");
        else
            System.out.println("Denied");
    }
}
```

Indentation lies: the else pairs with the inner if, so age 16 prints nothing at all.

Run the DanglingElse program and it prints nothing, which surprises almost everyone. The indentation suggests the else belongs to the age check, but Java actually pairs it with the inner withAdult check, so a 16-year-old falls through both conditions in silence. Add braces around the outer body and the meaning changes to match the indentation. This is why experienced programmers brace every if, even one-statement ones.

TIP

Name flags so they read as facts: hasTicket, isRaining, passed, gameOver. A condition like if (gameOver) then needs no comment, because the variable name already tells the story. Avoid names such as flag1 or check2, which force every reader to hunt for the meaning.

Lesson Check 4.3

Choose the one best answer for each question. Answers are in the Answer Key.

1. When braces do not show which if an else belongs to, Java pairs the else with...

A. nothing; it reports a syntax error
B. the first if in the file
C. the outermost if
D. the nearest if that has no else yet

2. int score = 84; What is the value of score >= 80 && score <= 89?

A. true
B. It will not compile
C. It depends on other variables
D. false

3. boolean isRaining = true; Which if prints Take an umbrella?

A. if (isRaining != true)
B. if (isRaining)
C. if (!isRaining)
D. if (isRaining == false)

4. In the DanglingElse program (age = 16, withAdult = false), what appears on screen?

A. Denied
B. An error message
C. Nothing at all
D. Allowed

Lesson 4.4 — The switch Statement

An else-if chain works for any conditions, but when you are comparing ONE variable against a series of fixed values, Java offers a tidier tool: the switch statement. A switch jumps directly to the case whose label equals the value in parentheses, runs from there, and finishes at a break. So far in this book the switch value can be an int or a char; a String switch also exists, but comparing strings correctly needs a method you will meet in Chapter 8. Think of a switch as a labeled elevator panel: press the button that matches your floor and you go straight there.

```
public class MenuPick {

    public static void main(String[] args) {
        int choice = 2;
        switch (choice) {
            case 1:
                System.out.println("Vanilla");
                break;
            case 2:
                System.out.println("Chocolate");
                break;
            case 3:
                System.out.println("Strawberry");
                break;
            default:
                System.out.println("Unknown choice");
        }
    }
}
```

choice is 2, so execution jumps straight to case 2 and prints Chocolate.

Each case label must be a constant: a literal such as 3 or 'B' whose value is fixed before the program runs. You may not use a variable or a calculation as a label. The colon ends each label, and the statements after it form that case's body. The break statement jumps out of the whole switch the moment it runs, which is what stops one case from bleeding into the next, and the default section runs when no label matches.

```
public class FallThrough {

    public static void main(String[] args) {
        int level = 1;
        switch (level) {
            case 1:
                System.out.println("Easy");
            case 2:
                System.out.println("Medium");
                break;
            case 3:
                System.out.println("Hard");
                break;
        }
    }
}
```

No break after Easy, so execution falls into case 2: this prints Easy AND Medium.

When a case has no break, execution falls through into the next case's body, and that is usually an accident. But fall-through has an honest use: stacking several labels in front of one shared body groups values together. In Weekday below, days 1 through 5 share the Weekday message, and 6 and 7 share the Weekend message. One body, many labels, a single break.

```
public class Weekday {

    public static void main(String[] args) {
        int day = 6;
        switch (day) {
            case 1:
            case 2:
            case 3:
            case 4:
            case 5:
                System.out.println("Weekday");
                break;
            case 6:
            case 7:
                System.out.println("Weekend");
                break;
            default:
                System.out.println("Invalid day");
        }
    }
}
```

day is 6, so the run lands on the shared Weekend body.

```
public class GradePoints {

    public static void main(String[] args) {
        char grade = 'B';
        switch (grade) {
            case 'A':
                System.out.println("4 points");
                break;
            case 'B':
                System.out.println("3 points");
                break;
            case 'C':
                System.out.println("2 points");
                break;
            default:
                System.out.println("0 points");
        }
    }
}
```

Char labels use single quotes; grade B prints 3 points.

WARNING

The missing break is the most common switch bug in student code, and it rarely causes a compile error: the program just prints too much. Trace FallThrough again and predict its exact output before you run it. Make it a reflex to end every case with break unless you stacked labels on purpose.

NOTE

One day you will want to switch on words, such as a command typed by the user. String switches do exist, but string comparison has a trap called .equals that we resolve properly in Chapter 8. Until then, switch on ints and chars only.

NOTE

Modern Java also offers an arrow form, written like case 1 ->, which runs one body and needs no break. This book sticks with the classic case / break style because it appears in millions of existing programs. If you meet arrows in other material, you already understand the idea they simplify.

TIP

Put default last even when every case seems covered. It costs one line and turns silent surprises or future edits into visible, predictable behavior. For a switch on days 1 to 7, a default like Invalid day makes impossible values announce themselves.

Lesson Check 4.4

Choose the one best answer for each question. Answers are in the Answer Key.

1. What happens if a case's body has no break?

A. Execution falls through into the next case's body

B. The compiler reports an error

C. The switch is skipped entirely

D. The default case runs automatically

2. Which of these can be used directly as a case label today?

A. case n + 1:

B. case total:

C. case 3:

D. case score:

3. What does the default section do?

A. Runs every time, like a footer

B. Runs when no case matches the value

C. Runs before the matching case

D. Ends the whole program

4. A switch runs on a char variable named grade. Which case label is written correctly?

A. case A:

B. case "A":

C. case (A):

D. case 'A':

Practice Questions

Work through all ten questions. They are ordered from easiest to hardest. Write your answers on paper before checking the Answer Key.

1. int temp = 31; if (temp > 30) { System.out.println("Hot"); } System.out.println("Bye"); What prints?

A. Hot, then Bye on the next line

B. Bye only

C. Hot only

D. Nothing at all

2. In an else-if chain, how many branches can run for one value?

A. Exactly one

B. At least two

C. None, unless there is an else

D. All whose conditions are true

3. Which condition is true exactly when n is between 1 and 10 inclusive?

A. n >= 1 || n <= 10

B. n >= 1 && n <= 10

C. n > 0 && n < 10

D. 1 <= n <= 10

4. int n = 7; if (n = 10) { ... } What does the compiler say?

A. Java swaps it for n == 10 automatically
B. The body never runs for any n
C. The body runs only when n is 10
D. Compile error: the condition is not a boolean

5. In the FallThrough program (level = 1, no break after Easy), what prints?

A. Easy, Medium, and Hard
B. A compile error
C. Easy, then Medium
D. Easy only

6. When several conditions in an else-if chain are true at the same time, Java runs every matching branch.

[ ] True [ ] False

7. What is the exact output of this program?

```
public class ScoreCheck {
    public static void main(String[] args) {
        int score = 78;
        if (score >= 90) {
            System.out.println("Excellent");
        } else if (score >= 70) {
            System.out.println("Good");
        } else {
            System.out.println("Keep trying");
        }
        System.out.println("Score: " + score);
    }
}
```

Output:

8. This program compiles but prints Adult even though age is 12. Find the bug.

```
public class SemicolonBug {
    public static void main(String[] args) {
        int age = 12;
        if (age >= 18);
        {
            System.out.println("Adult");
        }
    }
}
```

Bug:

9. Why does the ORDER of conditions matter in an else-if chain? Answer in two or three sentences using a grading example.

10. Write a complete program called EvenOdd that reads an integer with a Scanner and prints Even when it is divisible by 2 and Odd otherwise. Use if/else and the % operator.

Mixed Quiz

Fifteen questions mixing everything from this chapter. Score 12 or more before moving on to the challenges.

1. Which if header is written correctly?

A. if (score >= 60);
B. if score >= 60
C. if { score >= 60 }
D. if (score >= 60)

2. A chain checks: if (age < 13) prints child; else if (age < 20) prints teen; else prints adult. What prints for age = 12?

A. child
B. teen
C. adult
D. child and teen

3. Which condition is true exactly when n is OUTSIDE the range 1 to 10?

A. n < 1 && n > 10
B. n <= 1 || n >= 10
C. n < 1 || n > 10
D. !(n >= 1) && !(n <= 10)

4. What is the usual job of break inside a case?

A. To leave the switch after the matching case
B. To jump back to the first case
C. To separate labels from their bodies
D. To end the whole program

5. case 1: case 2: System.out.println("Early"); break; For day = 2, what prints?

A. Nothing; case 2 has an empty body
B. Early twice
C. Early
D. A compile error

6. boolean started = false; Which condition is equivalent to !started?

A. started != false
B. started == false
C. started
D. started == true

7. An outer if checks x > 0, and its only content is an inner if checking x < 10. Which single condition replaces the nesting?

A. x > 0 || x < 10
B. x > 0 && x < 10
C. !(x > 0) && x < 10
D. x > 0 and x < 10

8. Without braces: if (a) if (b) one(); else two(); When does two() run?

A. Whenever a or b is false
B. Never; the code will not compile
C. When a is false
D. When a is true and b is false

9. An else branch needs its own condition in parentheses.

[ ] True [ ] False

10. A switch compares its value against the case labels in top-to-bottom order, exactly like an else-if chain.

[ ] True [ ] False

11. In Java, the line if (0) { } is legal and its body never runs.

[ ] True [ ] False

12. What is the exact output of this program?

```
public class TraceSwitch {
    public static void main(String[] args) {
        int code = 3;
        switch (code) {
            case 1:
                System.out.println("One");
                break;
            case 2:
                System.out.println("Two");
            case 3:
                System.out.println("Three");
            case 4:
                System.out.println("Four");
                break;
            default:
                System.out.println("Many");
        }
    }
}
```

Output:

13. Both lines should print ONLY when temp is above 25. With temp = 10 the second line still prints. Find the bug.

```
public class BraceBug {
    public static void main(String[] args) {
        int temp = 10;
        if (temp > 25)
            System.out.println("Hot outside");
            System.out.println("Drink lots of water");
    }
}
```

Bug:

14. What is fall-through in a switch? Name one bug it causes and one way programmers use it on purpose.

15. Write a complete program called PassFail that reads an integer score with a Scanner and prints Pass when the score is 60 or more, and Fail otherwise.

Challenges

Five harder thinking problems. Each includes a hint; full step-by-step solutions are in the Answer Key. Try the problem seriously before reading the hint.

Challenge 4.1 — Grade Classifier with Input

Write a program that asks for a score from 0 to 100 and prints the letter grade: 90 or more is A, 80 to 89 is B, 70 to 79 is C, 60 to 69 is D, and below 60 is F. If the user types a number outside 0 to 100, print Invalid score instead of a grade. Use one else-if chain.

Hint: Handle the invalid case FIRST, then order the grade checks from the highest threshold down.

Challenge 4.2 — Leap Year Checker (Fix and Finish)

The starter below is broken twice over: it does not compile, and even after fixing that, it gives wrong answers for some years. A leap year is divisible by 4, EXCEPT years divisible by 100, UNLESS they are also divisible by 400. So 2024 is a leap year, 1900 is not, and 2000 is. Fix the code and correct the logic so it prints one clear line for every year.

```
import java.util.Scanner;

public class LeapYear {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter a year: ");
        int year = in.nextInt();
        if (year % 4 = 0) {
            System.out.println("Leap year");
        }
        if (year % 4 != 0) {
            System.out.println("Not a leap year");
        }
    }
}
```

Hint: You need the = versus == fix, then a condition that combines the three rules with && and ||, plus one if/else instead of two ifs.

Challenge 4.3 — Rock, Paper, Scissors (Two Players)

Two players each enter 1 for rock, 2 for paper, or 3 for scissors. Rock beats scissors, scissors beats paper, and paper beats rock; equal choices are a tie. Read both choices and print Player 1 wins!, Player 2 wins!, or It is a tie!. If either number is not 1, 2, or 3, print Invalid choice and decide nothing else.

Hint: Check validity first. For the winner, remember that && binds more tightly than ||, and wrap each pairing in parentheses.

Challenge 4.4 — Triangle Type Classifier

Read three whole numbers that are the side lengths of a triangle. If any side is zero or negative, print Invalid sides. Otherwise print Equilateral when all three sides are equal, Isosceles when exactly two sides are equal, and Scalene when all three are different.

Hint: Check the all-equal case before the two-equal case: order in the chain decides which branch a triangle with sides 5, 5, 5 reaches.

Challenge 4.5 — BMI Category Calculator

Read the user's weight in kilograms and height in meters as doubles, compute the BMI as weight divided by height squared (use Math.pow(height, 2)), and print the value. Then classify it: below 18.5 prints Underweight, from 18.5 up to but not including 25 prints Normal weight, from 25 up to but not including 30 prints Overweight, and 30 or more prints Obese. The BMI will print with many decimal places; that is fine for now.

Hint: Compute the BMI once into a double variable, then build one else-if chain. Do not recompute the formula in every branch.

Chapter Recap

- An if runs its brace-enclosed body only when its boolean condition is true; otherwise the body is skipped entirely.
- if/else picks exactly one of two bodies; else takes no condition and usually comes last as a safety net.
- In an else-if chain the FIRST true branch wins and the rest are never tested, so order branches from most specific to most general.
- First-match branching picks one branch among alternatives; short-circuit evaluation (from Chapter 3) skips one operand inside a single expression. Different levels, both useful.
- Nested ifs handle decisions that depend on other decisions; a range can also be one condition joined with &&, such as score in the 80s.
- Store a comparison in a boolean flag, then if (flag) is enough: never compare a boolean to true or false.
- Without braces, an if owns only its next statement and an else pairs with the nearest if; brace everything and indentation stops lying.
- switch jumps to a matching constant case label (int or char for now); break leaves the switch; missing break means fall-through (deliberate when labels are stacked, a bug otherwise); default catches leftovers.

Common Mistakes

Beginners run into these again and again. Read them now to avoid the pain later.

> **WARNING**
> **Using = instead of == in a condition**
> if (x = 5) assigns instead of comparing. With an int like x the compiler rejects the line, but with a boolean flag, flag = true compiles and always makes the condition true - a silent logic bug.
> **Fix:** Read == aloud as is equal to and save = for assignments. When the compiler reports incompatible types: int cannot be converted to boolean, look for a stray single =.

> **WARNING**

Forgetting the braces around a multi-statement body
An unbraced if or else owns only the first statement below it; the remaining statements run unconditionally, and the indentation lies about what belongs where.
Fix: Brace every if, else, and else-if body, even when it holds a single statement. The braces cost two characters and prevent a whole family of silent bugs.

WARNING
Wrong order in an else-if chain
A broad condition such as score >= 60 placed before score >= 90 catches every high score first, making the later branches unreachable and the answers too general.
Fix: Order chains from most specific to most general, with the final else as the catch-all. Trace one high value through the chain to confirm the right branch fires.

WARNING
Missing break in a switch case
Without break, execution falls through into the next case and prints its output too; the compiler reports nothing, so the program looks fine until you read the output carefully.
Fix: End every case with break unless you deliberately stacked labels in front of one shared body. Predict the output of each case on paper before running.

WARNING
A semicolon right after the if header
if (age >= 18); ends the statement with an empty body, so the if controls nothing and the block below runs every single time. The compiler accepts it, which makes the bug invisible at compile time.
Fix: Never put a semicolon between an if header and its opening brace. If a block seems to run always, check for a stray ; right after the condition.

CHAPTER 5

Loops

Chapter 4 gave your programs the ability to choose; this chapter gives them the ability to repeat. Instead of writing the same println ten times, you will write one loop and let Java do the repetition for you. You will meet the three loop forms Java offers - while, for, and do-while - and learn when each one is the natural choice. You will also learn to steer loops with break and continue, to build loops that read input until a special value appears, and to nest loops inside each other. Loops are where most beginners first feel real programming power, and also where the famous infinite loop lives, so we will handle both.

In this chapter you will learn to:

- write while loops whose condition is checked before every pass
- write for loops with init, condition, and update, and know the loop variable's scope
- use do-while for actions that must happen at least once, and build re-prompting validation loops
- control loops with break and continue, and read input until a sentinel value
- trace nested loops and predict their total number of passes

Lesson 5.1 — while Loops

The programs so far have had one weakness: to print something ten times, you had to write ten statements. A loop repeats a block of statements automatically, and the while loop is the simplest form. It looks like an if that runs again and again: the condition in parentheses is checked, and while it stays true, the brace-enclosed body runs one pass after another. The moment the condition is false, the loop ends and the program continues below it.

```
public class CountUp {

    public static void main(String[] args) {
        int i = 1;
        while (i <= 5) {
            System.out.println("Count is " + i);
            i = i + 1;
        }
        System.out.println("Done!");
    }
}
```

Prints Count is 1 through Count is 5, one per line, then Done!

Every while loop needs three moving parts: something that STARTS the loop, here int i = 1 placed before it; something that is CHECKED, the condition i <= 5; and something that CHANGES, the line i = i + 1 inside the body. The change is what pushes the loop toward its end: without it, the condition would never become false. The condition is tested before every pass, including the very first one. Trace the passes: 1 prints and i becomes 2; 2 prints and i becomes 3; and so on until i is 6 and the condition fails. The table below records every pass in order.

Pass	i before the check	i <= 5?	What happens
1	1	true	print Count is 1, then i becomes 2
2	2	true	print Count is 2, then i becomes 3
3	3	true	print Count is 3, then i becomes 4
4	4	true	print Count is 4, then i becomes 5
5	5	true	print Count is 5, then i becomes 6
6	6	false	loop ends; Done! prints

A while loop can legitimately run zero times. If the condition is false on the very first check, Java skips the body and moves on without complaint. In the next program the fuel tank starts at 0, so the burning loop is passed over in complete silence. That maybe-zero behavior makes while the right tool when you cannot know in advance how many passes are needed.

```
public class ZeroRun {

    public static void main(String[] args) {
        int fuel = 0;
        while (fuel > 0) {
            System.out.println("Burning fuel: " + fuel);
            fuel = fuel - 1;
        }
        System.out.println("Engine never started");
    }
}
```

fuel is 0 before the first check, so the body never runs; only the last line prints.

WARNING

If nothing in the body moves the condition toward false, the loop is INFINITE: it runs forever and freezes your program. The fragment below prints i is 1 forever because i is never changed. In a terminal, press Ctrl+C to stop a runaway program; in an IDE, use the stop button.

```
int i = 1;
while (i <= 5) {
    System.out.println("i is " + i);
    // nothing here changes i - this runs forever
}
```

i stays 1 forever, so the condition i <= 5 never becomes false.

NOTE

The condition must be a boolean, exactly like an if condition from Chapter 4. Keep the body in braces even when it holds one statement, for the same reasons as with if. The variable that the condition watches is called the loop variable; updating it inside the body is your job, not Java's.

TIP

Before you run any while loop, ask one question out loud: what makes this condition false? If no line in the body moves toward that answer, you have built an infinite loop. Fix it by adding or correcting the update line.

Lesson Check 5.1

Choose the one best answer for each question. Answers are in the Answer Key.

1. When is a while loop's condition checked?

A. Before every pass, including the first
B. After every pass only
C. Only once, before the loop starts
D. Only when the body prints something

2. int fuel = 0; while (fuel > 0) { ... } How many times does the body run?

A. Until fuel becomes 0
B. Once
C. Forever
D. Zero times

3. What makes the StuckLoop example infinite?

A. println cannot run in a loop
B. Nothing in the body changes i
C. The condition uses <= instead of <
D. The loop variable starts at 1

4. int i = 3; while (i > 0) { System.out.print(i + " "); i = i - 1; } What prints?

A. 3 2 1 0
B. 2 1 0
C. 3 2 1
D. 1 2 3

Lesson 5.2 — for Loops

The while loop spreads its three parts across three places: before the loop, in the condition, and at the bottom of the body. A for loop gathers all three into one header, which makes counting loops compact and harder to break by accident. The header reads for (init; condition; update): run init once, check the condition, run the body, run update, check again, and repeat. When you can say for each number from 1 to 5, a for loop is probably the right tool.

```
public class ForCount {

    public static void main(String[] args) {
        for (int i = 1; i <= 5; i = i + 1) {
            System.out.println("Pass number " + i);
        }
        System.out.println("Loop finished");
    }
}
```

Exactly the same output as the while version: five passes, then Loop finished.

Order of operations matters: the init part runs exactly once, then the condition is checked, then the body runs, then the update runs, and only then is the condition checked again. The update therefore runs after every pass, not before it. The variable declared in the header, int i, exists ONLY inside the loop: after the closing brace it is out of scope, and any attempt to use it is a compile error. If you need the final value after the loop, declare the variable before the loop instead.

```
for (int i = 0; i < 3; i++) {
    System.out.println(i);
}
System.out.println(i);   // compile error: i is gone here
```

A loop variable declared in the header is out of scope below the loop.

Java has a shorthand for the most common updates: i++ means i = i + 1, and i-- means i = i - 1. For a plain statement the prefix form ++i does the same job; the difference only matters when the result is used inside a larger expression, which we avoid for now. Two counting patterns cover most needs: for (int i = 0; i < n; i++) runs exactly n times with i visiting 0 to n-1, while for (int i = 1; i <= n; i++) also runs n times but visits 1 to n. Use the first when filling positions, the second when counting things, and add steps such as i = i + 2 or i-- when the sequence demands it.

The most famous loop pattern is the accumulator: a sum variable that starts at 0 and gathers one contribution per pass. Watch the order of events in SumOneToTen below: sum is created before the loop, the loop adds i to it on every pass, and only after the loop ends is sum printed. Printing inside the loop would show the running total; printing after it shows the final total. Counting matches works the same way, except a counter increases by 1 only when an if finds a candidate.

```
public class SumOneToTen {

    public static void main(String[] args) {
        int sum = 0;
        for (int i = 1; i <= 10; i++) {
            sum = sum + i;
        }
        System.out.println("Sum: " + sum);
    }
}
```

Adds 1 + 2 + ... + 10 and prints Sum: 55.

```
public class CountMultiples {

    public static void main(String[] args) {
        int count = 0;
        for (int i = 1; i <= 20; i++) {
            if (i % 3 == 0) {
                count = count + 1;
            }
        }
        System.out.println("Multiples of 3: " + count);
    }
}
```

Six numbers from 1 to 20 are divisible by 3, so this prints Multiples of 3: 6.

How do you choose between while and for? Use for when the number of passes is known before the loop starts, or the loop walks a simple counting sequence. Use while when the loop depends on something discovered along the way, such as user input or a value shrinking toward a limit. The two loops can always replace each other, so the choice is about clarity, not correctness.

NOTE

Both i++ and ++i add 1 when they stand alone as a statement. You will see i++ in almost every for header in real code, which is why this book uses that form. Chapter 6 revisits the tiny difference between the prefix and postfix versions.

TIP

Trace for loops with a two-column table: one column for i, one for your accumulator such as sum. Filling one row per pass turns a mysterious loop into a predictable list. Try it on SumOneToTen: after pass 4 the sum column reads 10.

Lesson Check 5.2

Choose the one best answer for each question. Answers are in the Answer Key.

1. How many times does the body of for (int i = 0; i < 3; i++) run?

A. 3 times **B.** It never stops

C. 4 times **D.** 2 times

2. In a for header for (init; condition; update), when does the update part run?

A. Only if the body runs zero times **B.** Once, before the loop starts

C. After each pass through the body **D.** Never; you must update inside the body

3. Where can the loop variable i be used if it was declared in the for header?

A. Inside the loop and after it **B.** Only inside the loop

C. Anywhere in main **D.** Only before the loop

4. for (int i = 2; i <= 6; i = i + 2) { System.out.print(i + " "); } What prints?

A. 2 3 4 5 6 **B.** 4 6

C. 2 4 6 8 **D.** 2 4 6

Lesson 5.3 — do-while and Input Validation

Both while and for check their condition first, which means their body can be skipped entirely. The do-while loop flips that: it runs the body first and checks the condition at the bottom, after each pass. The syntax is the word do, a brace-enclosed body, then while (condition) followed by a semicolon. A do-while body always runs at least once, no matter what the condition says.

```
public class DoWhileDemo {

    public static void main(String[] args) {
        int i = 10;
        do {
            System.out.println("i is " + i);
            i = i + 1;
        } while (i < 10);
        System.out.println("The body ran once");
    }
}
```

The condition 11 < 10 is false immediately, yet i is 10 still prints once.

At-least-once behavior is exactly right for input validation: the user must be asked at least once, and asked again whenever the answer is unacceptable. The while version below reads once before the loop and re-prompts inside it, so the same prompt text appears twice in the code. Trace a stubborn user who types -3, then 0, then 4: the loop body runs twice, re-prompting each time, and 4 finally escapes the loop.

```
public class PositiveOnly {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter a positive number: ");
        int n = in.nextInt();
        while (n <= 0) {
            System.out.print("Not positive. Try again: ");
            n = in.nextInt();
        }
        System.out.println("You entered " + n);
    }
}
```

Sample run: the user types -3, then 0, then 4, and only 4 escapes the loop.

The do-while version below puts the prompt inside the body, so the asking code appears once and is guaranteed to run. Read, check at the bottom, and repeat whenever the value is still not positive: the flow is exactly the user's experience. Choose the shape that matches reality: use do-while when the action must happen at least once, and while when it might legitimately be skipped.

```
public class PositiveDoWhile {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int n;
        do {
            System.out.print("Enter a positive number: ");
            n = in.nextInt();
        } while (n <= 0);
        System.out.println("Thanks! You entered " + n);
    }
}
```

The prompt lives inside the loop, so it cannot be forgotten on repeat passes.

WARNING

Two do-while traps: the semicolon after while (condition) is required, and forgetting it is a compile error. Also remember the condition sits at the BOTTOM, so the body runs before any check. Never use do-when zero runs must be possible.

NOTE

If a validation loop mixes nextInt and nextLine, remember the leftover-newline pitfall from Chapter 2: nextInt leaves the newline unread, so a following nextLine grabs an empty string. Plan your reads in the same order the user will type them. Chapter 8 adds string tools that make mixed input easier.

TIP

Menu programs are the classic do-while home: show the menu, read a choice, act, and repeat until the user picks exit. The menu must appear at least once, which is precisely the do-while guarantee. Any time asking is unavoidable, do-while is the honest shape.

Lesson Check 5.3

Choose the one best answer for each question. Answers are in the Answer Key.

1. What is the minimum number of times a do-while body runs?

A. Exactly one
B. Two
C. It depends on the condition
D. Zero

2. Which punctuation does a do-while loop need that a while loop does not?

A. A colon after do
B. A comma between do and while
C. A semicolon after while (condition)
D. An extra pair of braces

3. Why does input validation fit do-while well?

A. The condition is checked before any input
B. do-while runs faster than while
C. It avoids using a Scanner
D. The user must be asked at least once

4. In DoWhileDemo (int i = 10; do { ... } while (i < 10);), what prints?

A. i is 10 forever
B. i is 10, once
C. i is 10, then i is 11
D. Nothing at all

Lesson 5.4 — break, continue, and Nested Loops

Sometimes a loop should end in the middle of a pass, based on what happened inside it. The break statement exits the NEAREST enclosing loop immediately, jumping to the first line after the loop's closing brace. Combined with while (true), a header that is always true, break gives you a loop whose exit decision lives inside the body where the data arrives. This is the classic sentinel-controlled loop: it reads values until a special agreed value appears.

```
public class SentinelSum {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int sum = 0;
        while (true) {
            System.out.print("Enter a number (0 to stop): ");
            int n = in.nextInt();
            if (n == 0) {
                break;
            }
            sum = sum + n;
        }
        System.out.println("Total: " + sum);
    }
}
```

Sample run: the user types 5, then 8, then 0, and the program prints Total: 13.

The continue statement is break's gentler cousin: instead of leaving the loop, it abandons the rest of the current pass and jumps to the next one. In a for loop, continue jumps to the update part and then the condition; that is the safe case, because the update always runs. In a while loop, continue jumps straight to the condition check, skipping everything below it in the body, including the update line. That difference is the trap the next fragment demonstrates.

```
public class SkipOdds {

    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (i % 2 == 1) {
                continue;   // odd: jump to i++
            }
            System.out.print(i + " ");
        }
        System.out.println();
    }
}
```

The odd numbers never reach the print line, so the output is 2 4 6 8 10.

```
int i = 0;
while (i < 5) {
    if (i % 2 == 1) {
        continue;   // skips i = i + 1 below: stuck!
    }
    System.out.println(i);
    i = i + 1;
}
```

At i = 1, continue jumps to the check without updating i, so the loop spins forever.

A loop inside a loop is called a nested loop, and the rule is simple: for every single pass of the outer loop, the inner loop runs from start to finish. If the outer loop runs 3 times and the inner loop runs 4 times per pass, the innermost statement executes 3 times 4, or 12 times, in total. The outer variable usually controls one dimension, such as a row, and the inner variable another, such as a column. StarGrid below prints three rows of four stars by exactly this mechanism.

```
public class StarGrid {

    public static void main(String[] args) {
        for (int row = 1; row <= 3; row++) {
            for (int col = 1; col <= 4; col++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

The inner loop builds one row of 4 stars; the println after it ends each row.

Nested loops power grids, tables, and every text shape you will draw in the challenges. A break inside the inner loop exits only the inner loop: the outer loop keeps going with its next pass. The times-table challenge at the end of this chapter multiplies row by column and uses the escape from Chapter 2 to line the columns up.

Trace nested loops on paper with one line per outer pass, and they stop being mysterious.

> **NOTE**
> break and continue act on the nearest loop only. A break inside an inner loop leaves that inner loop, not the whole nest; to leave both loops you would need a flag condition in the outer loop. Java also has a rarely used labeled break; ignore it until much later.

> **TIP**
> When a loop misbehaves, check two things before anything else: does the condition eventually become false, and does every continue still allow the update to run? Those two questions catch most loop bugs in seconds. Only then start adding print statements to trace values.

Lesson Check 5.4

Choose the one best answer for each question. Answers are in the Answer Key.

1. What exactly does break do inside a loop?

A. Restarts the loop from the top
B. Exits the nearest enclosing loop immediately
C. Skips to the next pass of the loop
D. Ends the whole program

2. In a for loop, where does continue jump to?

A. It exits the loop
B. The top of the body, skipping the update
C. The update part, then the condition
D. The line after the loop

3. An outer loop runs 3 passes; its body contains an inner loop that runs 4 passes. How many times does the innermost statement run?

A. 4
B. 3
C. 7
D. 12

4. Why can continue inside a while loop cause an infinite loop?

A. It may skip the line that updates the loop variable
B. It restarts the loop from the first line
C. It makes the condition permanently true
D. continue is not allowed in while loops

Practice Questions

Work through all ten questions. They are ordered from easiest to hardest. Write your answers on paper before checking the Answer Key.

1. How many times does the body of this loop run?
for (int i = 0; i < 4; i++) { ... }

A. 4 times
B. 3 times
C. 5 times
D. Forever

2. What happens when a while loop's condition is false the very first time it is checked?

A. The body runs exactly once
B. The program crashes
C. The body runs forever
D. The body never runs

3. Which loop is guaranteed to run its body at least once?

A. for **B.** None of them

C. do-while **D.** while

4. The loop **for (int i = 2; i <= 8; i += 2)** runs how many times?

A. 6 times **B.** 4 times

C. 3 times **D.** 8 times

5. An outer loop runs 3 times and, inside it, an inner loop runs 4 times per outer pass. How many times does the inner body run in total?

A. 9 **B.** 12

C. 4 **D.** 7

6. In a for loop, continue jumps straight to the update step (for example i++) before the next condition check.

[ ] True [ ] False

7. What is the exact output of this program?

```
public class Countdown {
    public static void main(String[] args) {
        for (int i = 3; i >= 1; i--) {
            System.out.println(i);
        }
        System.out.println("Liftoff!");
    }
}
```

Output:

8. This program is supposed to print 1, 2, 3 on separate lines. Find the bug.

```
int count = 1;
while (count <= 3);
{
    System.out.println(count);
    count++;
}
```

Bug:

9. Describe a situation where a do-while loop fits better than a while loop, and explain why in two or three sentences.

10. Write a complete program called Evens that uses a for loop to print the even numbers from 2 to 10, one per line.

Mixed Quiz

Fifteen questions mixing everything from this chapter. Score 12 or more before moving on to the challenges.

1. What is missing from this loop if it never stops?
```
int n = 5;
while (n > 0) { System.out.println(n); }
```

A. The println needs a semicolon
B. The condition uses > instead of <
C. The loop variable is never updated, so n stays 5
D. Nothing is missing; this loop ends normally

2. Which order does a for loop header use?

A. for (condition; init; update)
B. for (init; condition; update)
C. for (update; condition; init)
D. for (init; update; condition)

3. An outer loop runs 2 times and an inner loop inside it prints one star per pass for 3 passes. How many stars appear?

A. 6
B. 2
C. 3
D. 5

4. What does the break statement do inside a loop?

A. Skips to the next iteration
B. Exits the nearest enclosing loop immediately
C. Restarts the loop from the beginning
D. Ends the whole program

5. What is a sentinel value in a loop?

A. A variable that counts loop passes
B. The condition in the loop header
C. The first value the loop processes
D. A special input value that tells the loop to stop

6. Which statement about a do-while loop is correct?

A. It cannot contain break
B. It checks its condition before the first pass
C. It can never cause an infinite loop
D. A semicolon is required after while (...); at the end

7. After **for (int i = 0; i < 5; i++) { ... }** finishes, what happens to i?

A. It still exists with value 4
B. It resets to 0
C. It no longer exists; it was created inside the loop
D. It still exists with value 5

8. What does this loop print?
```
int i = 0;
while (i < 3) { i++; }
System.out.println(i);
```

A. 3
B. Nothing; the println never runs
C. 0
D. 2

9. A while loop always checks its condition before each pass, including the first.

[ ] True [ ] False

10. The loop variable of a for loop can still be used after the loop ends.

[ ] True [ ] False

11. In a while loop, continue jumps to the condition check, skipping any statements below it in the body.

[ ] True [ ] False

12. What is the exact output of this program?

```
public class TotalUp {
    public static void main(String[] args) {
        int total = 0;
        for (int i = 1; i <= 4; i++) {
            total += i;
        }
        System.out.println(total);
    }
}
```

Output:

13. This loop should print the numbers 1 through 5. Find the bug.

```
for (int i = 1; i < 5; i++) {
    System.out.println(i);
}
```

Bug:

14. Explain in two or three sentences how an infinite loop happens and how a programmer usually stops one that is already running.

15. Write a complete program called OddSum that uses a while loop to add up the odd numbers 1, 3, 5, 7, 9 and prints the total (25).

Challenges

Five harder thinking problems. Each includes a hint; full step-by-step solutions are in the Answer Key. Try the problem seriously before reading the hint.

Challenge 5.1 — FizzBuzz Warm-Up

Print the numbers 1 to 15, one per line, but for multiples of 3 print Fizz instead of the number, for multiples of 5 print Buzz, and for multiples of both print FizzBuzz.

Hint: Check the "both" case FIRST with % 15 or % 3 == 0 && % 5 == 0, then the single cases, and use else for the plain number.

Challenge 5.2 — Digit Sum

For the number 4729, compute the sum of its digits (4 + 7 + 2 + 9 = 22) using a while loop with the % 10 and / 10 patterns from Chapter 3. Print the sum. Your program should print 22.

Hint: n % 10 peels off the last digit; n / 10 removes it. Keep going while n > 0.

Challenge 5.3 — Countdown on One Line

Use one loop to print exactly this single line of output:
3... 2... 1... GO!

Hint: print (no ln) keeps everything on one line. Decide what the loop prints and what prints after it.

Challenge 5.4 — Star Triangle

Use nested loops to print a right triangle of stars with 5 rows:
*
**
***
****
*****

Hint: The outer loop counts rows 1 to 5; the inner loop prints as many stars as the current row number, then a println ends the row.

Challenge 5.5 — Collatz Counter

The Collatz rule says: if n is even, replace it with n / 2; if n is odd, replace it with 3n + 1. Every positive number eventually reaches 1. Count how many steps the number 27 needs to reach 1. Your program should print 111.

Hint: One "step" is one application of the rule. Loop while n != 1, apply the correct branch, and add 1 to a counter each time.

Chapter Recap

- A while loop checks its condition before every pass, so its body can run zero times; a do-while checks at the bottom and always runs at least once.
- A for loop bundles initialization, condition, and update in one header: for (int i = 0; i < n; i++).
- The loop variable declared in a for header exists only inside the loop.
- Loops need a moving part: if nothing in the body changes the condition, the loop runs forever (infinite loop).
- break exits the nearest loop immediately; continue skips to the next cycle point (the update in for, the check in while).
- Nested loops multiply their pass counts: 3 outer x 4 inner = 12 inner passes.
- The accumulator pattern (sum += i) collects a total across passes; the counter pattern (count++) tallies matches.
- Sentinel values such as 0 or -1 let a loop end based on the data instead of a fixed count.

Common Mistakes

Beginners run into these again and again. Read them now to avoid the pain later.

WARNING

Forgetting to update the loop variable
If the body never changes the variable the condition tests, the condition stays true forever and the program hangs in an infinite loop, usually with no output because the screen buffer never flushes.
Fix: Before running a loop, ask: what makes this condition eventually false? Make sure the body moves toward it (i++, n = n / 10, and so on).

WARNING
Semicolon directly after while (...) or for (...)
while (count <= 3); is a complete loop with an EMPTY body. The brace block below then always runs exactly once (or the loop spins forever), which is rarely what you meant.
Fix: Never put a semicolon between a loop header and its brace block. (do-while is the one exception: it ends with while (...); on purpose.)

WARNING
Off-by-one boundaries
Using i < 5 when the range should include 5 (or the reverse) silently processes one value too few or too many, and the program still runs perfectly happily, so nothing warns you.
Fix: Decide whether the boundary is inclusive or exclusive and pick <= or < deliberately; then test the loop's first and last values.

WARNING
Using continue in a while loop without an update before it
In a while loop, continue jumps straight back to the condition. If the variable update sits BELOW the continue, it is skipped every time and the loop freezes forever.
Fix: Place the update above the continue, or restructure with an if so the update always runs.

WARNING
Expecting a while loop body to run at least once
Because while checks first, a false starting condition skips the body completely; beginners assume the prompt or calculation inside ran and get confused when nothing happened.
Fix: If the body must run at least once (asking for input, for example), choose do-while instead.

CHAPTER 6

Methods

For six chapters your whole program has lived inside one method: main. That works while programs are small, but real software is built from many small, named pieces, and Java's word for such a piece is **method**. In this chapter you will finally open up that mysterious line public static void main and learn to write methods of your own, hand information in, and get answers back. Along the way you will meet parameters, return values, overloading, variable scope, and Javadoc comments. By the end you will be able to split a program into clean, reusable parts instead of one long scroll of code.

In this chapter you will learn to:

- define your own methods and call them from **main**
- pass information into methods with parameters and predict what each call prints
- write methods that return values and use those results in variables and expressions
- overload a method name and explain how the compiler picks a version
- explain variable scope and document methods with **Javadoc** comments

Lesson 6.1 — Your First Methods

So far, every program you have written has squeezed all of its work into main, and for small programs that is fine. But imagine a 500-line program where every step lives in one giant method: finding anything in it would be like finding one sentence in a book with no chapter titles. A method is a named block of code that performs one job, and it solves three problems at once. It lets you **reuse** code you have already written, it gives the job a **name** so readers know what the code is for, and it **organizes** a program into pieces that each fit in your head.

Here is a complete program with one method of our own. Read main first: it calls cheer() twice, and each call runs the code inside the cheer method from top to bottom. After the second call finishes, main continues with its last statement and the program ends. Type the program in and run it before reading on.

```
public class Cheer {

    public static void main(String[] args) {
        cheer();
        cheer();
        System.out.println("Back in main. All done!");
    }

    // Prints one cheer, then returns to the caller.
    public static void cheer() {
        System.out.println("Three cheers for methods!");
    }
}
```

cheer is defined once but runs twice - that is reuse

Look closely at the pieces of the definition. **public** means other code is allowed to call this method, the same word you have seen on every class. **static** means the method belongs to the class itself, which is what lets main call it directly without any setup; that is an honest simplification you can lean on throughout this book. **void** means the method hands nothing back when it finishes, a promise explored in Lesson 6.3. Then comes the name, cheer, followed by parentheses that will hold parameters (empty here) and braces that hold the body.

Part	What it means
public	Other code may call this method
static	Belongs to the class, so main can call it directly
void	The method returns no value
cheer	The method's name - pick a verb describing the job
( )	Holds the parameters, if any; empty here
{ ... }	The body: statements that run on each call

Two rules about calling methods are worth stating plainly. First, a method runs only when it is called: defining cheer does nothing by itself, just as writing down a recipe cooks no dinner. Second, the order of definitions inside a class does not matter; Java reads the whole class before running anything, so a method defined below main is exactly as callable as one above it.

```
public class OrderDemo {

    public static void main(String[] args) {
        fanfare();      // defined below main - fine!
    }

    public static void fanfare() {
        System.out.println("Ta-da!");
    }
}
```

Definition order does not matter; only calls run code

NOTE
main is itself a method: the JVM calls main by name when you run a class, exactly the way your code calls cheer(). Every System.out.println you have ever written was a method call too, just one defined in Java's library.

WARNING
Methods cannot nest. A method definition must sit inside the class, next to main, never inside main's braces. Writing a method inside main is a compile error that beginners hit surprisingly often.

TIP
Name methods after actions, using a verb in lowerCamelCase: printReport, drawMenu, calculateTotal. A method name that reads like an instruction makes the calling code almost self-explanatory.

Lesson Check 6.1

Choose the one best answer for each question. Answers are in the Answer Key.

1. What is a method in Java?

A. A special kind of variable
B. A file that stores your program
C. A named block of code that runs when called
D. A comment the compiler reads

2. Where must the methods you write in this book be defined?

A. Inside main's braces
B. Inside the class, next to main
C. Anywhere after all the calls
D. In a separate file for each method

3. In the header public static void cheer(), what does void mean?

A. The method returns no value
B. The method runs instantly
C. The method cannot be called twice
D. The method has no parameters

4. When does the body of cheer() actually run?

A. Once, when the program starts
B. When the compiler reads the definition
C. Every time any statement runs
D. Only when some code calls cheer()

Lesson 6.2 — Parameters

Often a method needs information from the caller: a name to greet, a number to double, a score to report. A **parameter** is a variable written in the method's header that will hold that information; an **argument** is the actual value the caller places inside the call's parentheses. The distinction is small but every programmer uses it: parameters wait in the definition, arguments arrive with each call. Think of a parameter as a blank label on a form and an argument as the filled-in answer.

```
public class Greeter {

    public static void main(String[] args) {
        greet("Amal");
        greet("Priya");
        greet("Diego");
    }

    // Prints a greeting for whoever is passed in.
    public static void greet(String name) {
        System.out.println("Welcome, " + name + "!");
    }
}
```

One method, three calls, three different arguments

Inside the method, the parameter works exactly like a variable you declared yourself, except its starting value arrives from the caller. Each call delivers a fresh value: greet("Amal") creates a name holding Amal, and greet("Priya") creates a brand-new name holding Priya. A method may take several parameters, separated by commas in the header. The caller's arguments must match the parameter list in count, in order, and in type, or the code will not compile.

```
public class ShowScore {

    public static void main(String[] args) {
        report("Mina", 95);
        report("Omar", 88);
    }

    // The list fixes the order: name first, score second.
    public static void report(String name, int score) {
        System.out.println(name + " scored " + score
                + " points.");
    }
}
```

Two parameters, filled in the same order at each call

One more fact about parameters matters so much that it gets its own rule: primitive parameters are copies. When you pass an int, the method receives a photocopy of the value, not the original variable. Changing the parameter inside the method changes the copy only, and the caller's variable never notices. This behavior is called **pass-by-value**, and the next program proves it.

```
public class CopyDemo {

    public static void main(String[] args) {
        int cups = 3;
        tryToDouble(cups);
        System.out.println("cups is still " + cups);
    }

    // Gets a COPY of cups, so x cannot touch it.
    public static void tryToDouble(int x) {
        x = x * 2;
        System.out.println("Inside, x is " + x);
    }
}
```

Output: Inside, x is 6, then cups is still 3

WARNING

Argument order and types must match the header. If report is declared as report(String name, int score), the call report(95, "Mina") will not compile, even though both values appear somewhere in the call. Match the list exactly: count, order, and type.

NOTE

Parameters are local to the method. The x in CopyDemo exists only inside tryToDouble; main has never heard of it, and it vanishes the moment the method returns. Lesson 6.4 turns this fact into the full idea of scope.

Lesson Check 6.2

Choose the one best answer for each question. Answers are in the Answer Key.

1. In the call greet("Amal"), the text "Amal" is best described as...

A. the return value of greet

B. the parameter of greet

C. a comment read by the compiler

D. the argument for the name parameter

2. With cups = 3, the call tryToDouble(cups) runs x = x * 2 inside the method. Afterward, main prints cups. What prints?

A. 3
B. The program crashes
C. 6
D. 0

3. A method is declared as book(String title, int pages). Which call compiles?

A. book(350)
B. book("Java 101")
C. book("Java 101", 350)
D. book(350, "Java 101")

4. What happens to a parameter when its method finishes?

A. It turns into a global variable
B. It disappears; it exists only inside the method
C. It moves back into main
D. It stays for the whole program

Lesson 6.3 — Return Values

A void method performs an action and hands nothing back, like a printer that prints but never tells you what it counted. Many methods, though, exist to compute an answer: Math.max(3, 7) answers 7, and you have been using that answer since Chapter 3. To write such a method, replace void in the header with the type of the answer: int, double, boolean, or any other type. The header now promises the caller one value of exactly that type.

The return statement keeps the promise. The line return n * n; computes the expression and sends its value back to the caller, and it also ends the method immediately: no statement below a completed return ever runs. The expression's type must match the declared return type, so an int method returns an int. Here is the same idea in a full program, with the result used three different ways.

```
public class SquareDemo {

    public static void main(String[] args) {
        int result = square(5);   // 1: store it
        System.out.println(result);
        System.out.println(square(9)); // 2: print it
        int bigger = square(3) + square(4); // 3: use it
        System.out.println(bigger);
    }

    // Returns the square of n.
    public static int square(int n) {
        return n * n;
    }
}
```

Output: 25, then 81, then 25

Callers treat the returned value like any other value of that type. Store it in a variable, print it directly, or drop it straight into a bigger expression, as SquareDemo shows in three different ways. The call square(9) is an expression that evaluates to 81, so anywhere an int makes sense, a call to an int-returning method makes sense too. Boolean-returning methods are especially pleasant: they read like questions and slot straight into if statements.

```
public class PassingCheck {

    public static void main(String[] args) {
        System.out.println(isPassing(72));
        System.out.println(isPassing(58));
        if (isPassing(84)) {
            System.out.println("See you next term!");
        }
    }

    // Returns true when score is 60 or higher.
    public static boolean isPassing(int score) {
        return score >= 60;
    }
}
```

Output: true, then false, then the celebration line

Void methods can leave early too, using a bare return; with no value after it. This is handy for rejecting bad input at the top of a method, as TicketPrinter does for a negative age. The promise runs in the other direction as well: a method with a non-void return type must return a value on every path through its code. Miss one path, for example an if branch without its own return, and the compiler reports a missing return statement error.

```
public class TicketPrinter {

    public static void main(String[] args) {
        printTicket(-3);
        printTicket(42);
    }

    // Prints a ticket line, but refuses bad ages.
    public static void printTicket(int age) {
        if (age < 0) {
            System.out.println("Invalid age.");
            return;   // leaves the method right now
        }
        System.out.println("Ticket for age " + age);
    }
}
```

Output: Invalid age., then Ticket for age 42

NOTE

Exactly one value comes back from each call. The line return x, y; is a compile error; return one value, or combine the data first (arrays, coming in Chapter 7, are one classic way to carry several values).

WARNING

A statement placed after the final return of a method can never run, and the compiler refuses it with an unreachable statement error. If you see that error, the statement after the return is the one that is stuck.

Lesson Check 6.3

Choose the one best answer for each question. Answers are in the Answer Key.

1. What does the return type in a method's header tell you?

A. The type of the method's name
B. How many times the method may run
C. The type of value the method sends back
D. How many parameters the method takes

2. A method is defined as int doubleIt(int n) { return n * 2; }. What is the value of doubleIt(6)?

A. 6
B. void
C. 62
D. 12

3. Besides sending a value back, what else does return do?

A. Ends the method immediately
B. Pauses the method until later
C. Deletes the parameters
D. Restarts the method from the top

4. A void method should stop early in the middle of its body. Which statement does that?

A. return 0;
B. return;
C. stop;
D. break;

Lesson 6.4 — Overloading, Scope, and Javadoc

Java allows several methods to share one name as long as their parameter lists differ in the number or the types of their parameters. This is called **overloading**, and you have used it all along: println accepts strings, ints, doubles, chars, and booleans, and it always prints the right way. When a call appears, the compiler matches the arguments against the available parameter lists and picks the best-fitting version. The same name, three different jobs:

```
public class Printer {

    public static void main(String[] args) {
        show(7);
        show(2.5);
        show("seven");
    }

    // This version handles whole numbers.
    public static void show(int n) {
        System.out.println("int version: " + n);
    }

    // Same name, different type: a legal overload.
    public static void show(double d) {
        System.out.println("double version: " + d);
    }

    // Same name, different count: also legal.
    public static void show(String s) {
        System.out.println("String version: " + s);
    }
}
```

The compiler routes each call to the matching overload

Overloads must be genuinely different. Two methods with the same name and identical parameter lists cannot coexist; the compiler has no way to choose and reports a duplicate method error. Ambiguity is the second trap: if two overloads would both fit the arguments only after type conversions, with no exact or clearly better

match, the compiler refuses the call instead of guessing. Clear, distinct parameter lists keep overloading friendly.

Every variable lives inside the innermost braces where it was declared, and those braces are called its **scope**. A variable declared in main does not exist inside other methods, which is exactly why parameters exist: they carry values across the brace wall. A loop variable declared in a for statement exists only inside that loop. Two different methods may each declare a variable named i without any conflict, because the two variables never meet.

```
public class ScopeDemo {

    public static void main(String[] args) {
        int messages = 2;   // lives only inside main
        announce(messages);
        System.out.println("main still sees "
                + messages);
    }

    public static void announce(int messages) {
        // A separate variable that shares the name.
        for (int i = 0; i < messages; i++) {
            System.out.println("Message " + i);
        }
        // i no longer exists here.
    }
}
```

Two independent variables, each in its own braces

The last tool of this chapter is the **Javadoc comment**, Java's official way to document a method. It looks like a block comment but starts with /**, sits directly above the method, and uses special tags: one @param line per parameter and an @return line for any method that returns a value. The first sentence should summarize the method in a single line. The javadoc tool can turn these comments into web pages, and IDEs show them as tooltips, which is why real libraries feel so well explained.

```
public class TicketPrice {

    /**
        * Calculates the total price for a group visit.
        *
        * @param tickets how many tickets are bought
        * @param pricePerTicket cost of one ticket
        * @return the total cost for all tickets
        */
    public static double totalPrice(int tickets,
            double pricePerTicket) {
        return tickets * pricePerTicket;
    }

    public static void main(String[] args) {
        double cost = totalPrice(4, 12.50);
        System.out.println("Total: " + cost);
    }
}
```

Javadoc sits above the method; tags match the header

TIP
Write the Javadoc comment before writing the method. Explaining the parameters and the return value forces you to decide what the method does, which is half of designing it.

NOTE
This is the book's official documentation style. From the final project onward, every method you share should carry a Javadoc comment with @param and @return lines.

Lesson Check 6.4

Choose the one best answer for each question. Answers are in the Answer Key.

1. What is method overloading?

A. Calling one method from inside another
B. Renaming a variable inside a method
C. A method that calls itself
D. Several methods sharing a name with different parameter lists

2. A class has go(int n) and go(double d). What does the call go(7) run?

A. The double version, because double holds more
B. The int version, because 7 matches int exactly
C. Neither; the call is illegal
D. Both versions, one after the other

3. A variable is declared inside a for loop's braces. Where can it be used?

A. Only inside that loop's braces
B. Also after the loop ends
C. Anywhere in the class
D. Anywhere in the method

4. Which Javadoc tag documents one input of a method?

A. @arg
B. @input
C. @param
D. @value

Practice Questions

Work through all ten questions. They are ordered from easiest to hardest. Write your answers on paper before checking the Answer Key.

1. Which line correctly calls a method named cheer from main?

A. call cheer();
B. cheer;
C. cheer();
D. void cheer();

2. A method's header is void show(String label, int count). Which call matches it?

A. show(12);
B. show(12, "Seats");
C. show("Seats");
D. show("Seats", 12);

3. main runs int total = 8; addTen(total); System.out.println(total); where addTen is void addTen(int t) { t = t + 10; }. What prints?

A. 8
B. Nothing; the code does not compile
C. 10
D. 18

4. Which header belongs to a method that answers a question with true or false?

A. public static void isFull(int level)
B. public static boolean isFull(int level)
C. public static int isFull(int level)
D. public static String isFull(int level)

5. A class defines print(int n) and print(double d). Which version does the call print(2) run?

A. Both, one after the other
B. print(int), because 2 is an int literal
C. print(double), because double is wider
D. Neither; the call is ambiguous

6. A method must be defined above main in the file before main can call it.

[ ] True [ ] False

7. What is the exact output of this program?

```
public class Trace {

    public static void show(int n) {
        n = n + 10;
        System.out.println(n);
    }

    public static void main(String[] args) {
        int x = 4;
        show(x);
        System.out.println(x);
        show(x);
    }
}
```

Output:

8. This program does not compile. Find the bug.

```
public class BrokenSpot {

    public static void main(String[] args) {
        System.out.println("Starting");

        public static void announce() {
            System.out.println("Announce!");
        }
    }
}
```

Bug:

9. Explain, in two or three sentences, when you would write a void method and when you would write a method with a return type.

10. Write a complete program called LineMaker with a method public static void repeat(String word, int times) that prints word on its own line, times times in total. main should call repeat("Java", 3);.

Mixed Quiz

Fifteen questions mixing everything from this chapter. Score 12 or more before moving on to the challenges.

1. Which word in a header marks a method that hands nothing back?

A. empty **B.** void
C. static **D.** null

2. int x = sayHello(); fails to compile when sayHello is declared public static void sayHello(). Why?

A. A void method produces no value to store **B.** The call needs the word new
C. sayHello must be called with two arguments **D.** int variables cannot hold method calls

3. In this chapter's methods, what does the word static on the header allow?

A. the method to change while running **B.** the method to run twice as fast
C. the method to skip its parameters **D.** main to call the method directly, with no setup

4. Parameters in this chapter's methods are...

A. shared with every method in the class **B.** remembered between calls
C. copies of the values passed by the caller **D.** the original variables from main

5. report(String name, int score) is called as report(95, "Mina"). What happens?

A. The compiler rejects the call: types and order do not match **B.** It prints 95 scored Mina points.
C. It prints Mina scored 95 points. **D.** It runs but prints nothing

6. A class has greet(String name) and greet(String name, int times). Which call runs the two-parameter version?

A. greet("Ana"); **B.** greet();
C. greet("Ana", 3); **D.** greet(3);

7. Where does a variable declared at the top of main exist?

A. Everywhere in the program **B.** Only inside loops in main
C. Inside every method of the class **D.** Only inside main's braces

8. Which Javadoc tag documents the value a method sends back?

A. @value **B.** @return
C. @sends **D.** @out

9. A method's statements run as soon as the JVM reads its definition.

[ ] True [ ] False

10. The statement return ends a method immediately, even with statements left below it.

[ ] True [ ] False

11. Two methods in one class may share a name if their parameter lists differ.

[ ] True [ ] False

12. What is the exact output of this program?

```
public class QuizTrace {

    public static int triple(int n) {
        return n * 3;
    }

    public static void main(String[] args) {
        int a = triple(4);
        System.out.println(a);
        System.out.println(triple(1) + triple(2));
    }
}
```

Output:

13. This program does not compile. Find the bug.

```
public class Bug6 {

    public static void printHello() {
        System.out.println("Hello!");
    }

    public static void main(String[] args) {
        int x = printHello();
        System.out.println(x);
    }
}
```

Bug:

14. A friend claims that calling changeNumber(x) can change the value of x in main when the method's parameter is an int. In two or three sentences, explain what really happens.

15. Write a complete program called Salary with a method public static double addRaise(double salary) that returns the salary plus 500.0. main should keep the original in a variable, call the method, and print both the original and the raised salary.

Challenges

Five harder thinking problems. Each includes a hint; full step-by-step solutions are in the Answer Key. Try the problem seriously before reading the hint.

Challenge 6.1 — Greeting Machine

Write a program called GreetingMachine with a method public static void greet(String name, int times) that prints Hello, name! exactly times times, each on its own line. main should call greet("Layla", 2); and greet("Sam", 3); so the output shows two greetings for Layla and three for Sam.

Hint: The method needs a loop that runs times times, printing a line built from the name parameter each pass. Keep the counting loop and the printing inside the method, not in main.

Challenge 6.2 — Even or Odd Reporter

Write a program called EvenReporter with a method public static boolean isEven(int n) that returns true when n is divisible by 2 and false otherwise. main should use a loop from 1 to 10 and print, for each number, either 4 is even or 3 is odd, using the method's answer inside an if/else.

Hint: The remainder operator % from Chapter 3 answers the whole puzzle: a number is even exactly when n % 2 equals 0, and that comparison itself is a boolean expression you can return directly.

Challenge 6.3 — Max of Three

Write a program called MaxOfThree with a method public static int max3(int a, int b, int c) that returns the largest of its three arguments. main should print max3(4, 9, 2), max3(-5, -2, -9), and max3(7, 7, 3) to show it works, including with negative numbers and ties.

Hint: Keep a champion: start best at a, then let b and c challenge it one at a time with if statements. If you prefer, explore chaining Math.max calls and compare the two styles.

Challenge 6.4 — Temperature Converter

Write a program called TempConverter with a method public static double fahrenheitToCelsius(double f) that returns the Celsius equivalent using the formula (f - 32.0) * 5.0 / 9.0. main should print the conversions for 32.0, 68.0, and 212.0 degrees Fahrenheit, one per line.

Hint: The method is a single return of the formula. Use 5.0 and 9.0 (with decimal points) so the division is decimal division, not integer division.

Challenge 6.5 — Table Trouble

The program below should print the multiplication tables for 2, 3, and 4, one table after another, using the method printTable(int n). As written it does not compile: find both bugs, fix them, and add the missing call so all three tables appear.

```
public class TableTrouble {

    public static void main(String[] args) {
        printTable;         // something is missing
        printTable(3)       // and here too
    }

    // Prints the multiplication table for n, 1 to 10.
    public static void printTable(int n) {
        for (int i = 1; i <= 10; i++) {
            System.out.println(n + " x " + i + " = "
                    + n * i);
        }
    }
}
```

Hint: Calling a method always needs parentheses, even with no arguments, and every statement needs its semicolon. Then check which tables main asks for.

Chapter Recap

- A method is a named block of code defined inside the class; it runs only when called, and definition order never matters.
- main is itself a method: the JVM calls it to start your program, and System.out.println is a method call too.
- A header reads public static void name(...) for an action; replace void with a type (int, double, boolean) when the method returns a value. static is what lets main call the method directly.
- Parameters are variables in the header; arguments are the values passed at the call site. Count, order, and types must match.
- Primitive parameters receive copies (pass-by-value), so changing a parameter never changes the caller's variable.
- return sends back exactly one value and ends the method immediately; void methods can exit early with a bare return;
- Overloading means one name with several parameter lists; the compiler picks by matching the arguments, and ambiguous calls do not compile.
- Variables live in the innermost braces that declare them (scope). Document methods with /** ... */ Javadoc comments using @param and @return.

Common Mistakes

Beginners run into these again and again. Read them now to avoid the pain later.

WARNING

Defining a method inside main
Methods cannot nest. A definition inside main's braces confuses the compiler completely, and the error messages it produces rarely point at the real problem.
Fix: Place every method directly inside the class, side by side with main, and call it from main with methodName();

WARNING
Using a void call where a value is expected
Writing int x = printHello(); when printHello is void tries to store a result that does not exist, so the compiler rejects the assignment.
Fix: Give the method a return type and a return statement, or drop the assignment and just call printHello();

WARNING
Expecting a parameter change to reach the caller
Primitive parameters are copies, so x = x * 2; inside a method changes only the method's private copy and main's variable stays as it was.
Fix: Return the new value and store it at the call site: cups = doubleIt(cups);

WARNING
Passing arguments in the wrong order or type
report(String name, int score) called as report(95, "Mina") fills the parameters positionally, so the types do not match and the call does not compile.
Fix: Match the header exactly: same number of arguments, same order, same types.

WARNING
Forgetting a return on one path of a non-void method
An if that returns but whose else path falls off the end leaves the method with no value to send back, and the compiler reports a missing return statement error.
Fix: Make every path through a non-void method end in a return, or restructure so a single return at the bottom always runs.

MINI-PROJECT 2

Number Guessing Game

Every programmer builds a guessing game at some point, and for a good reason: it needs input, decisions, loops, a bit of randomness, and clean methods working together. You will build it in three small steps, and the final version keeps asking until the player finds the secret number, then reports how many tries it took.

Concepts used: while loops, if-else chains, methods, parameters and return values, casting, Math.random, Scanner input

Step 1 — A Secret Number Helper

First a method that produces the secret. Math.random() returns a double from 0.0 up to (but not including) 1.0. Multiplying by max gives a value from 0 up to max, and the cast (int) chops off the decimals. Adding 1 shifts the range so the result is 1 through max, never 0 and never max + 1. Casting a double to int always truncates toward zero, which is exactly the chopping we need.

```
/** Returns a random integer from 1 to max. */
public static int secretNumber(int max) {
    return (int) (Math.random() * max) + 1;
}
```

Code	What it does
`/** Returns a random integer from 1 to max. */`	A Javadoc comment (Chapter 6) documents the method for anyone reading it later.
`public static int secretNumber(int max) {`	A method named secretNumber that takes one int parameter and returns an int.
`    return (int) (Math.random() * max) + 1;`	Random double in [0, 1), times max, cast to int (0 to max-1), plus 1 shifts to 1 through max.
`}`	Closes the method body.

Step 2 — One Round of Guessing

Now a method that plays one full round: it loops until the player guesses correctly, steering with higher/lower hints. The do-while would work here, but a while loop with a boolean flag shows the pattern you will reuse in menu programs. Note the two Scanners question: there is only ONE Scanner, created in main and passed in as a parameter, because building two Scanners on System.in can swallow input.

```
public static void playRound(int secret, Scanner in) {
    int tries = 0;
    int guess = 0;
    while (guess != secret) {
        System.out.print("Your guess: ");
        guess = in.nextInt();
        tries++;
        if (guess > secret) {
            System.out.println("Too high!");
        } else if (guess < secret) {
            System.out.println("Too low!");
        }
    }
    System.out.println("Correct! You got it in "
            + tries + " tries.");
}
```

Step 3 — main Brings It Together

The main method creates the shared Scanner, picks the secret, and calls playRound. Read the explanation table under the full listing below to see how the pieces talk to each other.

```
import java.util.Scanner;

public class GuessGame {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.println("I picked a number "
                + "from 1 to 100. Guess it!");
        int secret = secretNumber(100);
        playRound(secret, in);
    }

    // secretNumber method from Step 1
    // playRound method from Step 2
}
```

How It Works

The program is split by responsibility: secretNumber knows how to pick a number, playRound knows how to run the guessing conversation, and main just connects them. That separation is the whole point of Chapter 6: each method is small enough to test in your head, and main reads like a table of contents for the game.

The loop condition guess != secret keeps the round alive until the player is right. Inside the loop, an if-else chain from Chapter 4 sorts each guess into too high, too low, or correct. The tries counter increments on every read, no matter what the guess was, which is why the final message can report the honest number of attempts.

Passing the Scanner into playRound, rather than creating it there, keeps exactly one Scanner connected to the keyboard for the whole program. This is a habit worth copying in every project from now on, because a second Scanner on System.in is a classic source of input bugs.

Things to Try

- Count only wrong guesses by moving tries++ into the two hint branches instead of next to nextInt.
- Add a maximum of 7 guesses; end the round with a losing message if the player runs out (you will need break from Chapter 5).
- Let the player choose the range: ask for the maximum before the round starts and pass it to secretNumber.
- After a win, ask Play again? (y/n) and loop the whole round until the answer is n.
- Track the best score across rounds in a variable and announce it when the player quits.

CHAPTER 7

Arrays

Real programs rarely juggle one number at a time; they manage lists of scores, grids of pixels, and collections of readings. Java's tool for holding many values of one type under a single name is the **array**. In this chapter you will create arrays, walk through them with loops, and build the classic algorithms every programmer knows: sum, average, max, search, and reverse. You will also meet two-dimensional grids and the helpful methods of java.util.Arrays. These are the last core data tools you need before the book turns to strings, files, and your final project.

In this chapter you will learn to:

- create arrays with **new** and with initializer lists
- read and write elements using indexes and the **.length** field
- process arrays with index loops and for-each loops
- implement max, linear search, counting, and in-place reversal
- use 2D arrays and the **java.util.Arrays** helper methods

Lesson 7.1 — Creating Arrays

An **array** is a numbered row of boxes that holds a fixed number of values, all of the same type. Where five separate variables (score1, score2, and so on) would litter your code, one array named scores holds all five under a single name. The number of boxes is chosen when the array is created and can never change afterward; arrays are fixed size. Each box holds one value, called an **element**.

The most general way to create an array uses new: int[] scores = new int[5]; reads as an int array named scores with room for five ints. The type on the left, int[], is the array type; the number in the brackets on the right is the size. If you already know the values, an **initializer list** creates the array full: int[] ages = {9, 12, 15}; makes a three-element array in one line. Either way, the size is locked in at the moment of creation.

```
public class FirstArray {

    public static void main(String[] args) {
        int[] scores = new int[5];  // five boxes

        scores[0] = 97;   // write into box 0
        scores[1] = 84;
        scores[4] = 91;   // last box is index 4

        System.out.println(scores[0]);  // read box 0
        System.out.println(scores[3]);  // still 0
        System.out.println(scores.length);
    }
}
```

Output: 97, then 0, then 5

Array positions are numbered starting at 0, so a five-element array has indexes 0, 1, 2, 3, and 4; the last index is always length - 1. You read a value with scores[2] and store one with scores[0] = 98;. Every array also carries a built-in field named length: scores.length is 5 for the array above. Write it without parentheses; length is a stored value (a field), not a method, so scores.length() does not compile. A brand-new array is never empty of content: Java fills it with default values you can predict.

Element type	Default value in every box
int	0
double	0.0
boolean	false
char	the null character (\u0000)
String	null (no object yet)

```
public class Defaults {

    public static void main(String[] args) {
        int[] counts = new int[3];
        double[] prices = new double[2];
        boolean[] flags = new boolean[2];

        System.out.println(counts[0]);
        System.out.println(prices[1]);
        System.out.println(flags[0]);
    }
}
```

Output: 0, then 0.0, then false - nothing was assigned

What happens if you ignore the limits and read scores[5] from a five-element array, or scores[-1]? Java cannot return a value that does not exist, so the program stops immediately with a runtime error named ArrayIndexOutOfBoundsException. Nothing was wrong at compile time; the crash only happens if the bad line actually runs. Staying inside 0 to length - 1 is your job, and the loops of the next lesson make that easy.

WARNING
Going outside the valid indexes 0 to length - 1 stops the program with ArrayIndexOutOfBoundsException. This is a runtime error: the code compiles fine and only crashes when the bad index is actually used.

NOTE
length is a field, so it takes no parentheses: scores.length. A call with parentheses such as scores.length() is a compile error, because length is a stored value rather than a method.

TIP
Read scores[2] aloud as scores at index 2. Thinking index first, value second keeps off-by-one mistakes away.

Lesson Check 7.1

Choose the one best answer for each question. Answers are in the Answer Key.

1. For int[] data = new int[6];, which indexes are valid?

A. 1 through 6 **B.** 1 through 5
C. 0 through 6 **D.** 0 through 5

2. How do you read the number of elements stored in the array nums?

A. nums.size **B.** nums.length()
C. nums.length **D.** length(nums)

3. int[] counts = new int[4]; runs. What is counts[2] right after that line?

A. 0 **B.** It depends on the computer
C. 4 **D.** A crash happens

4. Reading data[7] from a 5-element array causes...

A. Java to return -1 **B.** a runtime error: ArrayIndexOutOfBoundsException
C. Java to return 0 **D.** a compile-time error

Lesson 7.2 — Arrays and Loops

Arrays and loops were made for each other. The index variable i runs from 0 while i < arr.length, and each pass of the loop body handles exactly one element: arr[i]. Change the body and the same loop shape sums, prints, doubles, or counts the whole array. This one pattern does most array work in the entire language.

```
public class ScoreStats {

    public static void main(String[] args) {
        int[] scores = {72, 91, 85, 64, 98};

        int sum = 0;
        for (int i = 0; i < scores.length; i++) {
            sum = sum + scores[i];
        }
        double average = (double) sum / scores.length;

        System.out.println("Sum: " + sum);
        System.out.println("Average: " + average);
    }
}
```

Output: Sum: 410, then Average: 82.0

The average needs one extra touch from Chapter 3: sum and scores.length are both ints, so sum / scores.length would do integer division and throw away the decimal part. Casting one operand with (double) forces decimal division, giving 82.0 instead of 82. Watch for this every time you divide two int values. Summing and averaging are now yours whenever an array of numbers walks by.

When you want every value but never the position, Java offers a shorter loop: for (int t : temps) reads as for each int t in temps. The loop hands you elements one at a time, front to back, with no index bookkeeping. But be careful: the loop variable t is a **copy** of each element. Assigning to t changes the copy, not the array, so a

for-each loop cannot replace elements.

```
public class ForEachDemo {

    public static void main(String[] args) {
        int[] temps = {18, 21, 25};

        for (int t : temps) {   // for each t in temps
            System.out.println(t);
        }

        for (int t : temps) {
            t = 0;   // changes the copy only
        }
        System.out.println(temps[0]);  // still 18
    }
}
```

Output: 18, 21, 25, then 18 - the reset changed nothing

Whenever position matters, use the index loop. Numbering the output, changing elements in place, and skipping the first value all need to know where you are. Java's index loop plays the role that Python's enumerate plays, and the equivalent of enumerate's start argument is simply doing arithmetic on i: print i + 1 to count from one. The next program prints each name with a human-friendly number.

```
public class SeatList {

    public static void main(String[] args) {
        String[] names = {"Ana", "Bo", "Cy"};
        for (int i = 0; i < names.length; i++) {
            System.out.println((i + 1) + ". "
                    + names[i]);
        }
    }
}
```

Output: 1. Ana, then 2. Bo, then 3. Cy

WARNING
for (int x : arr) { x = 0; } changes nothing but the copy. To modify an array, write to arr[i] inside an index loop.

TIP
Coming from Python's enumerate? The index loop for (int i = 0; i < arr.length; i++) is Java's version, and using i or i + 1 inside the body matches enumerate's start parameter.

NOTE
Prefer for-each whenever the index is unneeded; it removes the chance of an out-of-bounds mistake. Switch to an index loop the moment you need the position.

Lesson Check 7.2

Choose the one best answer for each question. Answers are in the Answer Key.

1. Which loop should you use to print each element together with its position number?

   A. No loop can print positions
   B. An index for loop, because it knows i
   C. A while loop with no condition
   D. A for-each loop, which is faster

2. int[] a = {5, 6, 7}; In for (int x : a), what is x on the first pass?

   A. 5
   B. 6
   C. The index 0
   D. 0

3. Why does the average calculation use (double) sum / scores.length?

   A. Because averages must be whole numbers
   B. To convert the array to a double
   C. Because length is a double
   D. To force decimal division instead of integer division

4. Inside for (int x : arr), the statement x = 99; does what?

   A. Changes arr[0] to 99
   B. Changes every element to 99
   C. Changes a copy; the array is untouched
   D. Crashes the program

Lesson 7.3 — Array Algorithms

Some array recipes are so common they are called classic algorithms, and the most famous finds the largest value. The idea is to keep a champion: start max at the first element, then walk the rest of the array, and whenever a value beats the champion, crown it. The starting choice matters more than beginners expect, as the warning below explains. The same pattern with < instead of > finds the minimum.

```
public class MaxHunter {

    public static void main(String[] args) {
        int[] lows = {-8, -3, -15, -1};

        int max = lows[0];   // start with element 0
        for (int i = 1; i < lows.length; i++) {
            if (lows[i] > max) {
                max = lows[i];
            }
        }
        System.out.println("Largest: " + max);
    }
}
```

Output: Largest: -1

> **WARNING**
> Never start max at 0 unless you know every value is nonnegative. For {-8, -3, -15, -1} a tracker starting at 0 would stay 0 and report a value that is not even in the array. Starting from the first element always works.

Searching asks a yes-or-no question with a location: is 8 in the array, and where? **Linear search** checks every element from the front, returning the index the moment the target appears. If the loop finishes without a match, the method returns -1, a value that can never be a real index, so the caller knows the search failed. Here is the whole algorithm as a reusable method from Chapter 6.

```
public class SearchDemo {

    public static void main(String[] args) {
        int[] data = {42, 17, 8, 99, 23};
        System.out.println(find(data, 8));
        System.out.println(find(data, 50));
    }

    // Returns target's index, or -1 if absent.
    public static int find(int[] arr, int target) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                return i;
            }
        }
        return -1;
    }
}
```

Output: 2, then -1

Two more recipes belong in your notebook. Counting matches keeps a counter and adds one each time a condition holds, like counting scores of 60 or more. Reversing an array **in place** swaps the first element with the last, the second with the second-to-last, and so on, meeting in the middle; each swap needs a temp variable so the first value is not lost when the second one lands on it.

```
public class ReverseDemo {

    public static void main(String[] args) {
        int[] nums = {10, 20, 30, 40, 50};

        for (int i = 0; i < nums.length / 2; i++) {
            int temp = nums[i];
            nums[i] = nums[nums.length - 1 - i];
            nums[nums.length - 1 - i] = temp;
        }

        for (int n : nums) {
            System.out.println(n);
        }
    }
}
```

Output: 50, 40, 30, 20, 10, one per line

One last pattern appears everywhere in real code: the **partial fill**. You create the array with generous capacity, then keep an int named size that counts how many slots are actually used. Loops then run while i < size instead of i < arr.length, and adding a value means storing it at index size and increasing size. The empty slots simply wait until needed, which is how a program can hold up to, say, 100 readings without knowing in advance how many will arrive.

> **NOTE**
> Why length / 2 swaps for reversing? Each swap fixes two positions at once; doing all length swaps would un-reverse the array back to its original order.

> **TIP**
> -1 is the standard code for not found, because real indexes start at 0. Returning it lets callers write if (index != -1) without any extra machinery.

Lesson Check 7.3

Choose the one best answer for each question. Answers are in the Answer Key.

1. Why initialize max with arr[0] instead of 0?

A. 0 is not an int
B. Java requires that starting value
C. 0 is wrong when every element is negative
D. arr[0] is always the largest

2. A search method returns -1. What should the caller understand?

A. The search crashed
B. The target is at index -1
C. The array is empty
D. The target is not in the array

3. Swapping two array elements needs a temp variable because...

A. arrays refuse odd numbers of assignments
B. the first assignment would overwrite a value still needed
C. temp makes the swap faster
D. Java cannot assign array elements directly

4. How many swaps reverse an 8-element array in place?

A. 4
B. 16
C. 8
D. 7

Lesson 7.4 — 2D Arrays and java.util.Arrays

An array can hold arrays: int[][] grid = new int[3][4]; creates three rows of four columns, twelve values in total. Think of grid[r][c] as row r, column c, with the row index written first. The initializer form lists rows inside braces: {{2, 4, 6}, {1, 3, 5}} makes two rows of three. Reading an element out of a grid uses both indexes in order.

```
public class FreshGrid {

    public static void main(String[] args) {
        int[][] grid = new int[3][4];

        grid[1][2] = 9;   // row 1, column 2
        System.out.println(grid[1][2]);
        System.out.println(grid.length);
        System.out.println(grid[0].length);
    }
}
```

Output: 9, then 3 (rows), then 4 (columns in row 0)

Two-dimensional data is walked with nested loops: the outer loop moves down the rows, the inner loop moves across each row. grid.length is the number of rows, and grid[r].length is the length of row r, which is how the inner loop knows when to stop. This row-by-row order is called **row-major** traversal, and it prints the grid the way you read a page: left to right, then down.

```
public class GridDemo {

    public static void main(String[] args) {
        int[][] grid = {{2, 4, 6}, {1, 3, 5}};

        for (int r = 0; r < grid.length; r++) {
            for (int c = 0; c < grid[r].length; c++) {
                System.out.print(grid[r][c] + " ");
            }
            System.out.println();
        }
    }
}
```

Output: 2 4 6 on row one, 1 3 5 on row two (each number is followed by one space)

Java ships a toolkit for arrays in java.util.Arrays, and using it starts with an import line. Arrays.toString(arr) turns an array into a readable String like [7, 2, 9, 4], perfect for printing. Arrays.sort(arr) arranges the elements in ascending order, and Arrays.fill(arr, value) stores the value into every element. Both of those change the array itself and return void: there is no return value to print, so you print the array afterward.

```
import java.util.Arrays;

public class ArraysHelpers {

    public static void main(String[] args) {
        int[] data = {7, 2, 9, 4};

        System.out.println(Arrays.toString(data));

        Arrays.sort(data);   // changes data itself
        System.out.println(Arrays.toString(data));

        Arrays.fill(data, 0);  // every element to 0
        System.out.println(Arrays.toString(data));
    }
}
```

Output: [7, 2, 9, 4], then [2, 4, 7, 9], then [0, 0, 0, 0]

Method	What it does
Arrays.toString(arr)	Returns a printable String such as [2, 4, 7]
Arrays.sort(arr)	Sorts arr in place, ascending; returns nothing
Arrays.fill(arr, v)	Stores v into every element of arr

WARNING

Arrays.sort does not hand back a new sorted array; it sorts in place and returns void. The line System.out.println(Arrays.sort(nums)); refuses to compile. Sort first, then print the array, most easily with Arrays.toString.

NOTE

Without the import you will see cannot find symbol: Arrays. One line, import java.util.Arrays; above the class, fixes it.

Lesson Check 7.4

Choose the one best answer for each question. Answers are in the Answer Key.

1. int[][] g = new int[3][4]; runs. What is g.length?

A. 0, the array is empty
B. 4, the number of columns
C. 3, the number of rows
D. 12, the number of elements

2. Which expression reaches row 1, column 2 of grid?

A. grid[2][1]
B. grid(1, 2)
C. grid[1, 2]
D. grid[1][2]

3. What does Arrays.toString(data) return?

A. The number of elements
B. A String like [7, 2, 9, 4]
C. void; it prints by itself
D. The sorted array

4. After Arrays.sort(nums); runs, what is true?

A. nums itself is now in ascending order
B. nums is reversed
C. nums is cleared to zeros
D. nums is unchanged; a new array is returned

Practice Questions

Work through all ten questions. They are ordered from easiest to hardest. Write your answers on paper before checking the Answer Key.

1. int[] t = new int[3]; runs. What is t[0] immediately after that line?

A. A crash occurs
B. It has no value yet
C. 0
D. 3

2. int[] nums = {2, 4, 6}; runs. What is nums.length + nums[2]?

A. 8
B. 12
C. The expression does not compile
D. 9

3. Which line correctly creates an array of five ints?

A. int a = new int[5];
B. int[] a = new int[5];
C. int[] a = new int;
D. int[5] a = new int[];

4. Which loop safely doubles every element of arr?

A. for (int i = 0; i < arr.length; i++) { arr[i] = arr[i] * 2; }
B. for (int x : arr) { x = x * 2; }
C. for (int x : arr) { arr[x] = 0; }
D. while (true) { arr[0] = 0; }

5. int[] lows = {-8, -3, -15}; runs, and the max algorithm starts at lows[0]. What does it end with?

A. -15
B. 0
C. -8
D. -3

6. An array can grow longer when you add more values to it.

[ ] True [ ] False

7. What is the exact output of this program?

```
public class Trace7 {

    public static void main(String[] args) {
        int[] a = {1, 2, 3, 4};
        int total = 0;
        for (int i = 0; i < a.length; i++) {
            a[i] = a[i] * 2;
        }
        for (int x : a) {
            total = total + x;
        }
        System.out.println(total);
        System.out.println(a[1]);
    }
}
```

Output:

8. This program compiles but crashes when it runs. Find the bug.

```
public class BuggyBounds {

    public static void main(String[] args) {
        int[] days = {31, 28, 31, 30};
        for (int i = 0; i <= days.length; i++) {
            System.out.println(days[i]);
        }
    }
}
```

Bug:

9. In two or three sentences, explain why Arrays.sort returns void and how a program should display a sorted array.

10. Write a complete program called Totaler with a method public static int sum(int[] arr) that returns the total of every element. main should create the array {4, 9, 1}, call the method, and print the result.

Mixed Quiz

Fifteen questions mixing everything from this chapter. Score 12 or more before moving on to the challenges.

1. The first element of any Java array lives at index...

A. -1 **B.** 0

C. It depends on the type **D.** 1

2. int[] tags = new int[9]; runs. tags.length equals...

A. 10 **B.** 8

C. 0 **D.** 9

3. Which line creates an array that starts as {3, 1, 4}?

A. int[] fib = {3, 1, 4}; **B.** int[] fib = new int[3, 1, 4];

C. int fib = {3, 1, 4}; **D.** int[] fib = {3 1 4};

4. In grid[r][c], which index chooses the row?

A. Neither; rows are chosen by length **B.** Either; they are equal

C. The second one, c **D.** The first one, r

5. for (double d : prices) requires that prices be...

A. an array of doubles **B.** any array at all

C. an int array **D.** a 2D array

6. What does Arrays.fill(data, 7) do?

A. Adds 7 to every element **B.** Resizes data to 7 elements

C. Stores 7 into every element of data **D.** Returns a new array of 7s

7. A search method returns -1. The most likely meaning is...

A. the array has one element **B.** the target is first

C. the target was not found **D.** the search found one match

8. Which of these is a runtime error rather than a compile-time error?

A. Calling Arrays.sort without importing Arrays **B.** Reading arr[10] of a 3-element array

C. Forgetting a semicolon **D.** Writing arr.length()

9. arr.length needs parentheses because it is a method.

[ ] True [ ] False

10. Arrays.sort changes the array you pass in and returns nothing.

[ ] True [ ] False

11. For int[][] grid = new int[2][5];, grid.length is 2.

[ ] True [ ] False

12. What is the exact output of this program?

```
public class QuizGrid {

    public static void main(String[] args) {
        int[][] g = {{1, 2}, {3, 4}};
        for (int r = 0; r < g.length; r++) {
            for (int c = 0; c < g[r].length; c++) {
                System.out.print(g[r][c]);
            }
            System.out.println();
        }
    }
}
```

Output:

13. This program does not compile. Find the bug.

```
import java.util.Arrays;

public class SortMixUp {

    public static void main(String[] args) {
        int[] nums = {5, 1, 9};
        System.out.println(Arrays.sort(nums));
    }
}
```

Bug:

14. Explain in two or three sentences why starting the max algorithm at 0 fails for arrays of all-negative numbers, and state the safe starting value.

15. Write a complete program called Finder with a method public static int find(int[] arr, int target) that returns the target's index or -1 when it is absent. main should search {8, 15, 23} for 15 and for 99, printing both results.

Challenges

Five harder thinking problems. Each includes a hint; full step-by-step solutions are in the Answer Key. Try the problem seriously before reading the hint.

Challenge 7.1 — Digit Counter

Write a program called DigitCounter that counts how many times each digit appears in the number 7721237. Use an int array of ten counters (one per digit 0-9), peel digits off with % 10 and / 10 from Chapter 3, then print a line for every digit that appeared at least once.

Hint: The expression number % 10 gives the last digit, and number / 10 removes it. That digit is also the right index into your counter array.

Challenge 7.2 — Reverse the List

Write a program called ReverseInput that asks the user for 5 whole numbers with Scanner, stores them in an array, and then prints them in reverse order, one per line, under the heading In reverse order:

Hint: Fill the array with a forward loop from 0 to length - 1, then print it with a backward loop from length - 1 down to 0, decreasing i each time.

Challenge 7.3 — Second Largest

Write a program called SecondLargest that finds the two biggest values in {12, 47, 3, 47, 30} in a single pass. Keep two trackers, best and second, and print Largest: 47 and Second largest: 30. Note the array contains 47 twice; the second largest must be a strictly smaller value.

Hint: When a new value beats best, the old best is demoted to second. Values that do not beat best may still beat second, but only if they are not equal to best.

Challenge 7.4 — Grade Buckets

Write a program called GradeBuckets that counts how many of the scores {95, 72, 88, 61, 55, 100, 83, 47} fall into each bucket: A (90 or more), B (80-89), C (70-79), D (60-69), and F (below 60). Use an int array of five counters and the if/else-if pattern from Chapter 4, then print one line per letter.

Hint: One array of five counters replaces five separate variables. Each score lands in exactly one bucket, so the checks must run from highest to lowest with else-if.

Challenge 7.5 — Magic Square Checker

A magic square is a 3x3 grid whose rows, columns, and two diagonals all sum to the same value. Write a program called MagicSquare that checks the grid {{8, 1, 6}, {3, 5, 7}, {4, 9, 2}} and prints Magic square: true when every one of the eight lines sums to 15, or false otherwise.

Hint: Use a boolean named magic that starts true, and let any wrong sum switch it to false. Three row sums, three column sums, and two diagonal sums must all be checked.

Chapter Recap

- An array is a fixed-size row of same-type values; create it with new int[n] or fill it at birth with an initializer list like {9, 12, 15}.
- Valid indexes run from 0 to length - 1; anything else stops the program with ArrayIndexOutOfBoundsException at runtime.
- .length is a field, written with no parentheses; fresh arrays start filled with 0, 0.0, false, the null character, or null.
- Index loops (i < arr.length) can do anything; for-each (for (int x : arr)) reads values only, because x is a copy.
- Average with (double) sum / arr.length so integer division does not destroy the decimals.
- Classic algorithms: max and min start at arr[0]; linear search returns the index or -1; counting uses a counter; reversal swaps ends inward with a temp, halfway through.

- 2D arrays use grid[r][c] with the row first; grid.length counts rows, grid[r].length counts columns, and nested loops walk the grid row by row.
- java.util.Arrays provides toString, sort, and fill; sort changes the array in place and returns void; import the class before using it.

Common Mistakes

Beginners run into these again and again. Read them now to avoid the pain later.

WARNING

Writing arr.length() with parentheses

length is a field, not a method, so the parentheses turn a working expression into a cannot find symbol or incompatible types compile error.

Fix: Write arr.length with no parentheses anywhere you need the array's size.

WARNING

Looping with i <= arr.length

The last valid index is length - 1, so <= lets the loop touch one box too many. The prints succeed until the final pass, which stops the program with ArrayIndexOutOfBoundsException.

Fix: Use the standard shape for (int i = 0; i < arr.length; i++) and double-check every loop that touches an array.

WARNING

Starting max at 0 for arrays that may hold negatives

Zero never appears in {-8, -3, -15}, so no value beats the tracker and the algorithm reports 0, a number that is not even in the array.

Fix: Initialize max (and min) with the first element, arr[0], then scan from index 1 onward.

WARNING

Printing the result of Arrays.sort

sort returns void because it edits the array in place, so System.out.println(Arrays.sort(nums)); does not compile.

Fix: Call Arrays.sort(nums); on its own line, then print nums with Arrays.toString(nums) or a loop.

WARNING

Using an array variable that was never created

Declaring int[] arr; only names a variable; it creates no boxes. Storing arr[0] = 5; before any new fails, and the compiler reports that arr might not have been initialized.

Fix: Always create the array before using it: int[] arr = new int[10]; or an initializer list.

CHAPTER 8

Working with Strings

Text is everywhere in programs: names, messages, file names, and search boxes all hold strings. So far you have stored strings, printed them, and glued them together with +; in this chapter you learn to measure strings, take them apart, compare them, and format them like a professional. You will also meet StringBuilder, the right tool for building long text piece by piece. By the end you will be able to write small text-processing programs such as vowel counters, word counters, and palindrome checkers.

In this chapter you will learn to:

- use the everyday String methods: **length**, **charAt**, **substring**, **indexOf**, **toUpperCase**, and **trim**
- compare strings safely with **equals** and explain why == is a trap
- explain that strings are immutable and store the new strings that methods return
- format neat output with **System.out.printf** and format specifiers
- build long text efficiently with **StringBuilder**

Lesson 8.1 — Useful String Methods

A String value is an **object**, and objects come with **methods**: built-in actions you trigger with a dot, a name, and parentheses. You have been calling methods all along without noticing: println is a method of System.out. String methods answer questions about the text or produce a new version of it. Write the string variable, a dot, the method name, and the arguments inside the parentheses, exactly like the method calls you learned to write yourself in Chapter 6.

The most basic question is “how many characters?” In Chapter 7 you measured arrays with a field: **nums.length**, no parentheses. Strings look similar but work differently: **word.length()** is a method, so the parentheses are required. Writing word.length without them does not compile, and writing nums.length() on an array does not compile either. This little pair of dots and parentheses trips everyone exactly once; now you know why.

```
public class MethodTour {

    public static void main(String[] args) {
        String word = "Elephant";
        System.out.println(word.length());
        System.out.println(word.charAt(0));
        System.out.println(word.charAt(word.length() - 1));
    }
}
```

Output: 8, then E, then t (the first and last letters)

charAt(i) answers a single-character question: what sits at index i? Indexes start at 0, so for “Elephant” charAt(0) is 'E' and charAt(7) is 't'. Character results are char values, so they wear single quotes. The last valid index is always s.length() - 1, a formula worth memorizing. Asking for an index that does not exist, such as

charAt(8) here, stops the program with an error; Chapter 9 shows how to handle such problems gracefully.

substring carves out a piece of the string. With one argument, as in substring(2), you get everything from that index to the end. With two arguments, as in substring(2, 5), you get the characters from the first index up to but **not including** the second one; the end index is exclusive. Meanwhile **indexOf** searches: it returns the index where a piece of text first appears, or the special value **-1** when the piece is nowhere in the string.

```
public class SliceDemo {

    public static void main(String[] args) {
        String word = "Elephant";
        System.out.println(word.substring(2));
        System.out.println(word.substring(2, 5));
        System.out.println(word.indexOf("ant"));
        System.out.println(word.indexOf("z"));
    }
}
```

Output: ephant, then eph, then 5, then -1 (not found)

Method	It answers	On "Elephant"
length()	how many characters are there?	8
charAt(3)	which character is at index 3?	'p'
substring(2, 5)	characters from index 2 up to (not including) 5	"eph"
indexOf("ant")	where does "ant" start? (-1 if absent)	5
toUpperCase()	same text, every letter a capital	"ELEPHANT"
trim()	text without spaces at the ends	"Elephant"

Two more handy methods round out the set. **toUpperCase** and **toLowerCase** build a copy of the text with every letter switched, which makes comparisons fair no matter how the user typed. **trim** builds a copy with leading and trailing spaces removed, perfect for cleaning up keyboard input. Both leave the middle of the text exactly as it was, and neither ever touches the original string; the next lesson explains that surprising fact in detail.

```
public class CaseDemo {

    public static void main(String[] args) {
        String messy = "  Hello There  ";
        System.out.println(messy.trim());
        System.out.println(messy.toUpperCase());
        System.out.println(messy.toLowerCase());
        System.out.println(messy.length());
        System.out.println(messy.trim().length());
    }
}
```

Output: the trimmed text, then ALL CAPS (spaces kept), then lowercase, then 15, then 11

TIP

Methods apply to any string expression, not only to variables: messy.trim().length() trims first and then measures the result. Java simply works left to right, the same way it evaluates 2 + 3 * 4 step by step.

WARNING
Calling a String method never changes the original string. If you want the new version, you must store it in a variable, as the next lesson shows. Also remember that charAt with an index of length() or more crashes the program, so double-check index math inside loops.

Lesson Check 8.1

Choose the one best answer for each question. Answers are in the Answer Key.

1. Which expression counts the characters of a String variable s?

A. s.size()
B. s.length
C. s.length()
D. length(s)

2. What does "Elephant".charAt(3) return?

A. 'p'
B. 'e'
C. 'E'
D. 'h'

3. What does "Elephant".substring(2, 5) return?

A. "phant"
B. "leph"
C. "ephant"
D. "eph"

4. What does "notebook".indexOf("o") return?

A. 0
B. 1
C. 5
D. -1

Lesson 8.2 — Comparing Strings and Immutability

Comparing strings is something every program does, and Java hides a famous trap here. For numbers, == compares values: 5 == 5 is true. But strings are objects, and for objects == asks a different question: are these two variables pointing at the **same object** in memory? Two separate String objects can hold exactly the same letters, and == will still call them different, because it never looks at the letters at all.

```
public class EqualTrap {

    public static void main(String[] args) {
        String a = new String("cat");
        String b = new String("cat");
        System.out.println(a == b);
        System.out.println(a.equals(b));
    }
}
```

Output: false, then true - same letters, two objects

The honest example above builds two separate objects with new String, and == really does print false. You may occasionally see == print true for two identical literals, because Java secretly shares one copy of each literal between variables; that sharing is an optimization, not a promise. The rule that never fails: to compare

what strings **say**, use the **equals** method, which walks through the characters. The == trap is silent, no error message, just wrong answers.

```
public class GuessTheCode {

    public static void main(String[] args) {
        String secret = "OPEN";
        String guess = "open";
        System.out.println(secret.equals(guess));
        System.out.println(secret.equalsIgnoreCase(guess));
    }
}
```

Output: false, then true - equals cares about case, equalsIgnoreCase does not

equalsIgnoreCase is equals with the case difference forgiven, ideal for user input where OPEN, open, and Open all mean the same word. For ordering, **compareTo** compares the two strings in dictionary order and returns a negative number if the first string comes earlier, zero if the strings are equal, and a positive number if it comes later. Never memorize the exact numbers; only the sign matters. This is the same logic that Arrays.sort from Chapter 7 uses to put strings in alphabetical order.

Now the deepest fact of the chapter: **strings are immutable**. Once a String object exists, its characters can never change. Every String method you met - toUpperCase, trim, substring, all of them - builds and returns a **brand-new string** and leaves the original untouched. That is why s.toUpperCase() on its own does nothing visible: Java creates the new uppercase string, and since you never store it, the result is simply thrown away.

```
public class ImmutabilityTrap {

    public static void main(String[] args) {
        String s = "whisper";
        s.toUpperCase();
        System.out.println(s);
        s = s.toUpperCase();
        System.out.println(s);
    }
}
```

Output: whisper, then WHISPER - only the assignment changed s

The fix is always the same: **store the result**. Writing s = s.toUpperCase() points s at the new string; the old one is recycled automatically. The same pattern works for every method: s = s.trim(), s = s.substring(1), and so on. One last everyday check: to ask whether a string is empty (holds no characters at all), compare its length to zero with s.length() == 0. An empty string is a real string, just one with zero characters, and it is different from a string holding spaces.

WARNING
== between strings compiles and runs without any error message, which is exactly why it is dangerous. The bug appears only as wrong behavior, often on some runs and not others. Make a habit today: strings are compared with equals, point.

TIP

When a String method result seems to vanish, look for the missing assignment. The compiler accepts s.toUpperCase(); happily, because calling a method and ignoring its answer is legal; the burden of storing the result is on you.

Lesson Check 8.2

Choose the one best answer for each question. Answers are in the Answer Key.

1. Two String variables may hold the same letters but be different objects. Which test compares their contents?

A. a == b **B.** a = b

C. a.length() == b.length() **D.** a.equals(b)

2. What does "Apple".compareTo("Banana") return?

A. a positive number **B.** a negative number

C. zero **D.** an error

3. After the statements String s = "day"; and s.toUpperCase(); what does s hold?

A. the code would not compile **B.** nothing; s is now empty

C. "day" **D.** "DAY"

4. Which expression is true exactly when s holds the empty string?

A. s.length() == 0 **B.** s.trim() == 0

C. s == "" **D.** s.length() == 1

Lesson 8.3 — printf and Formatting

Concatenation with + works, but the output gets ugly fast: doubles show six decimals, columns drift out of line, and long lines turn into quote-spaghetti. **System.out.printf** (print formatted) solves this. Its first argument is a **format string**: normal text with special placeholders that begin with %. Each placeholder is filled, in order, by the arguments that follow. The pairing is positional: first placeholder, first argument; second placeholder, second argument.

```
public class PrintfIntro {

    public static void main(String[] args) {
        String name = "Maya";
        int age = 12;
        double score = 91.5;
        System.out.printf("%s is %d years old%n",
                name, age);
        System.out.printf("Score: %f%n", score);
    }
}
```

Output: Maya is 12 years old, then Score: 91.500000 (six decimals by default - we will fix that)

The workhorse specifiers are **%d** for int values, **%f** for double values, and **%s** for text (and anything else). A precision such as **%.2f** sets how many decimals a double shows, rounded normally; 91.5 prints as 91.50 and 7.125 as 7.13. Between % and the letter you may also put a **width**: %5d reserves at least five character slots and right-aligns the number inside them, while a minus sign, as in %-10s, left-aligns the text in ten slots. Widths are how you print columns that line up.

```
public class PriceTag {

    public static void main(String[] args) {
        double price = 4.5;
        int count = 3;
        System.out.printf("%d x item%n", count);
        System.out.printf("Total: %.2f%n",
                price * count);
        System.out.printf("[%10s]%n", "hi");
        System.out.printf("[%-10s]%n", "hi");
    }
}
```

Output: 3 x item, Total: 13.50, [hi], then [hi] - same width, opposite alignment

Specifier	Fills the slot with	Example result
%d	an int value	%d with 42 gives 42
%f	a double, six decimals by default	%f with 2.5 gives 2.500000
%.2f	a double rounded to 2 decimals	%.2f with 2.5 gives 2.50
%s	text or any value as text	%s with "hi" gives hi
%5d	an int right-aligned in width 5	%5d with 42 gives ' 42'
%-10s	text left-aligned in width 10	%-10s with "hi" gives 'hi '
%n	a new line (or use \n)	ends the line

One difference from println matters every single time: **printf does not add a newline automatically**. It prints exactly what the format string says and leaves the output position right there. If you forget the %n (or \n), the next print continues on the same line and your output glues together. End each format string with %n whenever a line should end, and remember that %n and \n are interchangeable at this level.

```
public class NoNewline {

    public static void main(String[] args) {
        System.out.printf("A");
        System.out.printf("B%n");
        System.out.printf("C");
        System.out.println("D");
    }
}
```

Output: AB on the first line, CD on the second - the A line only ended because B carried %n

WARNING

If the type of an argument does not match its placeholder - for example passing a double to %d - the program compiles but stops at runtime with a format error. Match your placeholders to your arguments: one slot per value, right type, in order.

NOTE

printf comes from the C language, where it has been the standard formatter for decades, which is why the specifiers look cryptic. The %n specifier writes whatever newline your operating system prefers; \n always writes the plain newline character, and for console programs you will not notice the difference.

Lesson Check 8.3

Choose the one best answer for each question. Answers are in the Answer Key.

1. Which specifier prints an int value inside a printf format string?

A. %d **B.** %n
C. %f **D.** %s

2. What does System.out.printf("Total: %.2f%n", 7.125); print?

A. Total: 7.12 **B.** Total: 7.125
C. Total: 7.13 **D.** Total: 7.125000

3. How is printf different from println?

A. printf needs no arguments at all **B.** printf cannot print text, only numbers
C. printf is slower, so avoid it **D.** printf does not start a new line by itself

4. What does System.out.printf("[%-6s]%n", "hi"); print?

A. [hi] **B.** [hi]
C. [hi] **D.** [hi]

Lesson 8.4 — StringBuilder

Here is a puzzle: strings are immutable, so what really happens when a loop runs stars = stars + "*" a thousand times? Because no string can ever change, every single + builds a **whole new string** and copies everything accumulated so far into it. The first copy holds 1 character, the next 2, then 3, and so on - about half a million character copies for a thousand stars. The program works, but it quietly wastes time and memory doing all that re-copying.

```
public class SlowStars {

    public static void main(String[] args) {
        String stars = "";
        for (int i = 0; i < 5; i++) {
            stars = stars + "*";
        }
        System.out.println(stars);
    }
}
```

*Correct but wasteful: each pass rebuilds the entire string. Output: *****

Java's answer is **StringBuilder**, a class that holds text in a private, growable buffer. Unlike String, a StringBuilder **can** be changed in place: its **append** method tacks new text onto the end without recopying the whole buffer, so building a huge text takes one pass instead of thousands of copies. The class lives in java.lang, the same package as String and System, so it needs no import. When you are done, **toString** hands you an ordinary String, and **length** reports how many characters the buffer holds.

```
public class FastStars {

    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < 5; i++) {
            sb.append("*");
        }
        String stars = sb.toString();
        System.out.println(stars);
        System.out.println(sb.length());
    }
}
```

*Output: *****, then 5 - one buffer, changed in place*

StringBuilder has a few party tricks too. **reverse** flips the order of every character in the buffer, which makes palindrome checking almost embarrassingly easy. And append returns the StringBuilder itself, so calls can be chained: sb.append("Ja").append("va") appends both pieces in one statement. Chaining is a style you will recognize in real code, but separate statements are just as correct and often easier to read while you are learning.

```
public class ReverseDemo {

    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("abcdef");
        sb.reverse();
        System.out.println(sb.toString());
    }
}
```

Output: fedcba

TIP
Rule of thumb: if you are building text inside a loop, or the text could grow large, use StringBuilder with append. For joining two or three pieces once, plain + is perfectly fine and easier to read - Java's tools are optimized for the common case too.

NOTE
A StringBuilder is not a String and cannot be assigned to a String variable directly. Always call toString() at the end: String result = sb.toString();. Mixing the two types is a common compile error with a one-word fix.

Lesson Check 8.4

Choose the one best answer for each question. Answers are in the Answer Key.

1. Why is StringBuilder faster than + for building text in a long loop?

A. It changes one buffer instead of making a new string each pass

B. It can hold numbers as well as text

C. It prints text faster than println

D. It never needs a loop

2. A StringBuilder named sb holds "hello". What does sb.length() return?

A. 0 **B.** 6

C. 5 **D.** 4

3. Which method turns a StringBuilder back into an ordinary String?

A. asString() **B.** convert()

C. toText() **D.** toString()

4. After new StringBuilder("abc") and a call to reverse(), what does sb.toString() return?

A. "" **B.** "cba"

C. "abc" **D.** "bca"

Practice Questions

Work through all ten questions. They are ordered from easiest to hardest. Write your answers on paper before checking the Answer Key.

1. What does "banana".indexOf("an") return?

A. 1 **B.** 2

C. 3 **D.** -1

2. Which statement about String methods such as toUpperCase and trim is true?

A. They delete the string when finished **B.** They edit the original string in place

C. They only work on single characters **D.** They return new strings and leave the original alone

3. Which comparison asks whether two strings hold exactly the same letters, case included?

A. s.equalsIgnoreCase(t) **B.** s.equals(t)

C. s.compareTo(t) == 1 **D.** s == t

4. In a printf format string, what does %s do?

A. Starts a new line **B.** Rounds a number to two decimals

C. Fills the slot with the next argument as text **D.** Prints the letter s literally

5. Which method adds a piece of text to the end of a StringBuilder?

A. add **B.** concat

C. insert **D.** append

6. Calling s.trim() changes the string stored in s by removing its surrounding spaces.

[ ] True [ ] False

7. What is the exact output of this program?

```
public class Practice8 {
    public static void main(String[] args) {
        String word = "banana";
        System.out.println(word.substring(1, 4));
        System.out.println(word.indexOf("a"));
        System.out.println(word.toUpperCase());
        System.out.println(word);
    }
}
```

Output:

8. This program compiles and runs, but prints the wrong thing. Find the bug.

```
public class BugHunt8 {
    public static void main(String[] args) {
        String a = new String("cat");
        String b = new String("cat");
        if (a == b) {
            System.out.println("Same word");
        } else {
            System.out.println("Different words");
        }
    }
}
```

Bug:

9. In two or three sentences, explain what it means that strings are immutable, and what you must do to keep the result of a method such as toUpperCase.

10. Write a complete program called Shout that stores a name in a String variable, then prints three lines: the name in capital letters, the number of characters it has, and its first character. Use toUpperCase, length, and charAt.

Mixed Quiz

Fifteen questions mixing everything from this chapter. Score 12 or more before moving on to the challenges.

1. What does "Java".charAt(1) return?

A. 'J'
B. 'v'
C. 'a'
D. 'av'

2. What does "Elephant".substring(2) return?

A. "phant" **B.** "Ele"

C. "eph" **D.** "ephant"

3. What does trim() do to " hi "?

A. Removes the spaces at the start and end only **B.** Removes every space in the string

C. Converts the spaces to hyphens **D.** Removes the last character

4. What does "OPEN".equalsIgnoreCase("open") return?

A. -1 **B.** true

C. "OPEN" **D.** false

5. When does s.compareTo(t) return 0?

A. When s is shorter than t **B.** When the two strings are equal

C. It never returns 0 **D.** When t comes first in the dictionary

6. Which line prints a double named price with exactly two decimals?

A. System.out.printf("%2d%n", price); **B.** System.out.printf("%f %f%n", price);

C. System.out.printf("%.2f%n", price); **D.** System.out.printf("%s%n", price);

7. Which call prints [cat] - the word right-aligned inside width 6?

A. System.out.printf("[%.6s]%n", "cat"); **B.** System.out.printf("[%-6s]%n", "cat");

C. System.out.printf("[6s]%n", "cat"); **D.** System.out.printf("[%6s]%n", "cat");

8. String s = "hi"; then s.toUpperCase(); runs, then System.out.println(s);. What prints?

A. hi **B.** HI

C. Hi **D.** nothing

9. The expression "abc".substring(1, 2) returns the one-letter string "b".

[ ] True [ ] False

10. System.out.printf starts a new line automatically after printing, just like println does.

[ ] True [ ] False

11. indexOf returns -1 when the piece it searches for is not in the string.

[ ] True [ ] False

12. What is the exact output of this program?

```
public class Quiz8Out {
    public static void main(String[] args) {
        System.out.printf("Items: %d%n", 3);
        System.out.printf("Price: %.2f%n", 9.5);
        System.out.printf("[%6s]%n", "cat");
    }
}
```

Output:

13. This program compiles and runs, but the output is not what the programmer expected. Find the bug.

```
public class ShoutBug {
    public static void main(String[] args) {
        String word = "quiet";
        word.toUpperCase();
        System.out.println(word);
    }
}
```

Bug:

14. A program builds one long report out of thousands of small pieces inside a loop. Explain why repeated + concatenation is a poor choice, and which class solves the problem - name its most important method.

15. Write a complete program called LastHalf that stores the word Elephant in a String and prints the second half of the word. Compute the starting index with length and division so the program works for any even-length word.

Challenges

Five harder thinking problems. Each includes a hint; full step-by-step solutions are in the Answer Key. Try the problem seriously before reading the hint.

Challenge 8.1 — Vowel Counter

Write a program called VowelCount that stores a sentence in a String, loops over every character with charAt, and prints how many vowels it contains. Count a, e, i, o, and u in both upper and lower case - the easiest way is to convert the sentence to lowercase once before the loop. For the sentence “Java Is Fun To Learn” the answer is 7.

Hint: Convert with toLowerCase and store the result. Inside the loop, one if with several || conditions compares the current char to each vowel, all written in single quotes.

Challenge 8.2 — Initials Extractor

Write a program called Initials that stores a first name, a middle name, and a last name in three String variables, then prints the initials separated by dots, like M.C.R. Use charAt(0) to pull each first letter.

Hint: A char glued to a String with + becomes part of the text, so 'M' + "." builds "M." - exactly what you need between the letters.

Challenge 8.3 — Palindrome Checker

A palindrome reads the same forwards and backwards, like noon or level. Below is a broken attempt: it compiles and runs, but it always prints Not a palindrome, even for noon. Find and fix the bug, then finish the program so it asks the user for a word with Scanner and reports whether it is a palindrome. Tip: StringBuilder.reverse() makes the backwards copy in two lines.

```
public class PalindromeCheck {
    public static void main(String[] args) {
        String word = "noon";
        String backwards = "";
        for (int i = word.length() - 1; i >= 0; i--) {
            backwards += word.charAt(i);
        }
        if (word == backwards) {
            System.out.println("Palindrome!");
        } else {
            System.out.println("Not a palindrome");
        }
    }
}
```

Hint: The backwards string is built at runtime, so it is a brand-new object. Which comparison looks at letters, == or equals?

Challenge 8.4 — Word Counter

Write a program called WordCount that stores a sentence in a String and prints how many words it holds. For this challenge, words are separated by single spaces, so the number of words is the number of spaces plus one - unless the sentence is empty (length 0), which has 0 words. Loop with charAt to count the spaces. The sentence “the quick brown fox jumps” has 5 words.

Hint: Compare each character to the space character ' ' with ==. Handle the empty sentence with a plain if before adding the final +1.

Challenge 8.5 — Acronym Builder

Write a program called Acronym that stores a phrase in a String array, one word per slot, and builds the acronym: the first letter of every word joined together. Use a StringBuilder and append, then print the acronym in capital letters. For {"personal", "digital", "assistant"} the output is PDA.

Hint: A for-each loop visits every word in the array. Inside the loop, append w.charAt(0) to the builder; at the end, toString plus toUpperCase finish the job.

Chapter Recap

- Strings are objects, so they carry methods you call with a dot and parentheses: s.length(), s.charAt(i), s.substring(start, end), s.indexOf(text), s.toUpperCase(), s.trim().
- Arrays use a .length field without parentheses; Strings use a .length() method with them. Mixing the two forms is a classic compile error.
- Indexes start at 0 and the last valid index is s.length() - 1. substring's end index is exclusive: substring(2, 5) takes indexes 2, 3, and 4. indexOf returns -1 when the piece is not found.
- To compare contents use s.equals(t) (or equalsIgnoreCase for case-insensitive tests); == compares object references and is a silent trap, even though it sometimes accidentally says true for shared literals.
- compareTo returns a negative number, zero, or a positive number depending on dictionary order; zero means the strings are equal.

- Strings are immutable: every method returns a new string, so store the result (s = s.toUpperCase();) or it is lost. Check for the empty string with s.length() == 0.
- System.out.printf formats with placeholders: %d for ints, %f for doubles, %.2f for two decimals, %s for text, %5d and %-10s for widths. printf does not add a newline by itself - end formats with %n or \n.
- StringBuilder builds text in a changeable buffer: append adds pieces cheaply, toString converts back to String, length counts, and reverse flips the text. Prefer it for building text inside loops.

Common Mistakes

Beginners run into these again and again. Read them now to avoid the pain later.

WARNING

Comparing strings with ==

For objects, == asks whether two variables point at the same object, not whether the letters match. The program compiles and runs, so nothing warns you - some comparisons just come out wrong, depending on whether Java happened to share the literal.

Fix: Compare contents with equals (or equalsIgnoreCase when case should not matter): if (a.equals(b)). Reserve == for primitives like int and char.

WARNING

Discarding the result of a String method

Writing s.toUpperCase(); or s.trim(); on its own line compiles fine, but because strings are immutable the new string is created and then thrown away, and s keeps its old value. Beginners see “the method did nothing” when it did plenty - into thin air.

Fix: Store the result: s = s.toUpperCase();. Every String method that promises a changed version returns it, and an assignment is the only way to keep it.

WARNING

Thinking substring's end index is included

substring(0, 3) feels like it should give three letters from index 0 to 3, but the end is exclusive, so it gives indexes 0, 1, and 2. The off-by-one shows up as one extra or one missing character.

Fix: Read substring(start, end) as “from start up to but not including end”. The length of the result is always end - start.

WARNING

Mixing up s.length and s.length()

Arrays use the field nums.length with no parentheses; strings use the method s.length() with them. After a week of arrays, fingers keep typing the array form, and the compiler answers with a confusing cannot-find-symbol error.

Fix: For a String, always add the parentheses: word.length(). For an array, never add them: nums.length.

WARNING

Walking off the end with charAt
A loop that keeps going until i equals s.length() eventually asks for charAt(s.length()), an index that does not exist, and the program stops with an error at runtime - often only for certain inputs, which makes it look random.
Fix: Stop the loop while i is still below s.length(), and remember the last valid index is s.length() - 1. Chapter 9 shows how to catch such errors gracefully when they happen anyway.

CHAPTER 9

Files and Exceptions

Real programs deal with the outside world: they read data from files, write results back, and survive the many small things that can go wrong along the way - a missing file, a user typing letters where numbers belong, a division by zero. This chapter teaches the two tools every Java program uses for that: exceptions, which describe problems while the program runs, and the file classes, which connect your code to the disk. You will learn to catch trouble gracefully, to throw signals of your own, and to read and write text files.

In this chapter you will learn to:

- guard risky statements with **try** and respond in **catch** blocks
- use **finally** for cleanup and **throw** to signal problems from your own methods
- read text files with **File** and **Scanner**, line by line or number by number
- write and append text files with **PrintWriter** and **FileWriter**
- use **try-with-resources** so files close themselves, and explain checked versus unchecked exceptions

Lesson 9.1 — Exceptions: try and catch

Chapter 1 called them runtime errors: problems that appear while the program is running and normally stop it with a red report. Java's name for the report is an **exception**, and the name is precise - an exception is an **object** that describes what went wrong. One object describes a division by zero, another a bad array index, another a missing file. When an exception is created and nobody deals with it, the program stops right there; the statements after the crash never run.

```
public class CrashDemo {

    public static void main(String[] args) {
        int zero = 0;
        System.out.println("Before");
        int result = 7 / zero;
        System.out.println("After");
    }
}
```

Output: Before, then the program stops with an ArithmeticException report; After never prints

Instead of letting the program die, you can be ready for trouble. Wrap the risky statements in a **try block**, and follow it with a **catch block** naming the type of exception you are prepared for. The catch block receives the exception object as a parameter, usually called e, and its statements run only if that kind of problem actually happens. If nothing goes wrong, the catch block is skipped completely. Either way, the program continues with the first statement after the whole try/catch.

```
public class SafeDivide {

    public static void main(String[] args) {
        int zero = 0;
        System.out.println("Before");
        try {
            int result = 7 / zero;
            System.out.println(result);
        } catch (ArithmeticException e) {
            System.out.println("Problem: "
                    + e.getMessage());
        }
        System.out.println("After");
    }
}
```

Output: Before, Problem: / by zero, After - and the program reaches its end normally

The exception object carries useful information, and **e.getMessage()** returns it as a short text - read it whenever you are not sure what happened. One try block may be followed by several catch blocks, one per exception type, and Java runs only the first one whose type matches the problem. Common types worth knowing by sight: **ArithmeticException** for integer division by zero, **ArrayIndexOutOfBoundsException** for a bad array index, **NumberFormatException** for text that cannot become a number, and **InputMismatchException** when a Scanner's nextInt() meets keyboard input that is not a number.

```
public class TwoCatches {

    public static void main(String[] args) {
        int[] nums = new int[2];
        try {
            System.out.println(nums[9]);
        } catch (ArithmeticException e) {
            System.out.println("Math problem");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Index problem");
        }
        System.out.println("Still running");
    }
}
```

The bad index fires the second catch; a division by zero would fire the first. One exception, one catch.

Exception	Typical cause
ArithmeticException	an int was divided by zero
ArrayIndexOutOfBoundsException	an array index outside the valid range
NumberFormatException	Integer.parseInt met text like "12x"
InputMismatchException	Scanner.nextInt() read input that is not a number
FileNotFoundException	opening a file that does not exist (checked!)

Where does NumberFormatException come from in your own code? The standard source is **Integer.parseInt**, a library method that turns text like "12" into the int 12; when the text is not a whole

number, it throws NumberFormatException. One piece of vocabulary before we move on: the exceptions in this lesson all belong to the **RuntimeException** family, which Java calls **unchecked** - the compiler does not force you to handle them. They usually come from logic mistakes you should fix, and the full checked/unchecked story arrives in Lesson 9.3 with files.

WARNING
A catch block only catches problems that happen **inside its own try block**. Statements before the try, or after the catch, are outside its protection - put the risky lines inside.

TIP
When a catch block feels like a black hole, print e.getMessage() while debugging. The message often names the exact trouble, such as "/ by zero", which turns guessing into reading.

Lesson Check 9.1

Choose the one best answer for each question. Answers are in the Answer Key.

1. What is an exception in Java?

A. A compile-time spelling error
B. A file the program writes to
C. A loop that never ends
D. An object that describes a runtime problem

2. Which exception does the statement int x = 7 / zero; throw when zero is an int holding 0?

A. FileNotFoundException
B. NumberFormatException
C. ArithmeticException
D. ArrayIndexOutOfBoundsException

3. What does e.getMessage() return inside a catch block?

A. A short text describing the problem
B. The name of your class
C. A fixed value so the program can go on
D. The line number where you typed the bug

4. After a catch block finishes handling an exception, what does the program do?

A. Goes back into the try block
B. Continues with the statements after the try/catch
C. Restarts from the top of main
D. Exits immediately

Lesson 9.2 — finally and throw

Some code should run whether or not trouble occurred: closing a connection, tidying up, printing a closing report. Java gives such code a home: the **finally block**, attached after try and catch. The finally block **normally runs whether or not an exception happened** - if the try finishes cleanly, finally runs; if a catch handles a problem, finally runs after the catch. One honest footnote: "normally" is not "always". If the JVM itself exits, for example through System.exit, or the whole process is killed, a pending finally block is skipped, so never put life-critical work there. Its typical use is cleanup.

```
public class FinallyDemo {

    public static void main(String[] args) {
        int[] nums = new int[2];
        try {
            System.out.println(nums[1]);
            System.out.println(nums[9]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Bad index");
        } finally {
            System.out.println("Finally ran");
        }
        System.out.println("Program continues");
    }
}
```

Output: 0 (an int array's default), then Bad index, then Finally ran, then Program continues

Run FinallyDemo with a safe index and finally still runs - that is the point. But Java also lets you **create** trouble on purpose. The **throw** statement creates an exception object and launches it immediately, like pulling a fire alarm: throw new IllegalStateException("door is open");. From that moment the rest of the method is skipped, and the exception travels to whoever called the method, where a catch block can handle it. Three everyday types: **IllegalArgumentException** when a method received a bad argument value, **IllegalStateException** when the method was called at a wrong moment, and **RuntimeException** as a general-purpose signal.

```
public class SpeedCheck {

    static void setSpeed(int speed) {
        if (speed < 0) {
            throw new IllegalArgumentException(
                    "speed cannot be negative: "
                    + speed);
        }
        System.out.println("Speed set to " + speed);
    }

    public static void main(String[] args) {
        try {
            setSpeed(50);
            setSpeed(-5);
            setSpeed(80);
        } catch (IllegalArgumentException e) {
            System.out.println("Caught: "
                    + e.getMessage());
        }
        System.out.println("Done");
    }
}
```

Output: Speed set to 50, then Caught: speed cannot be negative: -5, then Done - setSpeed(80) never runs

Notice the division of labor: the method **throws** the alarm, the caller **catches** it. That split is a design tool. The setSpeed method does not know how to fix a bad speed, so it refuses the job loudly; main, which knows the context, decides how to respond. Always include a helpful message when you throw - it costs one argument and saves minutes of debugging, because the message is exactly what getMessage() and the red

report will show.

One more keyword completes the family: **throws** (with an s) on a method signature announces that the method may produce a given exception without handling it itself: static void load() throws IllegalArgumentException. For the unchecked exceptions of this lesson, declaring throws is optional - the compiler never forces it. For some file exceptions it is not optional, which is exactly the next lesson's topic.

> **WARNING**
> A throw statement ends its method at once: every statement after it is skipped, every loop it sits in is abandoned. If you write code after a throw in the same block, it is unreachable - the same feeling as code placed after break in a loop.

> **NOTE**
> Which type should you throw? Pick the most specific one that describes the trouble: bad argument values fit IllegalArgumentException, wrong timing fits IllegalStateException. Specific names make the catch blocks of your callers precise.

Lesson Check 9.2

Choose the one best answer for each question. Answers are in the Answer Key.

1. A method contains try/catch/finally. When does the finally block normally run?

A. Only when no exception happened
B. Only when the JVM exits
C. Whether or not an exception happened
D. Only when an exception happened

2. Which line correctly throws an exception carrying a message?

A. throw new IllegalArgumentException("bad input")
B. new IllegalArgumentException("bad input")
C. throw "bad input"
D. catch (IllegalArgumentException e)

3. What does the keyword throws on a method signature announce?

A. That the method must be private
B. That the method may produce that exception
C. That the method catches everything
D. That the method is finished

4. What happens to the statements in a method after a throw statement runs?

A. They run once, then the throw happens
B. They run twice for safety
C. They become comments
D. They are skipped; the method stops at once

Lesson 9.3 — Reading Files

Files are how programs remember things after they end. Java's plan for reading text files is friendly: the **File** class represents a file location, and the Scanner you already know from the keyboard can read from a File just as easily. You need two imports - **java.io.File** and **java.util.Scanner** - and then one constructor call connects Scanner to the file instead of System.in. After that, nextLine, nextInt, and friends work exactly as they always have.

```
Scanner in = new Scanner(new File("notes.txt"),
        StandardCharsets.UTF_8);
```

The key line: a Scanner reading the file notes.txt as UTF-8 text (a fragment - see the full program below)

The second constructor argument, **StandardCharsets.UTF_8**, names the text **encoding**: the agreed system for turning bytes on the disk into characters on the screen. UTF-8 is the modern worldwide standard and covers the letters of every language, so files saved on one computer read correctly on another. It needs the import **java.nio.charset.StandardCharsets**. If you omit the encoding, Java uses the computer's default, which can differ between machines; passing UTF-8 makes your program behave the same everywhere.

```
import java.io.File;
import java.io.FileNotFoundException;
import java.nio.charset.StandardCharsets;
import java.util.Scanner;

public class ReadNotes {

    public static void main(String[] args) {
        try {
            File file = new File("notes.txt");
            Scanner in = new Scanner(file,
                    StandardCharsets.UTF_8);
            while (in.hasNextLine()) {
                String line = in.nextLine();
                System.out.println(line);
            }
            in.close();
        } catch (FileNotFoundException e) {
            System.out.println("Could not find "
                    + "notes.txt");
        }
    }
}
```

The standard shape of a file-reading program

Look closely at that catch block: the compiler **demands** it. **FileNotFoundException** is a **checked exception**: the compiler checks, before the program ever runs, that you have made a plan for the realistic possibility of a missing file. You must either catch it in a try/catch or declare it with throws on the method, and refusing both is a compile error. This is the big difference from the RuntimeException family of Lessons 9.1 and 9.2, which the compiler never forces you to mention.

```
// The other legal form: declare it, do not catch it.
public static void main(String[] args)
        throws FileNotFoundException {
    // ... method body here ...
}
```

With throws, a missing file would stop the program with the standard red report instead of your message

Now the loop that reads a whole file: while (in.hasNextLine()) asks “is there at least one more line?”, and nextLine() fetches it. The loop ends exactly at the last line, with no counting and no guessing. For files of numbers, the same pattern works with **hasNextInt** and **nextInt**: read while a next int exists. Also know where your files live: a plain name like "notes.txt" is a **relative path**, and Java resolves it against the **current working directory** - the folder the program was launched from. In most IDEs that is your project folder, so save data files there.

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class ReadNumbers {

    public static void main(String[] args) {
        int total = 0;
        int count = 0;
        try {
            Scanner in = new Scanner(new File(
                    "scores.txt"));
            while (in.hasNextInt()) {
                total = total + in.nextInt();
                count++;
            }
            in.close();
            if (count > 0) {
                double average =
                        (double) total / count;
                System.out.println("Average: "
                        + average);
            }
        } catch (FileNotFoundException e) {
            System.out.println("scores.txt was not "
                    + "found");
        }
    }
}
```

Reads integers from scores.txt until the file runs out, then reports their average

WARNING

A missing file is not a compile error that points at your constructor line - the code compiles fine, and the FileNotFoundException appears only at runtime if the file is truly absent. When your message prints, first check the working directory: the file is usually one folder away from where Java is looking.

NOTE

Closing the Scanner with in.close() disconnects it from the file when you are done. It is a good habit even though small console programs survive without it - and in the next lesson, writing files, closing becomes genuinely important.

Lesson Check 9.3

Choose the one best answer for each question. Answers are in the Answer Key.

1. Opening a file with new Scanner(new File("notes.txt")) forces you to handle which exception?

A. IllegalStateException
B. ArithmeticException
C. InputMismatchException
D. FileNotFoundException

2. Why is FileNotFoundException called a checked exception?

A. Java checks the file for viruses first
B. The compiler forces you to catch it or declare throws
C. The Scanner re-checks it after every read
D. It only happens in debug mode

3. What does in.hasNextLine() return when every line of the file has been read?

A. It throws an exception **B.** -1

C. false **D.** true

4. The program opens "notes.txt" with a plain relative name. Where does Java look for it?

A. In the current working directory **B.** Everywhere on the disk

C. On the desktop **D.** In the folder where Java is installed

Lesson 9.4 — Writing and Appending Files

For writing text, Java pairs two classes: **FileWriter** knows how to open a file on the disk for writing, and **PrintWriter** gives it the comfortable methods you love - print, println, even printf. You build them together: a PrintWriter wrapped around a FileWriter, with the encoding named as before. This constructor pair arrived in Java 11, so any modern installation has it. From then on, every println sends a line into the file instead of onto the screen.

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.charset.StandardCharsets;

public class WriteDiary {

    public static void main(String[] args) {
        try (PrintWriter out = new PrintWriter(
                new FileWriter("diary.txt",
                        StandardCharsets.UTF_8))) {
            out.println("Dear diary,");
            out.println("Today I learned to save files.");
            out.printf("Day %d of Java%n", 9);
        } catch (IOException e) {
            System.out.println("Writing failed: "
                    + e.getMessage());
        }
        System.out.println("diary.txt is ready");
    }
}
```

After this runs, open diary.txt: the three lines are inside the file, not on the screen

That try with a pair of parentheses is **try-with-resources**, the modern standard for files. The variable declared in the header - here **out** - exists exactly for the length of the block: the file is **closed automatically** at the end of the block, even if an exception happened, and the variable is **out of scope** afterward, so it cannot be used outside the try block. The older alternative declares the variable first and calls out.close() yourself at the end - workable, but one forgotten close ruins everything, so prefer try-with-resources from day one.

```
// Old style, for recognizing it in other people's code:
PrintWriter out = new PrintWriter(
        new FileWriter("log.txt"));
out.println("saved");
out.close();   // also flushes the buffer to the disk
```

Manual close still works, but try-with-resources closes for you, even when an exception interrupts

Why does closing matter so much for writing? PrintWriter collects text in a **buffer** and sends it to the disk in efficient batches. Until the file is closed - or flushed - the text is not guaranteed to be on the disk at all, so a program that forgets to close can leave an empty or half-written file behind. Closing also **releases** the file so other programs can use it. With try-with-resources you get all of this for free, which is why the compiler is happy to see it.

One decision changes everything about a write: **append or overwrite**. Give FileWriter a second argument **true** and it opens the file in **append mode**: existing content is kept, and new text is added at the end - exactly right for diaries and logs. Without the flag (or with false), the file is **overwritten** from the very beginning: the old contents vanish the moment you open it. Same code, one boolean, opposite results - so decide before you open, not after you have lost your data.

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.charset.StandardCharsets;

public class AppendDiary {

    public static void main(String[] args) {
        try (PrintWriter out = new PrintWriter(
                new FileWriter("diary.txt",
                        StandardCharsets.UTF_8,
                        true))) {
            out.println("Second entry: append works");
        } catch (IOException e) {
            System.out.println("Writing failed: "
                    + e.getMessage());
        }
    }
}
```

The final true keeps the old diary and adds one line; run it twice and watch the file grow

WARNING

Both FileWriter constructors can throw **IOException**, a checked exception like FileNotFoundException but broader: it covers all manner of input/output trouble. Keep a catch (IOException e) around your writing code, and note that it also catches the FileNotFoundException case, since a missing file is one kind of I/O trouble.

TIP

Choose modes deliberately: append (the final true) for diaries, logs, and anything you collect over time; overwrite (no flag) for reports you regenerate fresh each run. When a test run “wipes” a file you meant to keep, the missing true is the first thing to check.

Lesson Check 9.4

Choose the one best answer for each question. Answers are in the Answer Key.

1. What does out.println do on a PrintWriter attached to a file?

A. Prints the text on the screen
B. Saves and closes the file
C. Writes the text into the file and moves to its next line
D. Reads a line back from the file

2. What happens at the end of a try-with-resources block?

A. The file is closed automatically
B. The file is deleted
C. The program pauses
D. The file stays open until main ends

3. Which exception should a catch block handle around PrintWriter and FileWriter work?

A. ArrayIndexOutOfBoundsException
B. IOException
C. ArithmeticException
D. NumberFormatException

4. After a try-with-resources block ends, what is true about its variable out?

A. It has changed into a String
B. It still works anywhere in main
C. It holds the file's contents
D. It is out of scope and cannot be used

Practice Questions

Work through all ten questions. They are ordered from easiest to hardest. Write your answers on paper before checking the Answer Key.

1. Which block contains the statements you want to guard against runtime problems?

A. catch
B. finally
C. try
D. throw

2. The user types hello where a number is expected, and the program calls in.nextInt() inside a try. Which catch block matches?

A. catch (InputMismatchException e)
B. catch (FileNotFoundException e)
C. catch (ArithmeticException e)
D. catch (IllegalStateException e)

3. What makes FileNotFoundException different from ArithmeticException?

A. The compiler forces you to handle FileNotFoundException
B. FileNotFoundException cannot be caught
C. ArithmeticException is checked; the other is unchecked
D. There is no difference between them

4. Which statement keeps a loop reading a file until the very last line?

A. until (in.end())
B. while (in.hasNextLine())
C. A for loop that runs exactly three times
D. while (in.nextLine())

5. With try-with-resources, when is the file closed?

A. When main ends
B. Only if you call close() yourself
C. When the file is full
D. Automatically at the end of the try block

6. Text sent to a PrintWriter is guaranteed to be on the disk immediately, even before the file is closed.

[ ] True [ ] False

7. What is the exact output of this program?

```
public class Flow {
    public static void main(String[] args) {
        System.out.println("A");
        try {
            System.out.println("B");
            int[] nums = new int[2];
            System.out.println(nums[1]);
            System.out.println(nums[5]);
            System.out.println("C");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("D");
        } finally {
            System.out.println("E");
        }
        System.out.println("F");
    }
}
```

Output:

8. This program will not even compile. Find the bug.

```
import java.io.File;
import java.util.Scanner;

public class NotesReader {
    public static void main(String[] args) {
        Scanner in = new Scanner(new File(
                "notes.txt"));
        while (in.hasNextLine()) {
            System.out.println(in.nextLine());
        }
        in.close();
    }
}
```

Bug:

9. In two or three sentences, explain the difference between checked and unchecked exceptions, and say which one forces you to write try/catch or throws.

10. Write a complete program called SafeDouble that asks the user for a whole number with Scanner, prints twice that number, and prints a friendly message instead of crashing when the user types something that is not a whole number. Handle the problem with try and catch.

Mixed Quiz

Fifteen questions mixing everything from this chapter. Score 12 or more before moving on to the challenges.

1. Where does an uncaught exception leave the program?

A. Paused until you press a key
B. Continuing from the next statement
C. Running quietly in the background
D. Stopped, with an exception report printed

2. In a try/catch/finally where NO exception happens, which part runs last?

A. catch
B. finally
C. They all run together
D. try

3. Which keyword deliberately creates and launches an exception?

A. throw
B. catch
C. try
D. throws

4. Which import gives your program the File class?

A. java.awt.File
B. java.util.File
C. java.io.File
D. java.io.Scanner

5. Integer.parseInt("abc") throws which exception?

A. InputMismatchException
B. ArithmeticException
C. IllegalStateException
D. NumberFormatException

6. What does the second argument true mean in new FileWriter("log.txt", true)?

A. Open in append mode, keeping old content
B. Allow only one line of text
C. Write faster
D. Overwrite from the beginning

7. Before calling in.nextInt() on file data, which method guards against text that is not a number?

A. in.nextInt()
B. in.hasNextLine()
C. in.hasNextInt()
D. in.close()

8. A try block is followed by several catch blocks. When one exception is thrown, how many catch blocks run?

A. None; catch blocks never run
B. Exactly one: the first whose type matches
C. Only the last one
D. All of them, in order

9. FileNotFoundException must be handled in a try/catch or declared with throws on the method.

[ ] True [ ] False

10. The variable declared in a try-with-resources header can still be used after the try block ends.

[ ] True [ ] False

11. Calling System.exit can prevent a pending finally block from running.

[ ] True [ ] False

12. What is the exact output of this program?

```
public class Order {

    static void check(int n) {
        if (n < 0) {
            throw new IllegalArgumentException(
                    "negative!");
        }
        System.out.println("ok");
    }

    public static void main(String[] args) {
        try {
            check(5);
            check(-1);
            check(9);
        } catch (IllegalArgumentException e) {
            System.out.println("caught: "
                    + e.getMessage());
        }
        System.out.println("done");
    }
}
```

Output:

13. This program compiles and runs, but it destroys data instead of preserving it. Find the bug.

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;

public class DiaryBug {
    public static void main(String[] args)
            throws IOException {
        PrintWriter out = new PrintWriter(
                new FileWriter("diary.txt"));
        out.println("New entry");
        out.close();
    }
}
```

Bug:

14. Why is out.close() important after writing to a file, and which language feature closes the file automatically even when an exception happens?

15. Write a complete program called FirstLine that reads the first line of a file named welcome.txt and prints it. If the file exists but is empty, print The file is empty. If the file does not exist, print welcome.txt was not found. Handle the checked exception properly.

Challenges

Five harder thinking problems. Each includes a hint; full step-by-step solutions are in the Answer Key. Try the problem seriously before reading the hint.

Challenge 9.1 — Line Number Copier

Write a program called LineCopier that reads a file named notes.txt line by line and writes each line - with its line number in front - into a new file named numbered.txt. For a notes.txt holding two lines, milk and bread, numbered.txt should contain 1: milk and 2: bread. Handle a missing source file with a friendly message instead of a crash. This challenge reads and writes at the same time.

Hint: One catch (IOException e) can cover both the reading and the writing, because FileNotFoundException is a kind of IOException. Keep a counter that starts at 1 and grows with every line.

Challenge 9.2 — Diary Appender

Write a program called DiaryApp that lets the user type a multi-line diary entry at the keyboard and adds it to the end of a file named diary.txt without losing earlier entries. The entry ends when the user types a line containing exactly END - that sentinel line is not saved. Open the file in append mode and use try-with-resources so the file closes itself.

Hint: The append flag is the final true in new FileWriter("diary.txt", StandardCharsets.UTF_8, true). Read lines with nextLine() in a loop that breaks when the line equals "END".

Challenge 9.3 — Number-Sum Reader

Write a program called SumFile that reads whole numbers from a file named numbers.txt (one or more per line, any layout) and prints their total. If the file does not exist, print numbers.txt was not found instead of crashing. Test it with a file holding 10, 4, and 6 - the total should be 20.

Hint: hasNextInt() is true whenever the next item in the file can be read as an int, no matter how the numbers are spaced or arranged on lines.

Challenge 9.4 — Echo Machine

Write a program called EchoMachine that demonstrates the full write-then-read cycle: it writes three lines of your choosing into a file named echo.txt, and then - only after the file is safely closed - opens the same file with a Scanner and prints every line back to the screen. Use try-with-resources for the writing half.

Hint: Two separate stages, two separate try blocks: first a try-with-resources that writes and closes, then an ordinary try/catch with a Scanner that reads. The close at the end of the first block is what guarantees the text is on the disk.

Challenge 9.5 — Safe Divider

Write a program called SafeDivider that asks the user for two whole numbers, divides the first by the second, and prints the result - without ever crashing. Guard against two different troubles: read each number as text and convert it with Integer.parseInt inside a try, catching NumberFormatException for inputs like abc; and check the divisor with a plain if before dividing, so a second number of 0 prints Cannot divide by zero instead of an ArithmeticException.

Hint: Two defenses, two tools: the if test handles the legal input 0, and the catch handles input that is not a number at all. Both live inside the same try block.

Chapter Recap

- An exception is an object describing a runtime problem; with no catch, the program stops at that statement and prints a report. Catch blocks receive the object, and e.getMessage() returns its description.
- try guards risky statements; catch (SomeException e) responds to one type; several catch blocks may follow one try, and only the first matching one runs. After handling, the program continues after the try/catch.
- The RuntimeException family (ArithmeticException, NumberFormatException, InputMismatchException, ArrayIndexOutOfBoundsException) is unchecked: the compiler does not force handling.
- finally normally runs whether or not an exception happened - but not if the JVM itself exits or the process is killed. Its typical job is cleanup.
- throw new IllegalArgumentException("message") signals trouble from your own code; the rest of the method is skipped and the caller may catch it. Include a message when it helps debugging; throws on a signature declares possible trouble.
- Read files with Scanner in = new Scanner(new File("name"), StandardCharsets.UTF_8): loop with hasNextLine/nextLine for text or hasNextInt/nextInt for numbers, then close. FileNotFoundException is checked - catch it or declare it.
- Relative paths resolve against the current working directory (the project folder in most IDEs); a missing file throws at runtime, not at compile time. UTF-8 is the standard text encoding for all languages.
- Write with PrintWriter around FileWriter; the last argument true means append (keep old content), absent or false means overwrite. Content is guaranteed on disk only after closing, and try-with-resources closes automatically while keeping the variable scoped to the block.

Common Mistakes

Beginners run into these again and again. Read them now to avoid the pain later.

> **WARNING**
> **Forgetting to handle a checked exception**
> Opening a file or using FileWriter without a try/catch or a throws declaration is a compile error, and the message (unhandled exception) confuses beginners because the line itself looks harmless.
> **Fix:** Wrap the file code in try { ... } catch (FileNotFoundException e) { ... } (or IOException for writing), or add throws FileNotFoundException to the method signature.

> **WARNING**

Believing finally always runs
The finally block normally runs whether or not an exception happened, but if the JVM itself exits - for example through System.exit - or the process is killed, a pending finally is skipped. Code that assumes “always” can put vital cleanup in a place that never executes.
Fix: Say and think “normally runs”, and use finally for cleanup, not for critical logic. When a file really must be closed, try-with-resources is the stronger tool.

WARNING
Leaving out the append flag
FileWriter without the final true overwrites from the beginning, so a diary program that meant to add entries silently erases the whole history on every run - the worst kind of bug, because the program looks like it works.
Fix: When existing content must survive, open with new FileWriter("diary.txt", StandardCharsets.UTF_8, true). Reserve the no-flag form for files you regenerate from scratch.

WARNING
Using the try-with-resources variable after the block
The resource variable exists only inside the try block; after the closing brace it is out of scope, and the file is already closed. Using it outside is a compile error that puzzles people who expect the variable to live for the whole method.
Fix: Do all the file work inside the block. If a value must survive, store it in a separate variable declared before the try.

WARNING
Expecting written text on disk before close
PrintWriter buffers output and sends it to the disk in batches, so a program that writes and then crashes - or forgets to close - can leave an empty or partial file, even though the println lines all “worked”.
Fix: Close the file after writing - out.close() both flushes the buffer and releases the file - or better, use try-with-resources, which closes automatically at the end of the block.

MINI-PROJECT 3

Personal Journal (Diary) App

This project is where the whole book clicks together: a journal that APPENDS new entries to a file and can read the whole diary back. It uses methods for structure, a sentinel-controlled loop for the conversation, and try-with-resources plus UTF-8 encoding for every file operation, exactly the safe pattern from Chapter 9. The journal file appears in your program's current working directory.

Concepts used: file writing with append mode, try-with-resources, UTF-8 encoding, reading files with Scanner, checked exceptions, methods, while loops, String comparison

Step 1 — Writing an Entry (Append Mode)

The addEntry method opens journal.txt in append mode: the second argument true to FileWriter means keep what is already there. StandardCharsets.UTF_8 makes the file readable on every system. try-with-resources closes the writer automatically even if something goes wrong mid-write, and IOException is a checked exception, so the method declares throws instead of catching it here.

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.charset.StandardCharsets;

/** Appends one entry to the journal file. */
public static void addEntry(String text)
        throws IOException {
    try (PrintWriter out = new PrintWriter(
            new FileWriter("journal.txt",
                    StandardCharsets.UTF_8, true))) {
        out.println("----");
        out.println(text);
    }
}
```

Code	What it does
`/** Appends one entry to the journal file. */`	Javadoc comment describing the method's job.
`public static void addEntry(String text)`	Takes the entry text as a parameter; throws IOException because FileWriter can fail and this method lets the caller handle it.
`        throws IOException {`	The checked-exception declaration the compiler requires for file operations.
`    try (PrintWriter out = new PrintWriter(`	try-with-resources: out is closed automatically at the end of the block, even on error.
`            new FileWriter("journal.txt",`	The file lives in the current working directory.

Code	What it does
`StandardCharsets.UTF_8, true))) {`	UTF-8 encoding plus append mode true: existing entries survive.
`out.println("----");`	A separator line before each entry makes reading back easier.
`out.println(text);`	Writes the entry itself.
`}`	Block ends; the writer is flushed and closed here automatically.

Step 2 — Reading the Journal Back

Reading uses the Scanner-on-File pattern. FileNotFoundException is checked, so this method catches it and prints a friendly message instead of crashing, which is the polite thing to do when the user has not written any entries yet. The loop keeps going while there is another line to read.

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

/** Prints every line of the journal. */
public static void readJournal() {
    try (Scanner fin = new Scanner(
            new File("journal.txt"), "UTF-8")) {
        while (fin.hasNextLine()) {
            System.out.println(fin.nextLine());
        }
    } catch (FileNotFoundException e) {
        System.out.println("No journal yet! "
                + "Write an entry first.");
    }
}
```

Step 3 — The Menu Loop in main

The main method runs a do-while menu: the body must run at least once, and it repeats until the user picks 3. Reading the menu choice with nextInt leaves the trailing newline, so the extra in.nextLine() afterwards clears it before any nextLine call reads the entry text, exactly the pitfall from Chapter 2.

```
import java.util.Scanner;

public class JournalApp {

    public static void main(String[] args)
            throws IOException {
        Scanner in = new Scanner(System.in);
        int choice = 0;
        do {
            System.out.println("1. Write entry");
            System.out.println("2. Read journal");
            System.out.println("3. Quit");
            System.out.print("Choose: ");
            choice = in.nextInt();
            in.nextLine();   // clear newline
            if (choice == 1) {
                System.out.print("Type your "
                        + "entry: ");
                addEntry(in.nextLine());
            } else if (choice == 2) {
                readJournal();
            }
        } while (choice != 3);
        System.out.println("Goodbye!");
    }

    // addEntry method from Step 1
    // readJournal method from Step 2
}
```

How It Works

Every file operation follows the Chapter 9 safety rules: append mode protects old entries, UTF-8 keeps the text portable, and try-with-resources guarantees the file is closed when the block ends. Because addEntry declares throws IOException, the compiler makes main acknowledge it, which is the checked-exception system doing its job.

The menu is a do-while because the user must see it at least once before any condition makes sense. The choice variable drives both the if-else inside the loop and the loop's own condition, so adding a new menu item later means one new branch and one new number, nothing more.

The add-then-read cycle demonstrates the two directions of file IO in one program: PrintWriter sends text out with println, Scanner brings it back line by line with nextLine guarded by hasNextLine. The ---- separators you wrote in Step 1 reappear in Step 2's output, closing the loop between writing and reading.

Things to Try

- Add a timestamp line under each entry (new java.util.Date() printed with out.println is a fun freebie to experiment with).
- Add menu option 3 that deletes the journal by opening a fresh PrintWriter in overwrite mode (append false) and immediately closing it.
- Number the entries when reading them back using a counter variable that increments per ---- separator.

- Ask for the file name once at startup and pass it into both methods as a parameter instead of hardcoding journal.txt.
- Count and display how many entries the journal holds each time the menu loops.

FINAL PROJECT

Student Grade Book

The final project combines every chapter of this book into one program: a menu-driven grade book that stores student names and scores in arrays, computes statistics with methods, and saves and loads its data as a UTF-8 text file. You will build it in six steps, and each step's code compiles on its own if you test it alone. Take your time, type everything, and read the line-by-line notes as you go.

Concepts used: arrays, methods and Javadoc, loops, if-else and switch, Scanner input, string formatting with printf, file writing/reading, try-with-resources, checked exceptions

Step 1 — The Data: Parallel Arrays

The grade book tracks up to 30 students using two parallel arrays: names[i] and scores[i] always describe the same student. A count variable records how many slots are actually used, so the arrays can start empty and grow as the user adds students. These live as static fields above main so every method in the class can see them; static fields are exactly what allows the helper methods in this project to share data without passing arrays around constantly.

```
import java.util.Scanner;

public class GradeBook {

    static String[] names = new String[30];
    static int[] scores = new int[30];
    static int count = 0;

    public static void main(String[] args) {
        // filled in Step 5
    }
}
```

Code	What it does
`static String[] names = new String[30];`	Room for 30 names; every element starts as null.
`static int[] scores = new int[30];`	Room for 30 scores; every element starts as 0.
`static int count = 0;`	How many slots are really used; the arrays themselves stay length 30.

Step 2 — The Brain: Helper Methods

Three small methods do the thinking. addStudent parks one name and score into the arrays and bumps count. average returns the class mean as a double, using the (double) cast from Chapter 3 to avoid integer division. letterGrade converts a score into A/B/C/D/F with an else-if chain where the ORDER matters, exactly like Chapter 4 taught: check the highest range first.

```
/** Stores one student in the next free slot. */
public static void addStudent(String name, int score) {
    names[count] = name;
    scores[count] = score;
    count++;
}

/** Returns the mean score, or 0 with no students. */
public static double average() {
    if (count == 0) {
        return 0;
    }
    int sum = 0;
    for (int i = 0; i < count; i++) {
        sum += scores[i];
    }
    return (double) sum / count;
}

/** Maps a score to a letter grade. */
public static String letterGrade(int score) {
    if (score >= 90) {
        return "A";
    } else if (score >= 80) {
        return "B";
    } else if (score >= 70) {
        return "C";
    } else if (score >= 60) {
        return "D";
    }
    return "F";
}
```

Code	What it does
`names[count] = name;`	Uses count as the next free index, then...
`count++;`	...grows the used size by one. Parallel arrays stay in step because both writes share the same index.
`return (double) sum / count;`	Cast BEFORE dividing, otherwise int / int would truncate the average to a whole number.
`if (score >= 90) {`	First-match branching: the highest bracket wins, so no upper bound is needed in later tests.

Step 3 — The Report: Loops and printf

printReport walks the used part of the arrays with a for loop and prints one formatted row per student. printf (Chapter 8) lines the columns up: %-12s left-aligns the name in 12 spaces, %5d right-aligns the score, and %2s fits the letter grade. The report finishes with the class average formatted to two decimals.

```
/** Prints every student plus the class average. */
public static void printReport() {
    if (count == 0) {
        System.out.println("(no students yet)");
        return;
    }
    System.out.println("---- GRADE REPORT ----");
    for (int i = 0; i < count; i++) {
        System.out.printf("%-12s %5d  %2s%n",
                names[i], scores[i],
                letterGrade(scores[i]));
    }
    System.out.printf("Average: %.2f%n", average());
}
```

Code	What it does
`if (count == 0) {`	Empty class? Say so and return early instead of printing an empty report and a fake average of 0.
`System.out.printf("%-12s %5d  %2s%n",`	The format string: %s for strings, %d for integers, %n for the newline, widths fixed with numbers.
`System.out.printf("Average: %.2f%n", average());`	%.2f shows exactly two decimals of the double average.

Step 4 — Saving and Loading the File

save writes each student as a line pair (name, then score) into grades.txt with UTF-8 and try-with-resources. load reverses the trip with a Scanner, reading a name line and then an int, and it catches FileNotFoundException so the first run with no file is friendly rather than fatal. The paired-line format is chosen on purpose: nextInt handles the number parsing for us, so no string-to-int conversion is needed.

```
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.charset.StandardCharsets;

/** Writes all students to grades.txt (UTF-8). */
public static void save() throws IOException {
    try (PrintWriter out = new PrintWriter(
            new FileWriter("grades.txt",
                    StandardCharsets.UTF_8))) {
        for (int i = 0; i < count; i++) {
            out.println(names[i]);
            out.println(scores[i]);
        }
    }
    System.out.println("Saved " + count
            + " student(s).");
}

/** Loads students from grades.txt if it exists. */
public static void load() {
    count = 0;
    try (Scanner fin = new Scanner(
            new File("grades.txt"), "UTF-8")) {
        while (fin.hasNextLine()) {
            String name = fin.nextLine();
            int score = fin.nextInt();
            fin.nextLine();  // consume newline
            addStudent(name, score);
        }
        System.out.println("Loaded " + count
                + " student(s).");
    } catch (FileNotFoundException e) {
        System.out.println("No saved file found;"
                + " starting empty.");
    }
}
```

Code	What it does
`try (PrintWriter out = new PrintWriter(`	try-with-resources closes the file even if a write fails halfway.
`out.println(scores[i]);`	The pair format: one line name, one line score, repeating.
`count = 0;`	Loading starts a fresh book so the file is the single source of truth.
`fin.nextLine();  // consume newline`	nextInt leaves the cursor before the newline, so this extra nextLine eats it - the exact pitfall from Chapter 2, now handled on purpose.
`int score = fin.nextInt();`	Reads the score on its own line as an int; no string conversion is needed with the paired-line format.

Step 5 — The Menu: do-while and switch

main becomes a small conversation: show a menu, read a number, switch on it (Chapter 4), and repeat until the user chooses 6. A do-while is the natural fit because the menu must appear at least once. Each case stays tiny because the real work lives in the methods, and every case ends with break so the fall-through trap from Chapter 4 cannot bite.

```
public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    int choice = 0;
    do {
        System.out.println("== Grade Book ==");
        System.out.println("1. Add student");
        System.out.println("2. Show report");
        System.out.println("3. Save");
        System.out.println("4. Load");
        System.out.println("6. Quit");
        System.out.print("Choose: ");
        choice = in.nextInt();
        in.nextLine();   // clear the newline
        switch (choice) {
            case 1:
                System.out.print("Name: ");
                String name = in.nextLine();
                System.out.print("Score: ");
                int score = in.nextInt();
                in.nextLine();
                addStudent(name, score);
                break;
            case 2:
                printReport();
                break;
            case 3:
                try {
                    save();
                } catch (IOException e) {
                    System.out.println("Save "
                            + "failed: "
                            + e.getMessage());
                }
                break;
            case 4:
                load();
                break;
            case 5:
                System.out.println("Bye!");
                break;
            default:
                System.out.println("Unknown "
                        + "choice.");
        }
    } while (choice != 6);
}
```

Step 6 — Combine Into One File

To assemble the finished program, put the pieces in this order inside one file called GradeBook.java: (1) ALL the import lines from Steps 1 and 4 at the top, exactly once each; (2) the class declaration with the three static fields from Step 1; (3) main from Step 5; (4) the helper methods from Steps 2, 3, and 4, one after another inside the class. Compile with javac GradeBook.java and run with java GradeBook. The complete file is about 130 lines; every method is already shown in full above, so assembly is copying in order, and the only things you must not duplicate are the imports and the class braces.

How It Works

The program's architecture is a deliberate tour of the book: parallel arrays from Chapter 7 hold the data, methods from Chapter 6 give every job a name, loops from Chapter 5 walk the data, decisions from Chapter 4 grade and steer, printf from Chapter 8 makes the report readable, and the file plumbing from Chapter 9 makes the data survive between runs. Nothing here is new, which is the point: a final project should feel like all your tools arriving at once.

Data flow is one-way and easy to trace: the menu reads a choice, dispatches to a method, and the methods read or write the shared arrays. save and load are mirror images, and because load rebuilds count from the file, saving then loading always returns you to the same state. When something misbehaves, add a temporary println inside the suspicious method and watch the values flow.

Notice the defensive touches everywhere: the average guards against an empty class, printReport returns early, load treats a missing file as normal first-run behavior, and save reports how much it wrote. Real programs assume things WILL go wrong and stay polite about it, and that mindset matters more than any single language feature you learned.

Things to Try

- Add a menu option that prints the names of every student scoring below 70, using a loop and an if.
- Track the highest score: a method that returns the best score and the name that goes with it (remember to start from scores[0], not 0).
- Extend save and load to a third parallel array of class periods (1-4) for each student.
- Sort the report alphabetically using Arrays.sort on a copy of the used names and a little index juggling.
- Ask before quitting if there are unsaved changes (compare count with what was last loaded or saved).

REFERENCE

Glossary

Every important term from this book, in alphabetical order, with a plain-English definition. Terms in *italics* inside a definition are defined elsewhere in this glossary.

Term	Definition
Append mode	A file-writing mode that KEEPS the file's existing content and adds new text at the end. In Java you get it by passing true as the last argument to FileWriter. Without it, opening a file for writing erases what was there.
Argument	The actual value you put between the parentheses when CALLING a method, such as 7 in Math.abs(7). The argument is copied into the method's parameter.
Array	A fixed-size container that holds several values of the same type in numbered slots. The slots are numbered from 0, and the size is available as the array's length field.
Assignment operator	The single equals sign =. It stores the value on the right into the variable on the left. Read it as becomes, not equals.
Bytecode	The intermediate instructions the Java compiler produces and stores in a .class file. The JVM executes bytecode, which is why the same compiled program can run on Windows, macOS, and Linux.
Call a method	To run a method's code by writing its name followed by parentheses, for example playRound(secret, in). Execution jumps into the method, runs it, and comes back.
Casting	Forcing a value from one type to another by writing the type in parentheses, as in (int) 7.9 or (double) sum. Casting a double to an int truncates the decimal part toward zero.
char	The primitive type holding ONE character in single quotes, such as 'A' or '?'. Do not confuse it with String, which uses double quotes and can hold any number of characters.
Checked exception	An exception the compiler forces you to deal with, either by catching it or by declaring throws in the method header. FileNotFoundException is the classic example from file reading.
Class	The container that holds a Java program's code. Every program has at least one class, its file name matches the public class name, and execution starts inside that class's main method.
Compile	To translate human-readable source code into bytecode using the javac compiler. Compile-time errors are found during this step, before the program ever runs.
Compiler	The program (javac) that reads your .java files, checks them for errors, and produces .class bytecode files.
Concatenation	Joining strings together with the + operator, as in 'Java' + 'Rocks'. If either side is a String, + joins rather than adds.
Condition	An expression that is true or false, used by if statements and loops to decide what to do next, such as score >= 60 or count < 10.

Term	Definition
Constant	A value that should never change while the program runs, usually given a name in ALL_CAPS, such as a tax rate. Java's final keyword enforces this.
continue	A statement inside a loop that skips the rest of the body and jumps to the loop's next cycle point: the update step in a for loop, the condition check in a while loop.
Current working directory	The folder where a program was launched. A relative file name such as journal.txt is created or looked for HERE, which is why your program's files appear in the project folder.
Default value	The value an array element holds before you assign anything: 0 for numeric types, false for boolean, and null for String arrays.
do-while loop	A loop that runs its body FIRST and checks the condition at the bottom, so the body always runs at least once. Note the required semicolon after while (...);
else	The branch of an if statement that runs when the condition is false. Chained else-if blocks are tested top to bottom and the FIRST true branch wins.
Escape sequence	A backslash pair inside a string that means one special character: newline, tab, ' a double quote, \ a backslash.
Exception	An object Java creates when something goes wrong while the program is running, such as dividing by zero. Uncaught exceptions stop the program with an error report.
Expression	Any combination of values, variables, and operators that produces a value, such as 3 + 4 * 2 or score >= 60.
File	A named collection of data saved on disk. Programs read files with a Scanner and write them with a PrintWriter; the file lives in the current working directory unless you give a fuller path.
finally	The block of a try statement that normally runs whether or not an exception happened, typically used for cleanup such as closing resources.
for loop	A loop that bundles initialization, condition, and update into one header line: for (int i = 0; i < n; i++). Best when the number of passes is known or counted.
for-each loop	A compact loop over every element of an array: for (int x : nums). Use it when you do not need the index; the loop variable is a COPY, so reassigning it does not change the array.
Identifier	The name you invent for a variable, method, or class. It must start with a letter and contain no spaces; Java treats uppercase and lowercase versions as different names.
Immutable	Unchangeable. Strings are immutable: String methods never change the original string, they return a NEW string you usually store back into a variable.
Import	The statement at the top of a file that lets you use a class from Java's library by its short name, such as import java.util.Scanner; before using Scanner.
Index	An element's position inside an array, written in square brackets and starting at 0. A 5-element array has indexes 0 through 4, and using a bigger index throws an exception at runtime.
Infinite loop	A loop whose condition never becomes false, usually because the body never updates the loop variable. It hangs the program until you interrupt it, such as with Ctrl+C.

Term	Definition
Integer division	Division between two int values, which truncates: the fraction is discarded toward zero, so 7 / 2 is 3 and -7 / 2 is -3.
Javadoc comment	A special /** ... */ comment written above a class or method, often with @param and @return tags. Tools can turn these comments into official documentation pages.
JDK	The Java Development Kit: the free package containing the compiler (javac), the JVM, and the standard library. You need a JDK to write and run your own Java programs.
JVM	The Java Virtual Machine: the program, started with the java command, that loads and executes bytecode. Every operating system has its own JVM, which is how one .class file runs everywhere.
Keyword	A reserved word with fixed meaning in the language, such as public, class, while, or return. Keywords cannot be used as variable names.
Literal	A value written directly in the code, such as 42, 3.14, 'A', true, or 'Hello'. Its type comes from how it is written.
Local variable	A variable declared inside a method or block. It exists only from its declaration to the end of the innermost braces around it, and local variables have NO default value: using one before assigning is a compile-time error.
Logic error	A mistake that lets the program compile and run but produces the wrong result, such as using print where you meant println. Fixing it means correcting your instructions, not the spelling.
main method	The exact place every Java program starts: public static void main(String[] args). The JVM looks for this signature and begins executing its body.
Method	A named block of code you can run by calling it. Methods take parameters, may return a value, and keep programs organized so each job has a name.
Modulo / remainder	The % operator, which gives the remainder after integer division: 7 % 3 is 1. In Java the result's sign follows the left operand, so -7 % 3 is -1.
Object	A value that carries both data and methods, such as a String or a Scanner. Objects are created with new and their methods are called with a dot, as in in.nextInt().
Operator precedence	The fixed order in which operators are applied: *, /, and % before + and -, with parentheses overriding everything. Equal-precedence operators group left to right.
Overloading	Giving several methods the SAME name with DIFFERENT parameter lists. The compiler picks the right one by matching the arguments you pass.
Parameter	The named variable in a method's header that receives an argument's value, such as score in letterGrade(int score). Changing a parameter inside the method does not change the caller's variable.
Precedence	See Operator precedence.
Primitive type	The simple value types built into the language: int, double, boolean, char, and a few more. Primitives hold their value directly, unlike objects such as String.
printf	A printing method that builds output from a format string containing placeholders such as %d, %f, %s, and %.2f. Unlike println it does NOT end the line automatically.

Term	Definition
PrintWriter	The writing class used for text files. Wrapped around a FileWriter it offers print and println just like the screen version, and it must be closed (or used in try-with-resources) so data reaches the disk.
Return value	The value a method sends back to whoever called it, announced with the return statement. A method's return type (int, double, void...) declares what it can send back.
Runtime error	A problem that appears while the program runs, normally reported as an exception and stopping the program, such as using an array index that does not exist.
Scanner	The reading class used for both the keyboard (new Scanner(System.in)) and text files (new Scanner(new File(...))). nextInt, nextDouble, and nextLine pull values out of it.
Short-circuit evaluation	The behavior of && and || where the right side is skipped when the left side already decides the answer. Used to guard risky checks, for example count > 0 && sum / count > 90.
Statement	A single complete instruction, ended with a semicolon, such as a declaration, an assignment, or a method call. Statements run top to bottom unless a control structure says otherwise.
String	The type for text in double quotes. Strings are objects with many built-in methods (length, charAt, substring...), they are immutable, and they should be compared with equals, never with ==.
StringBuilder	A mutable text container for building strings piece by piece, especially inside loops. Call append to add text and toString to get the final String.
Switch statement	A decision structure that jumps to the case matching a value. Each case normally ends with break, and default runs when nothing matches. Forgetting break makes execution fall through into the next case.
Syntax	The grammar rules of the language: where semicolons, parentheses, and braces go, how names are spelled, and what order words appear in. Breaking syntax causes compile-time errors.
Try-with-resources	The try (resource) { ... } form that closes files automatically when the block ends, even if an exception happens. The resource variable is usable inside the block and out of scope after it.
Type	The kind of value a variable holds, such as int, double, boolean, char, or String. Java is strictly typed: every variable, parameter, and return value declares its type.
UTF-8	The standard text encoding that represents characters from every language. Specifying it (StandardCharsets.UTF_8) when reading or writing files keeps your text consistent across computers.
Variable	A named storage box for a value. It has a type, holds one value at a time, and its value can be replaced by assignment as the program runs.
while loop	A loop that checks its condition BEFORE every pass, so its body may run zero times. Best when you cannot predict how many passes you need.

BACK MATTER

Answer Key

This key has three parts: **Lesson Check Answers**, **Practice and Mixed Quiz Answers**, and **Challenge Solutions**. For every multiple-choice question the correct choice is shown in green, followed by the reason it is correct and the reason each other option is wrong.

Part 1 — Lesson Check Answers

Chapter 1 — Getting to Know Java

Lesson 1.1 — What Is Java?

1. **Answer: B** — James Gosling at Sun Microsystems

Why: Java was created in 1995 by a team led by James Gosling at Sun Microsystems; Oracle later acquired Sun and maintains Java today.

Why the other options are wrong:
A) Guido van Rossum created Python, a different language.
C) Bill Gates and Microsoft built Windows, not Java.
D) Dennis Ritchie created the C language at Bell Labs, a decade and a half earlier.

2. **Answer: A** — a list of instructions the computer follows in order

Why: A program is a precise, ordered list of instructions, and the computer executes them exactly as written, step by step.

Why the other options are wrong:
B) The computer follows your exact words, not your intentions, so vagueness does not work.
C) A program may display pictures, but the program itself is text instructions.
D) Computers never guess; they only follow instructions.

3. **Answer: D** — Java catches many mistakes before the program runs

Why: Java is checked before running: the compiler reads your code first and reports problems such as missing punctuation before the program ever starts.

Why the other options are wrong:
A) Java reports most syntax errors immediately at compile time, not weeks later.
B) No compiler silently rewrites your meaning; it either accepts the code or reports errors.
C) Checking is exactly what Java's compiler does.

4. **Answer: C** — They are two different languages with similar names

Why: Despite the names, Java and JavaScript are unrelated languages made by different people for different purposes.

Why the other options are wrong:
A) Java normally runs on servers, phones, and desktops; it is not limited to browsers.
B) They are separate languages, not versions of one language.
D) JavaScript is its own language, not a version of Java.

Lesson 1.2 — How Java Runs: From Text to Screen

1. **Answer: A** — Hello.class, a bytecode file

Why: javac translates source code into bytecode and stores it in a .class file with the same base name as the class.

Why the other options are wrong:
B) The source file itself is already the text file; the compiler produces bytecode.
C) Java does not produce .exe files; bytecode runs on the JVM instead.
D) A .jar is a packaged collection of files, not the direct output of compiling one class.

2. **Answer: C** — The Java Virtual Machine (JVM)

Why: Bytecode is designed for the JVM; the java command starts the JVM, which executes the bytecode instructions.

Why the other options are wrong:
A) javac produces bytecode; it does not execute it.
B) The shell only forwards your command to the JVM; it cannot read bytecode itself.
D) An editor only lets you type and save text; it never runs programs.

3. **Answer: B** — Greeting.java

Why: The file name must match the public class name letter for letter, with the .java ending, so the compiler can find it.

Why the other options are wrong:
A) .class is produced by the compiler; you write source code in .java files.
C) The lowercase g changes the name, and names must match exactly.
D) For a public class the name is not optional; it must match the class name.

4. **Answer: D** — Each system has its own JVM that understands the same bytecode

Why: Bytecode targets the JVM, not a specific operating system, so any system with a JVM can run the same file. That is the “write once, run anywhere” idea.

Why the other options are wrong:
A) The .class file contains bytecode, not operating-system source code.
B) One bytecode file is produced; no per-system recompilation is needed.
C) Compiling once produces one .class file.

Lesson 1.3 — Your First Program

1. **Answer: B** — In the main method

Why: The JVM always begins execution inside the main method, no matter where it appears in the file.

Why the other options are wrong:
A) Comments or other lines above main do not execute first; the JVM starts in main.
C) The class is a container; execution starts at a method inside it, specifically main.
D) Position in the file does not matter; only main starts the program.

2. **Answer: C** — println moves to a new line afterward; print does not

Why: The ln in println means line: after printing, println moves the output position to the next line, while print keeps it where it is.

Why the other options are wrong:
A) Both can print text and numbers; the difference is only the newline.
B) They are different methods with different behavior.
D) Speed is not the difference; both are equally fast for beginners' purposes.

3. **Answer: A** — System.out.println("Java is fun");

Why: The exact spelling System.out.println is required, and the text to print must be in double quotes.

Why the other options are wrong:
B) system with a lowercase s does not exist; Java is case-sensitive.
C) Out with a capital O is wrong; it must be out.
D) Without quotes, Java is fun is not valid text; the compiler reports an error.

4. **Answer: D** — A semicolon ;

Why: Statements end with a semicolon, just as sentences end with a period. Newlines alone do not end a statement.

Why the other options are wrong:
A) A closing brace ends a block of code, not an individual statement.
B) Periods appear in names like System.out.println but do not end statements.
C) The compiler ignores line breaks; a statement can span two lines if needed.

Lesson 1.4 — Comments and Error Messages

1. **Answer: B** — Ignores them completely

Why: Comments exist purely for human readers; the compiler skips them and they have zero effect on the running program.

Why the other options are wrong:
A) Nothing inside a comment is ever executed.
C) Bytecode contains only real instructions, not comments.
D) Comments never appear on screen unless you print them on purpose.

2. **Answer: C** — /* like this */

Why: Block comments start with /* and end with */ and may cover any number of lines.

Why the other options are wrong:
A) # is not a comment marker in Java.
B) // ends at the end of its line, so it cannot span lines.
D) ** is not a comment marker in Java.

3. **Answer: A** — A semicolon is probably missing at or before line 4

Why: "';' expected" is the compiler saying it reached a point where a semicolon should be; check the end of line 4 and the line before it.

Why the other options are wrong:
B) Compile-time errors stop the program before it ever runs.
C) The quoted ';' is punctuation, not a file name.
D) Deleting the line is not the fix; adding the missing punctuation usually is.

4. **Answer: D** — logic error

Why: A logic error lets the program run to the end but makes it behave incorrectly; fixing it means correcting your instructions.

Why the other options are wrong:
A) "Comment error" is not a category; comments cannot cause wrong output.
B) A runtime error normally crashes the program midway with an exception report.
C) Compile-time errors prevent the program from starting at all.

Chapter 2 — Variables and Data Types

Lesson 2.1 — Variables and Assignment

1. **Answer: B** — int apples = 40;

Why: int declares a whole-number variable, apples names it, and = stores the value 40, all in one initialization statement.

Why the other options are wrong:
A) == compares two values; it stores nothing and is not part of a declaration.
C) Assignment flows right to left: the variable name must come first, then =, then the value.
D) Without the type int in front, the compiler does not know what apples is; the line does not compile.

2. **Answer: D** — store the right-hand value into the left-hand variable

Why: = is the assignment operator: compute the right side, then store the result into the variable on the left.

Why the other options are wrong:
A) = does no comparing and no printing; it copies a value into a variable.
B) Mathematical equality is written == in Java; = is an instruction to store.
C) Nothing is swapped; only the left-hand variable changes.

3. **Answer: A** — 2

Why: The right side is computed first: 3 - 1 is 2, and that result is stored back into lives.

Why the other options are wrong:
B) The assignment replaced the old value; lives no longer holds 3.
C) Nothing adds 1 anywhere in these statements.
D) The subtraction happened once, so the result is 2, not 1.

4. **Answer: C** — 2fast

Why: Identifiers may contain digits, but they may not begin with one, so 2fast breaks the rule.

Why the other options are wrong:
A) topScore is legal: it starts with a letter and uses camelCase.
B) Names may end with digits; score2 is legal.
D) An underscore is a legal starting character for a name.

Lesson 2.2 — int and double

1. **Answer: B** — double temperature = 36.6;

Why: 36.6 has a fraction, so the variable must be a double; int can hold only whole numbers.

Why the other options are wrong:
A) 36 is a different value; the decimal part is gone.
C) int cannot store 36.6; the compiler rejects the line.
D) Without a type in front, the compiler does not know what temperature is.

2. **Answer: D** — 3

Why: Both operands are ints, so Java performs integer division and drops the fraction part: 7 / 2 is 3.

Why the other options are wrong:
A) 7 / 2 with two ints never produces 3.5; the fraction is dropped.
B) The line compiles; dropping the fraction is legal, just surprising.
C) Integer division drops the fraction; it does not round up to 4.

3. **Answer: C** — 8

Why: += adds the right side to the variable: n becomes 5 + 3, which is 8.

Why the other options are wrong:
A) n += 3 changes n; it does not leave it alone.
B) += means add and store, not glue digits; that behavior belongs to Strings.
D) += is a legal operator for int variables.

4. **Answer: A** — 5.0, a double

Why: When an int meets a double, Java widens the int to double and computes 5.0.

Why the other options are wrong:
B) Mixing is fine: the int is automatically widened to double.
C) Numbers add mathematically; only a String operand would glue text.
D) The double operand wins: the result keeps a .0.

Lesson 2.3 — Strings and chars

1. **Answer: D** — 23

Why: Because "2" is a String, + glues text: the int 3 is converted to "3" and joined, giving 23.

Why the other options are wrong:
A) The line compiles; mixing a String with an int is legal for +.
B) Concatenation inserts no space; the output is the characters glued tight.
C) Addition needs two numbers; a String operand switches + into concatenation.

2. **Answer: B** — char letter = 'Q';

Why: char literals use single quotes around exactly one character.

Why the other options are wrong:
A) Without quotes, Q is read as a name, and no such variable exists.
C) Single quotes make a char, which cannot initialize a String variable.
D) Double quotes make a String, which cannot go into a char variable.

3. **Answer: C** — Two lines: A, then B

Why: Inside a string literal, \n is the escape sequence for a newline, so println shows A on one line and B on the next.

Why the other options are wrong:
A) \n is an escape sequence, not a comment marker.
B) One escape sequence means one line break; the output has exactly two lines.
D) The backslash-n is not printed literally; it becomes a line break.

4. **Answer: A** — \\

Why: To get one real backslash you escape it: \\ stands for a single \ character.

Why the other options are wrong:
B) \" produces a double quote.
C) \t is a tab, not a backslash.
D) \n is a new line, not a backslash.

Lesson 2.4 — booleans and Reading Input

1. **Answer: C** — true and false, lowercase

Why: Java's boolean literals are the lowercase words true and false.

Why the other options are wrong:
A) The digits are ints; a boolean stores the words true and false, not numbers.
B) Capitalized True and False belong to other languages; Java reports an error.
D) yes and no are just words, not boolean literals.

2. **Answer: A** — Tells the compiler where to find the Scanner class

Why: import points the compiler at the library package java.util that holds Scanner; it runs no code itself.

Why the other options are wrong:
B) Creating the object takes new Scanner(System.in) inside main.
C) import prints nothing; it is a compile-time pointer.

D) Reading happens through methods such as nextLine(), not through the import.

3. **Answer: D** — in.nextLine()

Why: nextLine() consumes the whole line, spaces included, up to the Enter keystroke.

Why the other options are wrong:
A) nextDouble() expects a decimal number, not text.
B) nextInt() expects digits, not text.
C) next() stops at the first space, so it would read only the first word.

4. **Answer: B** — An empty string

Why: nextInt leaves the newline behind, so the nextLine reads that leftover and returns an empty string. Insert an extra in.nextLine(); to swallow it.

Why the other options are wrong:
A) nextInt already consumed the digits; only the newline is left.
C) There is a leftover newline to read, so the call returns at once with empty text.
D) nextLine never converts to numbers, and there are no characters left anyway.

Chapter 3 — Operators and Expressions

Lesson 3.1 — Arithmetic Operators

1. **Answer: D** — 1

Why: 9 divided by 4 is 2 with remainder 1, and % gives the remainder.

Why the other options are wrong:
A) 9 is not a multiple of 4, so the remainder is not 0.
B) 2 is the quotient of 9 / 4; % asks for what is left over.
C) % never produces a fraction; it gives the whole-number remainder.

2. **Answer: A** — 3.0, a double

Why: One operand is a double, so the int widens and double division runs: 6 / 2.0 is 3.0.

Why the other options are wrong:
B) Mixing int and double is legal and common.
C) Division never doubles the left operand; the value is 3.0.
D) A mixed expression produces a double; println would show 3.0.

3. **Answer: C** — Sum: 12

Why: Left to right: the String absorbs the 1 (Sum: 1), then the 2 (Sum: 12). Addition needs parentheses, as in "Sum: " + (1 + 2).

Why the other options are wrong:
A) + acts immediately; no symbols survive into the output.
B) Concatenation adds no spaces of its own.
D) For Sum: 3 you need parentheses around 1 + 2; without them the glue starts at the 1.

4. **Answer: B** — 3 * (a + b)

Why: Java requires the * operator, and the parentheses make the addition happen before the multiplication.

Why the other options are wrong:
A) Without parentheses this multiplies only the a: 3a + b.
C) This multiplies b by 3 and adds a: a + 3b.
D) 3(a + b) has no operator between 3 and (, which is a syntax error.

Lesson 3.2 — Integer Division and Remainder

1. **Answer: C** — 3

Why: Both operands are ints, so the fraction is truncated: 7 / 2 is 3. Use 7.0 / 2 for 3.5.

Why the other options are wrong:
A) Integer division drops the fraction; it does not round up.
B) 2 would be the quotient of 7 / 3; check the operands.
D) 3.5 would need a double operand; 7 / 2 runs as integer division.

2. **Answer: A** — -1

Why: The remainder's sign follows the dividend, the left operand, so -7 % 2 is -1.

Why the other options are wrong:
B) The sign comes from the dividend, not the divisor, so it stays negative.
C) -3 is the quotient of -7 / 2; % asks for the remainder instead.
D) -7 is odd; its remainder after dividing by 2 is not 0.

3. **Answer: D** — n % 10

Why: Dividing by 10 leaves the last digit as the remainder: 1234 % 10 is 4.

Why the other options are wrong:
A) n / 1000 gives the first digit, 1.
B) n / 10 removes the last digit, giving 123.
C) n % 100 gives the last two digits, 34.

4. **Answer: B** — 1500 / 60 and 1500 % 60

Why: 1500 / 60 is 25 whole minutes and 1500 % 60 is 0 leftovers; division gives the big unit and remainder the small one.

Why the other options are wrong:
A) A minute has 60 seconds, not 100.
C) Subtracting 60 once does not remove all the whole minutes.
D) The two results are swapped; division gives the minutes.

Lesson 3.3 — Precedence, Parentheses, and Casting

1. **Answer: C** — 14

Why: * has higher precedence than +: 3 * 4 runs first (12), then the 2 is added, giving 14.

Why the other options are wrong:
A) 9 ignores the multiplication entirely (2 + 3 + 4).
B) 12 is only the 3 * 4 part; the 2 still gets added.
D) 20 would need parentheses: (2 + 3) * 4.

2. **Answer: D** — 9

Why: Casting a double to int truncates toward zero, dropping the fraction: 9.7 becomes 9.

Why the other options are wrong:
A) The cast does not split the number; it throws the fraction away.
B) Casting never rounds up; rounding is Math.round's job.
C) An int cannot hold 9.7; the fraction is dropped, not kept.

3. **Answer: A** — 7

Why: Math.round returns the nearest whole number with halves going up: 6.5 becomes 7 (a long, printed as 7).

Why the other options are wrong:
B) The result is a whole number; the .5 does not survive.
C) Halves round up, not down.
D) Math.round returns a whole-number type, so no .0 appears.

4. **Answer: B** — (double) a / b

Why: Casting a widens it to 7.0 before the division, so the computation is 7.0 / 2, which is 3.5.

Why the other options are wrong:
A) Both ints give integer division: 3.
C) The inner a / b runs first as integer division (3), then widens to 3.0.
D) Casting an int to int does nothing; the division stays 3.

Lesson 3.4 — Comparison and Logical Operators

1. **Answer: D** — true

Why: 5 is greater than 3, so the comparison produces the boolean true; println shows the word true.

Why the other options are wrong:
A) Comparisons with > are legal expressions.
B) A comparison produces a boolean, not either operand.
C) The comparison 5 > 3 is factually true; false would need different operands.

2. **Answer: A** — false

Why: && needs both sides true; 2 == 2 is true but 3 < 1 is false, so the result is false.

Why the other options are wrong:
B) One false side is enough to make the AND false.
C) The expression computes a boolean, not a number.
D) Comparisons and logic on plain ints cannot crash; the result is false.

3. **Answer: B** — (x + 1) > 5

Why: Arithmetic binds tighter than comparison, so the addition groups first and the > compares the sum to 5.

Why the other options are wrong:
A) The > is part of the expression and produces the final boolean.
C) Adding a boolean to x makes no sense; arithmetic runs before comparisons.
D) Mixing arithmetic and a comparison is perfectly legal.

4. **Answer: C** — Only when the left side is false

Why: || can settle for true when the left side is true, so the right side runs only if the left side was false.

Why the other options are wrong:
A) That describes single |; || short-circuits a true left side.
B) The right side still runs whenever the left side is false.
D) When the left side is true, || stops without looking at the right.

Chapter 4 — Making Decisions

Lesson 4.1 — The if Statement

1. **Answer: C** — It is skipped and the program continues after the if

Why: A false condition means the brace-enclosed body is skipped entirely; execution resumes with the first statement after the if.

Why the other options are wrong:
A) The body runs zero times when the condition is false.
B) Skipping a body is normal, everyday behavior, never an error.
D) A skipped body does not run at all, not even quietly.

2. **Answer: B** — boolean - true or false

Why: Java requires the if condition to be a boolean expression; numbers are never treated as yes-or-no answers.

Why the other options are wrong:
A) Java does not convert positive numbers to true; the condition must be boolean.

C) A String cannot be a condition; you will learn proper string comparison in Chapter 8.
D) Java does not convert 0 to false; an int condition is a compile error.

3. **Answer: D** — end

Why: 5 > 10 is false, so the body is skipped; the println after the if is outside the body and always runs.

Why the other options are wrong:
A) The final println is not part of the body, so it prints.
B) The println outside the if still runs, so end appears.
C) big would require the condition to be true; it is false here.

4. **Answer: A** — Without braces, only the first statement after if belongs to it

Why: An unbraced if owns exactly one statement; everything after that runs unconditionally.

Why the other options are wrong:
B) Braces decide what belongs to the body, so they change behavior all the time.
C) A body may hold any number of statements, including one or none.
D) The compiler never rewrites your code; a missing brace is your responsibility.

Lesson 4.2 — else and else-if Chains

1. **Answer: B** — Exactly one

Why: The two bodies are alternatives: the if body runs when the condition is true, the else body when it is false, never both.

Why the other options are wrong:
A) The bodies are mutually exclusive by design.
C) Exactly one body always runs; that is the whole point of if/else.
D) One of the bodies always runs; two can never run together.

2. **Answer: A** — The first one, from the top, whose condition is true

Why: First-match branching: conditions are tested top to bottom and the first true one wins; the remaining branches are skipped.

Why the other options are wrong:
B) Only one branch ever runs, no matter how many conditions are true.
C) Position next to the else is irrelevant; only the first true condition counts.
D) The chain stops at the first match, so later true conditions are never reached.

3. **Answer: C** — Grade: B

Why: 85 fails the 90 check but passes the 80 check, so the B branch runs and the chain ends there.

Why the other options are wrong:
A) Grade: D needs a score below 70.
B) The chain never tests the 70 branch; the first match already won.
D) Grade: A requires a score of 90 or more.

4. **Answer: D** — First-match picks a branch; short-circuit skips an operand

Why: First-match branching chooses one branch of an if/else-if chain; short-circuit evaluation skips the right operand of && or || inside a single expression.

Why the other options are wrong:
A) That swaps the two terms.
B) They operate at different levels of the language and must not be confused.
C) That is backwards: first-match works across a statement chain, short-circuit inside one expression.

Lesson 4.3 — Nested ifs and Common Patterns

1. **Answer: D** — the nearest if that has no else yet

Why: The dangling-else rule: an else attaches to the closest unmatched if above it, whatever the indentation suggests.

Why the other options are wrong:
A) The code compiles fine; the pairing is automatic and silent.
B) Order in the file does not matter; nearness does.
C) The outermost if is chosen only if every nearer if already has an else.

2. **Answer: A** — true

Why: Both parts are true for 84, and && requires both, so the whole condition is true.

Why the other options are wrong:
B) Comparing ints with comparison operators and joining with && is perfectly legal.
C) The condition depends only on score, which is known.
D) 84 is inside the 80s, so both bounds hold.

3. **Answer: B** — if (isRaining)

Why: The flag is true, so the plain flag check runs the body; the other three conditions all evaluate to false.

Why the other options are wrong:
A) Not equal to true means false, so the body is skipped.
C) ! flips true to false, so the body is skipped.
D) That is true only when isRaining is false.

4. **Answer: C** — Nothing at all

Why: The else really belongs to the inner withAdult if; the outer age condition is false, so neither inner branch is ever reached.

Why the other options are wrong:
A) Denied runs only when age is 18 or more AND withAdult is false, because the else pairs with the inner if.
B) The program compiles and runs quietly; that is exactly the trap.
D) Allowed requires age 18 or more AND withAdult true.

Lesson 4.4 — The switch Statement

1. **Answer: A** — Execution falls through into the next case's body

Why: Without break, the program simply continues into the statements of the next case, which is called fall-through.

Why the other options are wrong:
B) Fall-through is legal Java, which is why it is such a sneaky bug.
C) The matched case still runs; only the stopping point changes.
D) default runs only when no label matched, not after a matched case.

2. **Answer: C** — case 3:

Why: Case labels must be constants such as 3 or 'B'; a variable or a calculation involving one is not allowed.

Why the other options are wrong:
A) Because n is a variable, n + 1 is not a constant label.
B) total is a variable, not a constant.
D) score is a variable, and labels must be fixed before the program runs.

3. **Answer: B** — Runs when no case matches the value

Why: default is the catch-all body for values that matched none of the case labels.

Why the other options are wrong:
A) default runs exactly when nothing matched, not on every run.

C) The jump goes to the matching case first; default is consulted only after every label fails.
D) default is ordinary code; only return or an error ends a program.

4. **Answer: D** — case 'A':

Why: A char literal uses single quotes, so the label reads case 'A':.

Why the other options are wrong:
A) Without quotes, A is a variable name, and labels must be constants.
B) Double quotes make a String, which is not the char type we are switching on here.
C) Parentheses do not turn a name into a constant.

Chapter 5 — Loops

Lesson 5.1 — while Loops

1. **Answer: A** — Before every pass, including the first

Why: A while loop re-tests its condition before each and every pass; that is how it knows when to stop.

Why the other options are wrong:
B) The check happens before the body, not after, including the very first check.
C) Checking once would make stopping impossible; the condition must be re-read every time.
D) Printing has nothing to do with the condition; it is tested every pass regardless.

2. **Answer: D** — Zero times

Why: The condition is false before the first pass, so the body is skipped entirely and the program moves on.

Why the other options are wrong:
A) fuel is already 0; there is nothing to wait for.
B) While checks BEFORE running the body, so an immediately false condition skips it.
C) A false condition stops the loop immediately.

3. **Answer: B** — Nothing in the body changes i

Why: The loop variable never moves, so the condition i <= 5 stays true forever and the loop never ends.

Why the other options are wrong:
A) println works perfectly well inside loops; it is the missing update that breaks things.
C) The operator is fine for the values involved; the problem is that i never changes.
D) Starting at 1 is normal; never leaving 1 is the bug.

4. **Answer: C** — 3 2 1

Why: The loop prints 3, 2, then 1, each with a space; when i becomes 0, the condition 0 > 0 is false and the loop ends before printing 0.

Why the other options are wrong:
A) The condition stops the loop at 0, so 0 is never printed.
B) The first pass runs with i = 3, so 3 prints first.
D) Statements run in order; counting up would need a different update.

Lesson 5.2 — for Loops

1. **Answer: A** — 3 times

Why: i takes the values 0, 1, and 2; when i becomes 3 the check 3 < 3 fails and the loop ends.

Why the other options are wrong:
B) i grows by 1 every pass, so the condition eventually fails.
C) i = 3 fails the check, so there is no fourth pass.
D) Starting at 0 gives three values below 3: 0, 1, and 2.

2. **Answer: C** — After each pass through the body

Why: The order is init once, then check, body, update, check, body, update, ... so the update runs at the end of every pass.

Why the other options are wrong:
A) If the body runs zero times, the update never runs either.
B) The init part is the one-time piece; the update repeats after every pass.
D) The header update is exactly what makes for loops hard to break by accident.

3. **Answer: B** — Only inside the loop

Why: A variable declared in the header exists only within the loop; below it the name is unknown and using it is a compile error.

Why the other options are wrong:
A) After the closing brace the variable is out of scope and gone.
C) Its scope is the loop, not the whole method.
D) It does not exist before the header runs, either.

4. **Answer: D** — 2 4 6

Why: i takes the values 2, 4, and 6; after printing 6 the update makes i = 8, which fails the check 8 <= 6.

Why the other options are wrong:
A) The update adds 2, not 1, so the odd numbers never appear.
B) The first pass runs with i = 2, so 2 prints first.
C) 8 fails the check 8 <= 6, so there is no fourth pass.

Lesson 5.3 — do-while and Input Validation

1. **Answer: A** — Exactly one

Why: The body runs first and the condition is checked at the bottom, so one pass is guaranteed.

Why the other options are wrong:
B) If the condition is false after the first pass, the loop stops right there.
C) One run is guaranteed no matter what the condition says.
D) Zero runs is the while/for behavior; do-while is different on purpose.

2. **Answer: C** — A semicolon after while (condition)

Why: The do-while statement ends with while (condition); leaving off that semicolon is a compile error.

Why the other options are wrong:
A) The word do stands alone; no colon follows it.
B) There is no comma anywhere in a do-while header.
D) One brace-enclosed body is enough, exactly like the other loops.

3. **Answer: D** — The user must be asked at least once

Why: Validation always requires one prompt and one read, which is exactly the at-least-once guarantee of do-while.

Why the other options are wrong:
A) In a do-while the condition is checked AFTER the body, which is the point.
B) All three loops run at similar speed; the choice is about behavior, not speed.
C) Reading input still needs a Scanner; the loop only controls repetition.

4. **Answer: B** — i is 10, once

Why: The body runs once, printing i is 10; then i becomes 11, the check 11 < 10 fails, and the loop ends.

Why the other options are wrong:
A) After one pass the condition is false, so the loop stops.
C) Only one println is inside the body, and it prints before i changes.
D) A do-while body always runs at least one time.

Lesson 5.4 — break, continue, and Nested Loops

1. **Answer: B** — Exits the nearest enclosing loop immediately

Why: break leaves the nearest loop at once and resumes with the first statement after that loop.

Why the other options are wrong:
A) Restarting from the top is the opposite of leaving.
C) Skipping to the next pass is continue's job.
D) Only the loop ends; the program continues below it.

2. **Answer: C** — The update part, then the condition

Why: In a for loop, continue hands control to the update expression and then the condition, which is why for loops are safe with continue.

Why the other options are wrong:
A) continue never exits; it only skips the rest of one pass.
B) The update always runs in a for loop, even after continue.
D) Leaving the loop is break's behavior.

3. **Answer: D** — 12

Why: The inner loop runs fully on every outer pass, so the total is 3 times 4 = 12.

Why the other options are wrong:
A) 4 counts one inner pass, not all of them.
B) 3 counts only the outer passes.
C) Adding the pass counts is the classic mistake; nested loops multiply.

4. **Answer: A** — It may skip the line that updates the loop variable

Why: In a while loop, continue jumps straight to the condition check; any update below it in the body is skipped, so the loop variable can freeze.

Why the other options are wrong:
B) The condition check comes next, not the body's first line.
C) The condition text never changes; the variables it watches do.
D) continue is legal in while loops; it is just riskier there.

Chapter 6 — Methods

Lesson 6.1 — Your First Methods

1. **Answer: C** — A named block of code that runs when called

Why: A method is a named block of code inside a class that performs one job each time some code calls it by name.

Why the other options are wrong:
A) A method contains statements and possibly parameters; it is not itself a variable.
B) Programs live in files; methods live inside classes within those files.
D) Comments are ignored by the compiler and never run.

2. **Answer: B** — Inside the class, next to main

Why: Our methods are defined directly inside the class body, side by side with main, never inside another method.

Why the other options are wrong:
A) Methods cannot nest inside main; that is a compile error.
C) Definition order does not matter, so a method may sit above or below the calls.
D) One file can hold a whole class of methods; no per-method files are needed.

3. **Answer: A** — The method returns no value

Why: void is the return type: it announces that the method hands nothing back to the caller when it finishes.

Why the other options are wrong:
B) void changes what the method returns, not how fast it runs.
C) A void method may be called as many times as you like.
D) Parameters live in the parentheses; void says nothing about them.

4. **Answer: D** — Only when some code calls cheer()

Why: Defining a method only prepares it. The body executes each time a call such as cheer() is reached, and not otherwise.

Why the other options are wrong:
A) The JVM starts in main; other methods run only when called.
B) The compiler only checks the definition; compiling never executes it.
C) Statements run where they are written; a definition runs nothing.

Lesson 6.2 — Parameters

1. **Answer: D** — the argument for the name parameter

Why: Values written inside a call's parentheses are arguments; name is the parameter that receives this one.

Why the other options are wrong:
A) greet is void, so it has no return value at all.
B) The parameter is the variable in the header; "Amal" is the value passed to it.
C) Compiler comments are // or /* */; quoted text here is a real value.

2. **Answer: A** — 3

Why: Primitive parameters are copies. The method doubled its own x, so main's cups still holds 3.

Why the other options are wrong:
B) Passing a copy is perfectly legal; nothing crashes.
C) 6 was printed inside the method for the copy; the original never changed.
D) Nothing sets cups to 0; the method cannot reach the original variable.

3. **Answer: C** — book("Java 101", 350)

Why: Arguments must match the parameter list in count, order, and type: a String first, an int second.

Why the other options are wrong:
A) Both parameters need values; an int alone cannot fill a String and an int.
B) One argument is not enough for a two-parameter method.
D) The order is flipped; 350 cannot fill a String parameter.

4. **Answer: B** — It disappears; it exists only inside the method

Why: Parameters are local variables. Each call creates them fresh, and the end of the method destroys them.

Why the other options are wrong:
A) There are no global variables in this part of Java.
C) Values come back only through return, and only if the return type is not void.
D) Only the main method runs for the whole program; parameters do not survive calls.

Lesson 6.3 — Return Values

1. **Answer: C** — The type of value the method sends back

Why: The return type is a promise: each completed call produces exactly one value of that type for the caller to use.

Why the other options are wrong:
A) The method's name is chosen by the programmer and has no type of its own.
B) A method may be called any number of times; the return type says nothing about that.
D) The parameter list between the parentheses describes the inputs, not the return type.

2. **Answer: D** — 12

Why: The call evaluates to 6 * 2, so the result 12 replaces the call wherever it is used.

Why the other options are wrong:
A) 6 is the argument that went in, not the value that came back.
B) The method returns an int; void methods produce no value at all.
C) The method multiplies rather than glues digits together.

3. **Answer: A** — Ends the method immediately

Why: return both delivers the value and stops the method on the spot; the caller resumes right after the original call.

Why the other options are wrong:
B) Methods do not pause; once return runs, the method is finished for good.
C) Parameters simply stop existing when the method ends; return does not delete them.
D) Restarting would need a fresh call, not a return statement.

4. **Answer: B** — return;

Why: A bare return with no value exits a void method immediately; it is the standard early-exit tool.

Why the other options are wrong:
A) return 0; would need a non-void return type; a void method returns no value.
C) stop is not a Java statement.
D) break ends a loop or switch, and is a compile error outside one.

Lesson 6.4 — Overloading, Scope, and Javadoc

1. **Answer: D** — Several methods sharing a name with different parameter lists

Why: Overloading gives one name several parameter lists; the compiler matches each call to the list that fits the arguments.

Why the other options are wrong:
A) Calling methods from other methods is just normal calling, not overloading.
B) Renaming a variable has nothing to do with method names.
C) A method calling itself is recursion, a different idea entirely.

2. **Answer: B** — The int version, because 7 matches int exactly

Why: An exact type match beats a conversion, so the int parameter list wins for the call go(7).

Why the other options are wrong:
A) The int 7 could widen to double, but the exact int match always wins.
C) A clear match exists, so the call compiles and runs normally.
D) Exactly one method runs per call; the compiler picks the single best fit.

3. **Answer: A** — Only inside that loop's braces

Why: A variable's scope is the innermost braces where it was declared, so the loop variable ends with the loop.

Why the other options are wrong:
B) When the loop's closing brace runs, the variable is gone.
C) Class-wide visibility is not how local variables work.
D) The loop's braces are inner; the variable never escapes them.

4. **Answer: C** — @param

Why: Each parameter gets its own @param line in the Javadoc comment above the method; a returning method also gets an @return line.

Why the other options are wrong:
A) @arg is not a Javadoc tag.
B) @input is not a Javadoc tag.
D) @value is not a Javadoc tag.

Chapter 7 — Arrays

Lesson 7.1 — Creating Arrays

1. **Answer: D** — 0 through 5

Why: Indexes start at 0 and end at length - 1, so six elements give the indexes 0, 1, 2, 3, 4, and 5.

Why the other options are wrong:
A) Array indexes start at 0, not 1.
B) Index 0 is valid and index 5 is the last one, so this range skips both ends.
C) Index 6 does not exist; the last valid index is length - 1.

2. **Answer: C** — nums.length

Why: length is a field built into every array, written with no parentheses.

Why the other options are wrong:
A) Arrays have no size field; the built-in name is length.
B) length is a field, not a method, so the parentheses cause a compile error.
D) Java has no length function that takes an argument.

3. **Answer: A** — 0

Why: Fresh numeric arrays are filled with default values, and the int default is 0.

Why the other options are wrong:
B) Defaults are guaranteed; int boxes start at exactly 0.
C) 4 is the array's length, not the contents of any box.
D) Index 2 is valid for a 4-element array, so nothing crashes.

4. **Answer: B** — a runtime error: ArrayIndexOutOfBoundsException

Why: The index type is fine, so the compiler accepts it; the crash happens only when the line runs with an index outside 0 to length - 1.

Why the other options are wrong:
A) -1 is a programmer's not-found code, not something the runtime returns here.
C) Java refuses to invent a value for an index that does not exist.
D) The compiler does not track array sizes, so it cannot reject the line at compile time.

Lesson 7.2 — Arrays and Loops

1. **Answer: B** — An index for loop, because it knows i

Why: Position work needs the index variable, and only the index loop for (int i = 0; i < arr.length; i++) provides it.

Why the other options are wrong:
A) The index loop prints positions comfortably.
C) A loop without a working condition is how programs hang.
D) A for-each loop hands you copies of the values, never their positions.

2. **Answer: A** — 5

Why: A for-each loop delivers the elements themselves, front to back, so the first pass copies the first element, 5.

Why the other options are wrong:
B) 6 is the second element; it arrives on the second pass.
C) for-each gives values, not indexes; only the index loop knows i.
D) x holds a copy of an element, and the elements here are 5, 6, and 7.

3. **Answer: D** — To force decimal division instead of integer division

Why: Both operands are ints, so plain / would truncate; casting one operand to double keeps the decimal part.

Why the other options are wrong:
A) Averages usually need decimals; integer division is exactly what destroys them.
B) The cast converts the sum value, not the array itself.

C) length on an int array is an int.

4. **Answer: C** — Changes a copy; the array is untouched

Why: The for-each variable is a copy of each element, so assigning to it affects nothing but that copy.

Why the other options are wrong:
A) Reaching the array through for-each is impossible; only an index loop can write elements.
B) One copy per pass is changed, and even that change evaporates.
D) Assigning to a plain variable is legal; nothing crashes.

Lesson 7.3 — Array Algorithms

1. **Answer: C** — 0 is wrong when every element is negative

Why: A tracker must start at a value actually in the array; starting at 0 reports 0 for all-negative arrays, which is wrong.

Why the other options are wrong:
A) 0 is a perfectly good int; the problem is that it may not appear in the array.
B) Java imposes no rule; choosing arr[0] is the logically safe habit.
D) The first element is just a safe starting champion; bigger values replace it later.

2. **Answer: D** — The target is not in the array

Why: -1 can never be a real index, so it is the standard signal for a failed search.

Why the other options are wrong:
A) The method returned normally; a crash produces an exception, not -1.
B) No array has an index -1; that is exactly why -1 works as a signal.
C) An empty array would also return -1, but -1 alone does not prove emptiness.

3. **Answer: B** — the first assignment would overwrite a value still needed

Why: After nums[i] is overwritten, the original value would be gone forever; temp stores it so it can land in the other slot.

Why the other options are wrong:
A) Arrays accept as many assignments as you like; the issue is lost data, not permission.
C) The temp variable is about correctness, not speed.
D) Array elements are assigned with = exactly like ordinary variables.

4. **Answer: A** — 4

Why: Each swap fixes two positions, so length / 2 = 4 swaps cover all eight elements.

Why the other options are wrong:
B) Sixteen is twice the length; each swap moves two values, not one.
C) Eight swaps would swap everything twice and rebuild the original order.
D) Seven is the count of neighboring pairs in the array, not the swaps needed.

Lesson 7.4 — 2D Arrays and java.util.Arrays

1. **Answer: C** — 3, the number of rows

Why: On a 2D array, .length answers with the number of rows; a row's own length lives at g[r].length.

Why the other options are wrong:
A) The grid holds twelve default zeros; it is not empty.
B) Column counts belong to individual rows: g[r].length.
D) No single field counts all twelve boxes; you would multiply rows by columns.

2. **Answer: D** — grid[1][2]

Why: The row index comes first: grid[row][column], so row 1, column 2 is grid[1][2].

Why the other options are wrong:
A) That order reads row 2, column 1.
B) Parentheses are for method calls; indexes always use brackets.
C) Java has no comma form; each index needs its own brackets.

3. **Answer: B** — A String like [7, 2, 9, 4]

Why: Arrays.toString builds one readable String from the elements, ideal for println; the array itself is not changed.

Why the other options are wrong:
A) That is data.length, a field, not a library method.
C) It returns a String value; printing is still your job with println.
D) Sorting is Arrays.sort's job; toString changes nothing.

4. **Answer: A** — nums itself is now in ascending order

Why: Arrays.sort works in place: it rearranges the elements of the array you passed in and returns void.

Why the other options are wrong:
B) Ascending order is not reversal; use your own swap loop to reverse.
C) Filling is Arrays.fill's job, not sort's.
D) sort produces no new array and no return value; it edits the original.

Chapter 8 — Working with Strings

Lesson 8.1 — Useful String Methods

1. **Answer: C** — s.length()

Why: Strings expose length as a method, so the call needs the parentheses, unlike the array field .length from Chapter 7.

Why the other options are wrong:
A) Strings have no size method; that name belongs to other languages and classes.
B) Without parentheses that is the array form; a String needs the method call s.length().
D) length is not a standalone function; you call it on the string with a dot.

2. **Answer: A** — 'p'

Why: Indexes start at 0: E is 0, l is 1, e is 2, so index 3 holds p.

Why the other options are wrong:
B) The second e sits at index 2, not 3.
C) E is the very first character, at index 0.
D) h is at index 4.

3. **Answer: D** — "eph"

Why: The start index 2 is included, the end index 5 is excluded, so you get the characters at indexes 2, 3, and 4: e, p, h.

Why the other options are wrong:
A) phant starts at index 4; substring starts at index 2.
B) leph would start at index 1, but the start index here is 2.
C) That is substring(2) with one argument; with two arguments the end is excluded.

4. **Answer: B** — 1

Why: indexOf returns the index of the FIRST match, and the first o sits at index 1.

Why the other options are wrong:
A) Index 0 holds n, not o.
C) 5 is where the second o appears; indexOf reports only the first.
D) -1 is the answer only when the piece is not found at all; o appears three times.

Lesson 8.2 — Comparing Strings and Immutability

1. **Answer: D** — a.equals(b)

Why: equals walks through both strings and compares the actual characters, which is what “same word” means.

Why the other options are wrong:
A) == asks whether the two variables point at the same object, not at the same letters.
B) a = b is an assignment that copies a reference; it is not a comparison at all.
C) Equal lengths do not mean equal letters; cat and dog would pass this test.

2. **Answer: B** — a negative number

Why: Apple comes before Banana in dictionary order, so compareTo returns a negative value; only the sign matters.

Why the other options are wrong:
A) A positive result means the first string comes later in the dictionary.
C) Zero is returned only when the two strings are exactly equal.
D) Comparing two ordinary strings is a normal operation and never throws an error.

3. **Answer: C** — "day"

Why: Strings are immutable: toUpperCase built a new string that nobody stored, so s keeps its original value.

Why the other options are wrong:
A) Ignoring a method's result is legal Java; the line compiles fine.
B) The call changes nothing at all, so s is not emptied either.
D) s would only change with an assignment: s = s.toUpperCase();

4. **Answer: A** — s.length() == 0

Why: An empty string has zero characters, so its length is 0; this test is safe and clear.

Why the other options are wrong:
B) trim returns a string, not a number, so this line does not even compile.
C) == compares object references, not contents; use length or equals.
D) Length 1 means one character was typed, not an empty string.

Lesson 8.3 — printf and Formatting

1. **Answer: A** — %d

Why: %d is the integer specifier; the d stands for decimal (base-10) integer.

Why the other options are wrong:
B) %n starts a new line and consumes no argument at all.
C) %f is for floating-point values such as double.
D) %s prints text; %d is the int specialist.

2. **Answer: C** — Total: 7.13

Why: %.2f rounds to exactly two decimals, so 7.125 becomes 7.13.

Why the other options are wrong:
A) %.2f rounds rather than chops; 7.125 goes up to 7.13.
B) Without a precision %f shows six decimals, not three.
D) That is plain %f with its default six decimals.

3. **Answer: D** — printf does not start a new line by itself

Why: printf prints only what the format string says; add %n or \n yourself when a line should end.

Why the other options are wrong:
A) printf always needs the format string as its first argument.
B) %s prints text perfectly well.
C) Speed is not the issue; formatting is.

4. **Answer: B** — [hi]

Why: The minus left-aligns the text in a width of 6, so hi is followed by four spaces inside the brackets.

Why the other options are wrong:
A) With %-6s all the padding goes to the right of the text, not around it.
C) The width is respected even for short text; spaces pad it up to 6.
D) Right-aligned padding is %6s without the minus.

Lesson 8.4 — StringBuilder

1. **Answer: A** — It changes one buffer instead of making a new string each pass

Why: append extends a private buffer in place, so nothing built so far is ever re-copied.

Why the other options are wrong:
B) StringBuilder holds text only.
C) Printing speed is unrelated; the win is in how the text is assembled.
D) You still write the loop yourself; StringBuilder just makes each pass cheap.

2. **Answer: C** — 5

Why: length counts the characters currently in the buffer, exactly like String's length().

Why the other options are wrong:
A) Zero would mean an empty buffer, and this one holds hello.
B) There is no extra invisible character at the end.
D) Count again: h, e, l, l, o are five characters.

3. **Answer: D** — toString()

Why: toString is Java's standard name for “give me the readable form of this object”, and StringBuilder follows it.

Why the other options are wrong:
A) No such method exists on StringBuilder.
B) convert is not a StringBuilder method.
C) toText is not a Java method; toString is the convention.

4. **Answer: B** — "cba"

Why: reverse flips every character in the buffer, so abc becomes cba.

Why the other options are wrong:
A) Reversing does not empty the builder.
C) abc is the untouched original; reverse really does reorder the buffer.
D) bca is a rotation, not a reversal.

Chapter 9 — Files and Exceptions

Lesson 9.1 — Exceptions: try and catch

1. **Answer: D** — An object that describes a runtime problem

Why: An exception is an object created while the program runs, carrying information about what went wrong.

Why the other options are wrong:
A) Compile-time errors stop the compiler before any program exists; exceptions happen at runtime.
B) Exceptions live in memory as objects; files live on the disk.
C) An endless loop is a logic problem and produces no exception at all.

2. **Answer: C** — ArithmeticException

Why: Integer division by zero always throws ArithmeticException at runtime.

Why the other options are wrong:
A) That one is about opening files that do not exist.
B) That one is about converting bad text to a number.

D) That one is about array indexes, not arithmetic.

3. **Answer: A** — A short text describing the problem

Why: The exception object carries a message, and getMessage returns it - for example “/ by zero”.

Why the other options are wrong:
B) It describes the exception, not your source code.
C) The message explains the trouble; it does not repair anything.
D) The message describes the problem; it is not a line number.

4. **Answer: B** — Continues with the statements after the try/catch

Why: Handling means life goes on: the statements after the whole try/catch construct run normally.

Why the other options are wrong:
A) The try block was abandoned the moment the exception was thrown.
C) Execution never rewinds to the top of main.
D) Exiting is what happens when NO catch handles the exception.

Lesson 9.2 — finally and throw

1. **Answer: C** — Whether or not an exception happened

Why: finally is for cleanup that should happen on every normal path, clean run or handled problem alike.

Why the other options are wrong:
A) Finally runs on a clean run as well - that is its purpose.
B) A JVM exit is exactly the rare case that can SKIP a pending finally.
D) That is the catch block's moment; finally runs after it too.

2. **Answer: A** — throw new IllegalArgumentException("bad input")

Why: throw needs a newly created exception object, and the message rides inside the constructor.

Why the other options are wrong:
B) That creates an object but never throws it; the alarm never rings.
C) You throw exception objects, not strings.
D) catch receives exceptions; it does not produce them.

3. **Answer: B** — That the method may produce that exception

Why: throws is a declaration: this method can let that exception escape, so callers know what to expect.

Why the other options are wrong:
A) Access is set by public or private, not by throws.
C) Catching happens in catch blocks; throws only declares a possibility.
D) Completion is signaled by return or the end of the body, not by throws.

4. **Answer: D** — They are skipped; the method stops at once

Why: throw is immediate, like return: nothing further in that block executes.

Why the other options are wrong:
A) The throw fires at its own line; code after it never gets its turn.
B) Nothing in Java runs twice by itself.
C) Statements stay in the source; they are skipped, not commented out.

Lesson 9.3 — Reading Files

1. **Answer: D** — FileNotFoundException

Why: A file may be missing, so the constructor declares FileNotFoundException - a checked exception the compiler watches.

Why the other options are wrong:
A) That one signals a method called at the wrong moment.
B) That one belongs to division by zero.
C) That one comes from nextInt meeting non-number keyboard input, not from opening a file.

2. **Answer: B** — The compiler forces you to catch it or declare throws

Why: Checked means compiler-enforced: no try/catch and no throws declaration means no compiled program.

Why the other options are wrong:
A) No virus scanning is involved; the check is about your code, not the file's contents.
C) The file is checked once, when it is opened.
D) Checked describes compile-time enforcement, not a special run mode.

3. **Answer: C** — false

Why: hasNextLine answers a yes/no question, and at the end of the file the answer is no - so false, which ends the while loop.

Why the other options are wrong:
A) Reaching the end of a file is normal and not an error at all.
B) It is a boolean method; it returns true or false, never -1.
D) true means at least one more line remains.

4. **Answer: A** — In the current working directory

Why: Relative paths resolve against the working directory - the folder the program was launched from, usually the project folder in an IDE.

Why the other options are wrong:
B) Java searches exactly one place: the working directory.
C) Only if the desktop happens to be the working directory, which it normally is not.
D) Java's own installation folder is never searched for your data files.

Lesson 9.4 — Writing and Appending Files

1. **Answer: C** — Writes the text into the file and moves to its next line

Why: PrintWriter's println behaves like System.out.println, but the destination is the file.

Why the other options are wrong:
A) The screen is System.out's job; this PrintWriter writes into the file.
B) Closing is done by close() or by the end of a try-with-resources block.
D) Reading is Scanner's job, not PrintWriter's.

2. **Answer: A** — The file is closed automatically

Why: Closing is the whole point: the resource declared in the header is closed at block end, exception or not.

Why the other options are wrong:
B) Closing a file never deletes it.
C) Nothing pauses; execution simply continues after the block.
D) That describes forgetting close in the old style; try-with-resources closes at once.

3. **Answer: B** — IOException

Why: FileWriter declares IOException, so the compiler expects a plan - typically catch (IOException e).

Why the other options are wrong:
A) That is a bad array index.

C) That is division by zero, not file work.
D) That is bad text-to-number conversion.

4. **Answer: D** — It is out of scope and cannot be used

Why: The variable exists only inside the try block, and by the time the block ends the file is already closed.

Why the other options are wrong:
A) Variables never change type in Java.
B) The variable is deliberately scoped to the block.
C) out is a PrintWriter object, not the text that was written.

Part 2 — Practice and Mixed Quiz Answers

Chapter 1 — Getting to Know Java

Practice Questions

1. **Answer: C** — Prints Hi and then moves to a new line

Why: println prints its text and then moves the output position to the beginning of the next line.

Why the other options are wrong:
A) Staying on the same line is what print does, not println.
B) Printing to the screen and saving files are different operations.
D) Computers do not speak; println writes text to the output screen.

2. **Answer: D** — Hello.class

Why: The compiler writes the bytecode into Hello.class, using the class name exactly as written in the source file.

Why the other options are wrong:
A) javac does not create backup copies.
B) The source code is already in Hello.java; the new file holds bytecode.
C) The class was written as Hello with a capital H, so the file is Hello.class.

3. **Answer: A** — public static void main(String[] args)

Why: The exact signature public static void main(String[] args) is the entry point the JVM looks for. For now, memorize it as a fixed phrase.

Why the other options are wrong:
B) Main with a capital M is a different name; the JVM only starts at main.
C) The return type must be void, not int, for the entry point.
D) void must come before main, and the parameter type String[] is required.

4. **Answer: D** — It executes Java bytecode

Why: The Java Virtual Machine loads .class bytecode and executes it, one instruction at a time.

Why the other options are wrong:
A) Drawing windows is done by GUI libraries running on top of the JVM, not by the JVM itself.
B) Internet connections come from library code, not from the JVM's core job.
C) You produce source files with an editor; the JVM never watches you type.

5. **Answer: B** — A missing semicolon at the end of a statement

Why: A missing semicolon is broken punctuation that the compiler rejects before the program runs.

Why the other options are wrong:
A) Both println and print are valid method names; choosing either compiles.
C) Printing twice is legal; it just produces extra output.
D) Comments are ignored, so even misspelled ones compile fine.

6. **Answer: False**. **Why:** Comments are skipped entirely. They never reach the bytecode and cannot affect the running program in any way.

7. **Output:** Java on the first line, Rocks! on the second line. The first two statements join into one line because print does not move to a new line, and println("va") finishes the line.

8. **Bug and fix:** A semicolon is missing at the end of the third line. It should read System.out.println("Line one");

Why: Every statement needs its ending semicolon. The compiler often reports this error on the line after the mistake, because that is where it noticed the problem.

9. **Model answer:**

A compile-time error is caught by the compiler before the program runs, so the program never starts until you fix it. A logic error is not caught at all: the program runs and finishes normally, but produces the wrong result because the instructions were wrong. Runtime errors sit in between: the program starts but stops midway with an exception.

10. **Model answer:**

```
public class MyInfo {

    public static void main(String[] args) {
        // Print two lines of information
        System.out.println("Sara");
        System.out.println("I am learning Java");
    }
}
```

Mixed Quiz

1. **Answer: C** — 3 + 4

Why: Everything inside double quotes is a string literal and is printed exactly as typed, so the characters 3 + 4 appear; no calculation happens.

Why the other options are wrong:
A) The spaces are part of the text, so they are printed too.
B) 7 would require real addition, which is taught in Chapter 3; quoted text is printed literally.
D) It is a real statement, not a comment, so it prints.

2. **Answer: A** — StarBox.java

Why: The file name must match the public class name exactly, letter for letter, and end in .java.

Why the other options are wrong:
B) Lowercase letters do not match the class name StarBox.
C) .class files are produced by the compiler, not written by you.
D) An underscore changes the name, so it would not match.

3. **Answer: D** — java Hello

Why: The java command launches the JVM and tells it to run the class named Hello.

Why the other options are wrong:
A) javac compiles source code; it does not run the result.
B) You pass the class name without .class to the java command.
C) run is not a Java tool command.

4. **Answer: A** — ABC on one line

Why: print keeps writing on the same line; only println ends the line, so A, B, and C join into ABC.

Why the other options are wrong:
B) Statements run top to bottom, in the order written.

C) All three statements execute; none is skipped.
D) Only the println moves to a new line, and it moves after printing C.

5. **Answer: C** — To leave notes that humans read and computers ignore

Why: Comments explain the code to people; the compiler skips them completely, so they never affect speed or output.

Why the other options are wrong:
A) To print text you use println, not comments.
B) Comments document code; hiding code permanently is not their job.
D) Comments are skipped before execution, so they cannot change speed.

6. **Answer: B** — System.out.Println("Hi");

Why: The method is println with a lowercase l (short for line). Println with a capital P does not exist, and Java is case-sensitive.

Why the other options are wrong:
A) Different text inside the quotes does not matter; the spelling of println is correct here.
C) This is the standard, correct spelling.
D) print is a real method; this line compiles.

7. **Answer: B** — The intermediate instructions the compiler produces for the JVM

Why: javac translates source code into bytecode stored in a .class file, and the JVM executes those instructions.

Why the other options are wrong:
A) The text you type is source code, which comes before bytecode.
C) Bytecode is ordinary program data, not a password.
D) That is not a technical term; bytecode is the .class instruction set.

8. **Answer: D** — One .class file running on any system that has a JVM

Why: Because bytecode targets the JVM rather than any one operating system, the same compiled file normally runs wherever a JVM exists.

Why the other options are wrong:
A) The slogan says nothing about downloads.
B) Testing is essential; the slogan is about running, not skipping tests.
C) Someone still has to write the code.

9. **Answer: True**. **Why:** Source files use .java. The compiler then produces .class files containing bytecode.

10. **Answer: False**. **Why:** Execution always begins inside the main method, even if other lines, comments, or statements appear earlier in the file.

11. **Answer: True**. **Why:** Block comments run from /* to the next */ and may span any number of lines in between.

12. **Output:** Line 1: Java. Line 2: Is Fun Today. The three print/println calls after the first join onto one line, and the final println ends it.

13. **Bug and fix:** The closing brace of the main method is missing. The class brace and the main brace each need their own }, so one more } must appear before the final one.

Why: Braces come in pairs. Here { is opened twice (class and main) but closed only once, so the compiler reports that it reached the end of the file while still inside a block.

14. **Model answer:**

The compiler does not translate Java into the machine language of one particular computer. Instead it produces bytecode, a neutral instruction set. Every operating system has its own JVM that knows how to execute that same bytecode, so the identical .class file runs on each of them.

15. **Model answer:**

```
public class Banner {

    public static void main(String[] args) {
        // prints the word JAVA
        System.out.println("JAVA");
        // prints the word ROCKS
        System.out.println("ROCKS");
    }
}
```

Chapter 2 — Variables and Data Types

Practice Questions

1. **Answer: C** — a is 2 and b is 2

Why: After a = b, a holds 2. The next line copies a's current value (2) into b, so both end at 2; the original 1 is gone.

Why the other options are wrong:
A) The swap never happens: by the time b = a runs, a already holds 2. Swapping needs a third variable to remember the lost value.
B) Both assignments overwrite the initial values.
D) a was reassigned; it no longer holds 1.

2. **Answer: B** — 5!

Why: Left to right: 2 + 3 adds numbers to 5, then 5 + "!" glues text, giving 5!.

Why the other options are wrong:
A) The final + involves a String, so the output ends as text with the ! attached.
C) The first + is numeric; gluing only begins once a String appears.
D) Concatenation inserts no spaces of its own.

3. **Answer: D** — double rate = 7.5;

Why: 7.5 has a fraction, so the type must be double, and the declaration follows type-then-name-then-value.

Why the other options are wrong:
A) int cannot hold 7.5; the compiler rejects the line.
B) Single quotes are for single chars, not numbers or multi-character text.
C) The type goes in front of the name, not after the value.

4. **Answer: B** — 10

Why: *= triples 6 to 18, then -= removes 8, leaving 10.

Why the other options are wrong:
A) 18 is the value after the first statement only.
C) The operations run in order: first multiply, then subtract.
D) Nothing goes negative; 18 - 8 is 10.

5. **Answer: A** — in.nextDouble()

Why: nextDouble reads a decimal number such as 9.5.

Why the other options are wrong:
B) nextInt cannot handle the decimal point.
C) nextLine reads a whole line as text; you could not compute with it as a number.
D) next reads one word of text, not a number.

6. **Answer: True**. **Why:** Reassignment stores a new value in the box and the old value is discarded. That is exactly why they are called variables.

7. **Output:** Line 1: 16. Line 2: x holds 16. The shortcuts turn 5 into 8 (+= 3) and then into 16 (*= 2), and the second println glues the label text onto that value.

8. **Bug and fix:** total is declared but never given a value, and the program reads it anyway; the compiler reports variable total might not have been initialized. Fix: assign first, for example int total = 0; or whatever starting value makes sense.

Why: Local variables get no automatic starting value in Java. Initialization must come before the first read.

9. **Model answer:**

A name is documentation that lives next to the code: descriptive names let any reader see what each value means, which makes programs easier to read, check, and fix. Short vague names force the reader to trace the code to recover the meaning, and that invites mistakes. The compiler does not care, but every human who reads the program, including future you, does.

10. **Model answer:**

```
import java.util.Scanner;

public class TwoNumbers {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("First number: ");
        int first = in.nextInt();
        System.out.print("Second number: ");
        int second = in.nextInt();

        System.out.println("Sum: " + (first + second));
    }
}
```

Mixed Quiz

1. **Answer: A** — my score

Why: Names cannot contain spaces; write myScore or my_score instead.

Why the other options are wrong:
B) myScore2 is legal; names may end with digits.
C) myScore is legal camelCase.
D) my_score is legal; underscores are allowed.

2. **Answer: A** — About 2.1 billion

Why: int values run from about -2.1 billion to +2.1 billion; beyond that you need a bigger type.

Why the other options are wrong:
B) That is far too small; int comfortably holds millions.
C) The range is generous but finite.
D) That is far too big; 2.1 trillion does not fit in an int.

3. **Answer: D** — char letter = 'A';

Why: A char literal is one character in single quotes, and the type is spelled char, all lowercase.

Why the other options are wrong:
A) Double quotes make a String, not a char.
B) string with a lowercase s does not exist; the type is String with quotes of the matching kind.

C) Char with a capital C is not a type name.

4. **Answer: D** — Score: 73

Why: Left to right: "Score: " + 7 gives "Score: 7", then + 3 glues again. Use "Score: " + (7 + 3) for 10.

Why the other options are wrong:
A) Concatenation inserts no spaces of its own.
B) The plus signs act now; no symbols survive into the output.
C) Once a String joins in, later + glues text; the 7 is never re-added.

5. **Answer: B** — in.next()

Why: next() reads one token of text up to the next space or Enter.

Why the other options are wrong:
A) nextLine() reads the entire line, spaces included.
C) nextWord() is not a Scanner method.
D) nextInt() reads a whole number, not text.

6. **Answer: C** — System.out.print("Your age: ");

Why: print keeps the cursor on the same line, so the user's typing lands right after the prompt.

Why the other options are wrong:
A) Without a prompt the user faces a silent, waiting program.
B) println ends the line, so the answer appears one line below the prompt.
D) The \n escape adds a line break, which defeats a same-line prompt.

7. **Answer: B** — true

Why: Printing a boolean shows the lowercase word true.

Why the other options are wrong:
A) println shows the value, never the variable's name.
C) Booleans print without quotes.
D) Booleans are not numbers; the digits belong to int.

8. **Answer: C** — 3

Why: 13 divided by 5 is 2 with remainder 3, and %= keeps the remainder. Chapter 3 covers % fully.

Why the other options are wrong:
A) 13 is not a multiple of 5, so the remainder is not 0.
B) The assignment replaced the old value 13.
D) 2 is the quotient, not the remainder.

9. **Answer: False**. **Why:** String literals need double quotes; single quotes are for one-char char values, and 'Kim' has several characters anyway. Write String name = "Kim";.

10. **Answer: False**. **Why:** int tops out near 2.1 billion in each direction; five billion is far beyond it.

11. **Answer: True**. **Why:** Imports live at the top of the file, before the class; the compiler needs the location before it reads the code that uses Scanner.

12. **Output:** Line 1: Coins: 12. Line 2: 3.0. The shortcuts leave coins at 12; dividing by the double 4.0 widens the computation, giving the double 3.0.

13. **Bug and fix:** Int with a capital I is not a type; Java is case-sensitive and the type is spelled int. Change Int to int and the program compiles.

Why: Types have exact spellings: int, double, char, and boolean are all lowercase, and String alone starts with a capital S.

14. **Model answer:**

The quotes make "2" a String, so the + is concatenation, not addition: Java converts the int 2 to the character "2" and glues, printing 22. To get 4, the friend must remove the quotes (2 + 2) or compute the addition separately and concatenate the result. Quotes turn digits into text, and text plus a number is longer text.

15. **Model answer:**

```
import java.util.Scanner;

public class AskAge {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("What is your age? ");
        int age = in.nextInt();

        System.out.println("You are " + age
                + " years old.");
    }
}
```

Chapter 3 — Operators and Expressions

Practice Questions

1. **Answer: C** — 3

Why: Both operands are ints, so 17 / 5 truncates to 3.

Why the other options are wrong:
A) 3.4 needs a double operand; ints truncate.
B) 2 is the remainder (17 % 5), not the quotient.
D) Truncation drops the fraction rather than rounding up.

2. **Answer: B** — 33

Why: 1 + 2 adds first (3), then 3 + "3" glues: 33.

Why the other options are wrong:
A) No spaces are inserted by concatenation.
C) The last operand is a String, so the output ends as the text 33.
D) The 1 and 2 add before the String appears; only the final + glues.

3. **Answer: B** — 12

Why: The cast truncates toward zero, dropping the fraction.

Why the other options are wrong:
A) The cast does not extract the fraction; it deletes it.
C) Casting never rounds up; that is Math.round's job.
D) An int cannot hold 12.9; the fraction is dropped.

4. **Answer: D** — score >= 60 && score <= 100

Why: && requires both ends: at least 60 and at most 100.

Why the other options are wrong:
A) That rejects exactly 100, which the range should include.
B) || is also true for 55 or 105; it never narrows a range.
C) Java has no chained comparisons; that line does not compile.

5. **Answer: A** — 4.0, a double

Why: The square root of 16 is 4.0, and Math.sqrt returns a double.

Why the other options are wrong:
B) 2.0 is the fourth root of 16; sqrt asks for the square root.
C) 8 would be 16 doubled, not its square root.
D) Even for whole answers the return type is double, so it prints 4.0.

6. **Answer: True**. **Why:** % works on doubles as well as ints (the remainder sign still follows the dividend), so 7 % 2.0 is 1.0.

7. **Output:** Line 1: 7. Line 2: 5. Line 3: 47 total. Six goes into 47 seven times (42), leaving 5; the last line glues the label onto the number.

8. **Bug and fix:** The program prints Half of 7 is 3, because 7 / 2 with two ints truncates the fraction. If the goal is 3.5, compute in double arithmetic: double result = 7.0 / 2; or (double) 7 / 2.

Why: The operand types decide the division; ints mean integer division even when the result is meant to be a decimal.

9. **Model answer:**

Parentheses override precedence. In (a + b) / 2 the sum is formed first and then divided by 2, which is the usual average. In a + b / 2 the division binds tighter, so only b is halved before the addition, giving a plus half of b. Same symbols, different groupings, different answers.

10. **Model answer:**

```
import java.util.Scanner;

public class SplitTime {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Seconds: ");
        int totalSeconds = in.nextInt();
        int minutes = totalSeconds / 60;
        int seconds = totalSeconds % 60;

        System.out.println(totalSeconds + " seconds is "
                + minutes + " min " + seconds + " sec");
    }
}
```

Mixed Quiz

1. **Answer: C** — 1

Why: 10 is three 3s with 1 left over; % gives the remainder.

Why the other options are wrong:
A) 10 is not a multiple of 3, so the remainder is not 0.
B) 3 is the divisor, not the remainder.
D) % gives a whole-number remainder, never a decimal.

2. **Answer: D** — 3.5

Why: A double operand forces double division, so the fraction survives: 3.5.

Why the other options are wrong:
A) 3.0 loses the .5 that double division keeps.
B) 3 would be the integer division of 7 / 2; the 2.0 prevents that.
C) Nothing rounds up here; the exact quotient is 3.5.

3. **Answer: A** — System.out.println("Sum: " + (2 + 3));

Why: The parentheses force the addition to happen before the String absorbs the result.

Why the other options are wrong:
B) Without parentheses the glue starts at the 2, printing Sum: 23.
C) The text Sum: needs double quotes.
D) * runs before the gluing anyway, printing Sum: 6.

4. **Answer: C** — -3

Why: Casting truncates toward zero: the fraction of -3.9 is dropped, leaving -3.

Why the other options are wrong:
A) The sign survives; only the fraction is dropped.
B) -4 would be rounding or flooring down; Java truncates toward zero instead.
D) An int cannot hold -3.9.

5. **Answer: D** — 3

Why: Halves round up: 2.5 becomes 3 (returned as a long, printed as 3).

Why the other options are wrong:
A) Halves go up, not down.
B) The result is a whole number; the .5 does not survive.
C) 0.5 is not related to rounding 2.5.

6. **Answer: B** — 8.0, a double

Why: pow always returns a double: 2 to the power 3 is 8.0.

Why the other options are wrong:
A) pow multiplies repeatedly (2 * 2 * 2), it does not add.
C) Even with int arguments the answer arrives as a double.
D) 9.0 would be 3 squared; the arguments are 2 then 3.

7. **Answer: A** — true

Why: && binds tighter: (true && false) is false, and false || true is true.

Why the other options are wrong:
B) println always prints something for a boolean.
C) The expression is legal; booleans mix freely with && and ||.
D) Evaluate && first: the AND fails, but the final OR with true succeeds.

8. **Answer: B** — n is even

Why: An even number divided by 2 leaves remainder 0, so the comparison is true exactly for even values.

Why the other options are wrong:
A) Odd numbers leave remainder 1, making the comparison false.
C) Zero satisfies it, but so does every other even number; the condition is not limited to zero.
D) Multiples of 10 are even, but so is every other even number; the condition is broader.

9. **Answer: True**. **Why:** Precedence runs ! first, then &&, then ||, so !hasTicket && ok means (!hasTicket) && ok.

10. **Answer: False**. **Why:** && short-circuits: a false left side settles the answer, and the right side is skipped entirely.

11. **Answer: False**. **Why:** It is the reverse: arithmetic runs first, so x + 1 > 5 compares the sum, not x to 6.

12. **Output:** Line 1: 7 (the last digit). Line 2: 6 (n / 10 is 456, then % 10 keeps its last digit). Line 3: 4 (the thousands digit). Division and remainder by powers of ten peel digits one at a time.

13. **Bug and fix:** The middle line stores instead of comparing: age = 18 assigns 18 to age, and an int cannot be stored into a boolean, so the compiler rejects it. The comparison meant here is age == 18 (or, logically, age >= 18).

Why: = and == look similar, but one stores and one compares; in a boolean context the mix becomes a type error.

14. **Model answer:**

Short-circuit evaluation is a rule about one expression: for &&, a false left side settles the answer so the right side is never computed, and for ||, a true left side does the same. It lets a safety check on the left guard a risky right side. First-match branching is a different mechanism: it picks which block of statements to run among alternatives, like if and else-if. One skips an operand; the other selects between blocks.

15. **Model answer:**

```
public class Reverse3 {

    public static void main(String[] args) {
        int number = 375;

        int last = number % 10;          // 5
        int middle = number / 10 % 10;   // 7
        int first = number / 100;        // 3
        int reversed = last * 100 + middle * 10 + first;

        System.out.println(reversed);    // 573
    }
}
```

Chapter 4 — Making Decisions

Practice Questions

1. **Answer: A** — Hot, then Bye on the next line

Why: 31 > 30 is true, so Hot prints; the last println is outside the if and always runs.

Why the other options are wrong:
B) Hot prints first because the condition is true.
C) The final println is not part of the if body, so it always runs.
D) At least one line always prints here.

2. **Answer: A** — Exactly one

Why: First-match branching: the chain stops at the first true condition, so at most one branch ever runs.

Why the other options are wrong:
B) The chain picks a single branch and finishes.
C) One branch always runs if any condition matches; else is only the leftover case.
D) Later true conditions are never even tested once a branch has matched.

3. **Answer: B** — n >= 1 && n <= 10

Why: Both bounds must hold at the same time, which is exactly what && expresses.

Why the other options are wrong:
A) With ||, every number satisfies at least one side, so it is always true.
C) This misses n = 10, which the range must include.
D) Java has no chained comparison; 1 <= n <= 10 does not compile.

4. **Answer: D** — Compile error: the condition is not a boolean

Why: A single = assigns 10 to n, and the result is the int 10, not a boolean, so the if condition is rejected.

Why the other options are wrong:
A) Java never rewrites = into ==; the compiler only reports the type error.
B) Whether the body would run is beside the point: the code does not compile at all.

C) Assignment is not comparison; the line never compiles.

5. **Answer: C** — Easy, then Medium

Why: Case 1 runs, finds no break, and falls into case 2, whose body prints Medium and then breaks.

Why the other options are wrong:
A) The fall-through stops at case 2's break, so Hard is never reached.
B) Fall-through is legal; the compiler stays silent.
D) Only a break (or the end of the switch) stops the run, and case 1 has neither.

6. **Answer: False**. **Why:** First-match branching stops at the FIRST true condition; later branches are skipped without being tested, even if they would also be true.

7. **Output:** Line 1: Good. Line 2: Score: 78. The 90 branch fails, the 70 branch matches and ends the chain, and the final println sits outside the chain so it always runs.

8. **Bug and fix:** The semicolon right after if (age >= 18) is the bug. It ends the if with an empty statement as its body, so the if controls nothing, and the brace block below runs every time. Delete the semicolon so the braces become the body.

Why: A lone semicolon is a legal empty statement, so the compiler stays silent while the logic quietly breaks.

9. **Model answer:**

Java tests the conditions top to bottom and stops at the first true one, so a broad condition placed early swallows values that later branches were meant to catch. If score >= 60 comes before score >= 90, a score of 95 prints the D-or-better message and the A branch is unreachable. The fix is to order branches from most specific to most general, ending with a catch-all else.

10. **Model answer:**

```
import java.util.Scanner;

public class EvenOdd {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter a whole number: ");
        int n = in.nextInt();
        if (n % 2 == 0) {
            System.out.println("Even");
        } else {
            System.out.println("Odd");
        }
    }
}
```

Mixed Quiz

1. **Answer: D** — if (score >= 60)

Why: The condition needs parentheses and no semicolon; the body follows the header.

Why the other options are wrong:
A) That stray semicolon ends the statement with an empty body, orphaning the code below it.
B) The condition must be wrapped in parentheses.
C) Braces wrap the body, not the condition.

2. **Answer: A** — child

Why: The first condition, 12 < 13, is true, so that branch runs and the chain ends immediately.

Why the other options are wrong:
B) The teen branch needs the first condition to fail, which it does not.
C) adult requires both earlier conditions to be false.
D) Only one branch of a chain can ever run.

3. **Answer: C** — n < 1 || n > 10

Why: Outside the range means below 1 or above 10, so the two cases join with ||.

Why the other options are wrong:
A) No number is both below 1 and above 10, so this is never true.
B) The <= and >= wrongly include 1 and 10, which are inside the range.
D) Unwrapping the ! shows below 1 AND above 10, the same impossible trap as option B.

4. **Answer: A** — To leave the switch after the matching case

Why: break jumps out of the switch, which stops one case's body from falling into the next.

Why the other options are wrong:
B) break only exits; jumping back would restart the match, which is not its job.
C) The colon after a label does that.
D) break leaves the switch, not the program.

5. **Answer: C** — Early

Why: Stacked labels share one body: day 2 jumps to case 2, finds no statements, and falls straight into the shared println.

Why the other options are wrong:
A) case 2 shares the body of case 1 through deliberate fall-through.
B) The shared body runs once; stacking labels does not duplicate it.
D) Empty case labels used for grouping are perfectly legal.

6. **Answer: B** — started == false

Why: !started is true when started is false, exactly what started == false tests.

Why the other options are wrong:
A) Not equal to false means equal to true, again the opposite.
C) The bare flag is false here, while !started is true.
D) That is true when started is true, the opposite.

7. **Answer: B** — x > 0 && x < 10

Why: Reaching the inner body required the outer condition AND the inner one, so the single condition joins them with &&.

Why the other options are wrong:
A) || is satisfied when only one side holds, which the nesting never allowed.
C) That demands x <= 0, reversing the outer check.
D) and is an English word, not a Java operator.

8. **Answer: D** — When a is true and b is false

Why: The else pairs with the NEAREST if, the b check, so two() runs when a passes but b fails.

Why the other options are wrong:
A) a false skips everything, so only the a true, b false case reaches two().
B) Nested ifs with one else compile legally; the pairing is just surprising.
C) When a is false, the whole inner if/else is skipped.

9. **Answer: False**. **Why:** else means otherwise: it runs when every condition above it failed and takes no condition of its own.

10. **Answer: False**. **Why:** A switch jumps directly to the label that equals the value; reordering cases changes nothing about which one matches (only how fall-through flows).

11. **Answer: False**. **Why:** The condition must be a boolean; 0 is an int, so the compiler rejects the line. Java never treats numbers as true or false.

12. **Output:** Two lines: Three, then Four. The match is case 3, there is no break after Three, so execution falls into case 4, prints Four, and the break there ends the switch.

13. **Bug and fix:** The if has no braces, so it owns only the first println. The second println sits outside the if and runs no matter what the temperature is. Wrap both println statements in one brace-enclosed body.

Why: Indentation fooled the eye; the compiler pairs the if with exactly one statement below it.

14. **Model answer:**

Fall-through is what happens when a case body has no break: execution continues into the next case's statements instead of leaving the switch. As a bug, one match prints several cases' output, such as Easy followed by Medium. On purpose, stacking labels like case 1: case 2: with one shared body groups several values behind a single break.

15. **Model answer:**

```
import java.util.Scanner;

public class PassFail {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter your score: ");
        int score = in.nextInt();
        if (score >= 60) {
            System.out.println("Pass");
        } else {
            System.out.println("Fail");
        }
    }
}
```

Chapter 5 — Loops

Practice Questions

1. **Answer: A** — 4 times

Why: i takes the values 0, 1, 2, and 3; each value passes the test i < 4 once, so the body runs four times before i becomes 4.

Why the other options are wrong:
B) Three would mean starting at 1; the loop starts at 0.
C) Five values would need i <= 4, not i < 4.
D) The update i++ eventually makes the condition false, so the loop stops.

2. **Answer: D** — The body never runs

Why: A while loop checks the condition BEFORE each pass, so a false first check skips the body entirely.

Why the other options are wrong:
A) Running at least once is the do-while loop's behavior, not while's.
B) Skipping a loop is perfectly normal; no error occurs.
C) A false condition ends the loop; nothing runs.

3. **Answer: C** — do-while

Why: do-while checks its condition at the bottom, after the body has already run one full pass.

Why the other options are wrong:
A) for also checks its condition before the first pass.
B) One of them does have this guarantee: do-while.
D) while checks first and can skip the body completely.

4. **Answer: B** — 4 times

Why: i takes the values 2, 4, 6, and 8; each satisfies i <= 8, then 10 fails the test.

Why the other options are wrong:
A) Six passes would need values up to 12.
C) That would count 2, 4, 6 only; 8 also passes the test.
D) The step is +2, so i never visits most values up to 8.

5. **Answer: B** — 12

Why: Nested loops multiply: 3 outer passes, each running the inner loop 4 times, gives 3 x 4 = 12 inner passes.

Why the other options are wrong:
A) 9 would be 3 x 3, but the inner loop runs 4 times.
C) That counts only one outer pass.
D) Loops do not add their counts when nested; each inner run happens once per outer pass.

6. **Answer: True**. **Why:** In a for loop, continue triggers the update expression, then the condition. In a while loop there is no update step, which is why continue is riskier there.

7. **Output:** 3, then 2, then 1, each on its own line, and then Liftoff! on a fourth line. The loop runs downward because of i--, and Liftoff! prints after the loop ends.

8. **Bug and fix:** There is a semicolon right after the while condition. That semicolon is an empty loop body, so count never changes and the loop spins forever; the brace block below is not part of the loop at all. Remove the semicolon.

Why: A semicolon directly after while (...) ends the loop statement. The loop then has no body, the condition stays true forever, and the program hangs without printing anything.

9. **Model answer:**

Input validation is the classic case: you must ask the user for a value at least once before you can check whether it is valid. A do-while runs the read-and-check body first and repeats only while the input is still invalid. A while loop would need a duplicated read before the loop just to have something to test.

10. **Model answer:**

```
public class Evens {

    public static void main(String[] args) {
        for (int i = 2; i <= 10; i += 2) {
            System.out.println(i);
        }
    }
}
```

Mixed Quiz

1. **Answer: C** — The loop variable is never updated, so n stays 5

Why: The body never changes n, so the condition n > 0 stays true forever. An update such as n-- is required inside the body.

Why the other options are wrong:
A) The println already ends with a semicolon.
B) With > and no update the direction does not even matter; the frozen variable is the real problem.
D) Something IS missing: without an update this is an infinite loop.

2. **Answer: B** — for (init; condition; update)

Why: The header is for (initialize; condition; update): run init once, test the condition before every pass, and apply the update after each pass.

Why the other options are wrong:
A) The condition is tested first, but it is written second in the header.
C) The header always starts with the initialization.
D) Swapping condition and update changes the meaning; the test must come between them.

3. **Answer: A** — 6

Why: Each of the 2 outer passes runs the 3-pass inner loop: 2 x 3 = 6 stars in total.

Why the other options are wrong:
B) That counts only the outer loop.
C) That counts a single outer pass.
D) Loops multiply when nested; they do not add.

4. **Answer: B** — Exits the nearest enclosing loop immediately

Why: break leaves the nearest loop at once; the program continues with the first statement after that loop.

Why the other options are wrong:
A) That is continue's job.
C) break does not restart anything; it exits.
D) The program keeps running after the loop ends.

5. **Answer: D** — A special input value that tells the loop to stop

Why: Sentinel-controlled loops keep processing input until a agreed-upon value appears, such as 0 or -1, which means stop.

Why the other options are wrong:
A) A counter is different; a sentinel arrives from the data, not from counting.
B) The loop condition is code; the sentinel is a data value.
C) The first value has no special stopping role.

6. **Answer: D** — A semicolon is required after while (...); at the end

Why: The do-while header ends with a semicolon: do { ... } while (condition); Forgetting it is a compile error.

Why the other options are wrong:
A) break works inside every kind of loop.
B) That is while's behavior; do-while checks at the bottom.
C) Any loop can be infinite if its condition never becomes false.

7. **Answer: C** — It no longer exists; it was created inside the loop

Why: A variable declared in the for header is scoped to the loop. After the loop ends, i is out of scope and cannot be used.

Why the other options are wrong:
A) The last value inside the loop was 4, but the variable is gone afterward.
B) Nothing resets it; it simply goes out of scope.
D) Even though i reaches 5 before the loop stops, it does not survive the loop.

8. **Answer: A** — 3

Why: The loop increments i until it is 3, then stops. The println after the loop prints the final value 3.

Why the other options are wrong:
B) The println is outside the loop, so it always runs.
C) i only starts at 0; it is incremented three times.
D) 2 would be the last value inside the loop's final pass, but i becomes 3 before the test fails.

9. **Answer: True**. **Why:** That is exactly why a while loop can run zero times: if the condition is false on the first check, the body is skipped entirely.

10. **Answer: False**. **Why:** It was declared inside the for header, so its scope ends with the loop. To keep the value, declare the variable before the loop.

11. **Answer: True**. **Why:** continue always jumps to the nearest loop's next-cycle point: the update in a for loop, the condition in a while loop. Statements after it in the body are skipped.

12. **Output:** 10. The accumulator adds 1 + 2 + 3 + 4, and the println after the loop prints the final sum.

13. **Bug and fix:** An off-by-one error: the condition i < 5 stops the loop before 5 is printed. It should be i <= 5 to include the number 5.

Why: With i < 5 the loop visits 1, 2, 3, and 4 only. When a range is meant to be inclusive, the condition needs <=.

14. **Model answer:**

An infinite loop happens when the loop's condition never becomes false, usually because the loop variable is never updated or is updated in the wrong direction. A running infinite loop is stopped from outside with an interrupt such as Ctrl+C in the terminal (or the stop button in an IDE). The real fix is editing the code so the condition can eventually fail.

15. **Model answer:**

```
public class OddSum {

    public static void main(String[] args) {
        int total = 0;
        int n = 1;
        while (n <= 9) {
            total += n;
            n += 2;
        }
        System.out.println(total);
    }
}
```

Chapter 6 — Methods

Practice Questions

1. **Answer: C** — cheer();

Why: A call is the method's name, then parentheses for any arguments, then a semicolon: cheer();

Why the other options are wrong:
A) call is not a Java keyword; the name and parentheses alone do the job.
B) Without parentheses you are not calling anything; the compiler reports an error.
D) void belongs in the definition's header, never in a call.

2. **Answer: D** — show("Seats", 12);

Why: Arguments fill the parameters in order: a String first, then an int, with both supplied.

Why the other options are wrong:
A) Again only one argument for a two-parameter method.

B) The order is reversed; 12 cannot fill a String parameter.
C) Only one of two parameters was filled; the count must match.

3. **Answer: A** — 8

Why: total's value is copied into t, only t is changed, and the copy vanishes; main's total is still 8.

Why the other options are wrong:
B) Passing an int and changing the parameter is legal; it just has no outside effect.
C) 10 is how much was added, but to the copy only.
D) 18 is the value of the copy t inside the method; the original never moved.

4. **Answer: B** — public static boolean isFull(int level)

Why: Answering true or false means returning a boolean, so boolean replaces void in the header.

Why the other options are wrong:
A) void hands nothing back, so it cannot answer a question.
C) An int can hold 0 or 1, but true/false questions deserve the boolean type.
D) A String could hold "true", but that is text, not the actual boolean values.

5. **Answer: B** — print(int), because 2 is an int literal

Why: The compiler prefers an exact match: 2 is an int, so the int overload runs without any conversion.

Why the other options are wrong:
A) Each call runs exactly one method.
C) The int could widen to double, but an exact match always beats a conversion.
D) Ambiguity needs two equally good fits after conversions; here one match is exact.

6. **Answer: False**. **Why:** Definition order inside a class does not matter. Java reads the whole class first, so main can call a method written below it.

7. **Output:** Line 1: 14. Line 2: 4. Line 3: 14. Each show call prints its parameter after adding 10, but the parameter is a copy, so x is still 4 afterward and the second call prints 14 again.

8. **Bug and fix:** The announce method is defined inside main. Methods cannot nest: every method must sit directly inside the class, next to main, not inside another method's braces.

Why: Move announce out of main so its braces sit at class level, then call it with announce(); from main.

9. **Model answer:**

Choose void when the method performs an action for its own sake, such as printing lines, and the caller needs no answer back. Choose a return type when the caller needs the result of a calculation to store, print, or use in a bigger expression. A quick test: if you can imagine writing int x = methodName();, the method needs a return type.

10. **Model answer:**

```
public class LineMaker {

    public static void main(String[] args) {
        repeat("Java", 3);
    }

    // Prints word on its own line, times times.
    public static void repeat(String word, int times) {
        for (int i = 0; i < times; i++) {
            System.out.println(word);
        }
    }
}
```

Mixed Quiz

1. **Answer: B** — void

Why: void is the return type meaning no value is sent back to the caller.

Why the other options are wrong:
A) empty is not a Java keyword.
C) static means the method belongs to the class, not that it returns nothing.
D) null is a value meaning no object; it is not a return type keyword for methods.

2. **Answer: A** — A void method produces no value to store

Why: void means the call produces no value, so there is nothing to assign to x.

Why the other options are wrong:
B) new creates objects and plays no role in calling methods.
C) sayHello has an empty parameter list; argument count is not the problem.
D) int variables hold method results all the time, as in int x = square(5);; the problem is that a void call has no result.

3. **Answer: D** — main to call the method directly, with no setup

Why: A static method belongs to the class itself, which is why main can call it by name without any extra steps.

Why the other options are wrong:
A) Method definitions are fixed text; static does not let code rewrite itself.
B) static changes how a method is found, not how fast it runs.
C) Parameters are always delivered to every call; nothing skips them.

4. **Answer: C** — copies of the values passed by the caller

Why: Primitive arguments are copied into the parameters (pass-by-value), so each method works on its own private values.

Why the other options are wrong:
A) Each call creates fresh parameters visible only inside that method.
B) Parameters are created anew at every call and destroyed when it ends.
D) The method cannot reach main's variables; that is what pass-by-value prevents.

5. **Answer: A** — The compiler rejects the call: types and order do not match

Why: The arguments must fill the parameter list in order and type; an int cannot fill a String parameter, so this call does not compile.

Why the other options are wrong:
B) The call never runs, so it cannot print anything in any order.
C) That output needs report("Mina", 95) with the values in the header's order.
D) A mismatched call is a compile-time error, not a silent run.

6. **Answer: C** — greet("Ana", 3);

Why: Overloads are chosen by the arguments: two arguments here match the two-parameter list.

Why the other options are wrong:
A) One argument matches the one-parameter version.
B) No overload takes zero arguments; this call does not compile.
D) Neither overload takes an int alone, so this call does not compile.

7. **Answer: D** — Only inside main's braces

Why: A variable's scope is the innermost braces where it is declared, so main's locals stop at main's closing brace.

Why the other options are wrong:
A) There are no program-wide variables in the code we write at this stage.
B) The whole of main, not just its loops, can use a variable declared at the top.
C) Other methods cannot see main's variables; that is why parameters exist.

8. **Answer: B** — @return

Why: The @return line describes what a non-void method gives back to its caller.

Why the other options are wrong:
A) @value is not a standard method-level Javadoc tag.
C) @sends is not a Javadoc tag.
D) @out is not a Javadoc tag.

9. **Answer: False**. **Why:** Definitions only prepare methods. The body runs each time the method is called, and never at definition time.

10. **Answer: True**. **Why:** return delivers its value (if any) and stops the method on the spot; code after it on the same path can never run.

11. **Answer: True**. **Why:** That is overloading: the compiler tells the calls apart by matching arguments to the parameter lists.

12. **Output:** Line 1: 12. Line 2: 9. The first call evaluates to 12 and is stored; the second line evaluates triple(1) as 3 and triple(2) as 6, and prints their sum, 9.

13. **Bug and fix:** printHello is void and produces no value, so int x = printHello(); cannot compile. Either change the method to return a value (for example return a String or int), or simply call printHello(); without storing anything.

Why: Assigning requires a value, and a void call has none. The return type must match what the caller expects to receive.

14. **Model answer:**

The parameter receives a copy of x's value, so assignments inside the method change only the copy. When the method ends, the copy vanishes and main's x still holds its original value. For a change to stick, the method must return the new value and main must store it, as in x = changeNumber(x);.

15. **Model answer:**

```
public class Salary {

    public static void main(String[] args) {
        double before = 1200.0;
        double after = addRaise(before);
        System.out.println("Before: " + before);
        System.out.println("After: " + after);
    }

    // Returns the salary with 500 added.
    public static double addRaise(double salary) {
        return salary + 500.0;
    }
}
```

Chapter 7 — Arrays

Practice Questions

1. **Answer: C** — 0

Why: New numeric arrays arrive filled with default values, and the int default is 0.

Why the other options are wrong:
A) Index 0 is valid for a 3-element array, so nothing crashes.
B) Defaults are guaranteed; every int box starts at exactly 0.

D) 3 is the length of the array, not the contents of box 0.

2. **Answer: D** — 9

Why: nums.length is 3 and nums[2] is 6, so the sum is 9.

Why the other options are wrong:
A) 8 comes from adding the first element (2) to nums[2] (6) instead of the length.
B) 12 multiplies 2 by 6; the operator here is plus.
C) length and element access both give ints, and int + int compiles fine.

3. **Answer: B** — int[] a = new int[5];

Why: The array type int[] goes on the left, and new int with the size in brackets creates the boxes.

Why the other options are wrong:
A) a is declared as a single int, which cannot hold an array.
C) new int needs a size in brackets to know how many boxes to build.
D) The size belongs with new, not inside the type name.

4. **Answer: A** — for (int i = 0; i < arr.length; i++) { arr[i] = arr[i] * 2; }

Why: Writing back to the array needs the index, and the index loop provides arr[i] for every position.

Why the other options are wrong:
B) x is a copy of each element; doubling the copy changes nothing in the array.
C) arr[x] uses a value as an index and zeroes boxes; it neither doubles nor stays in bounds.
D) That loop never ends and never touches anything but arr[0].

5. **Answer: D** — -3

Why: The champion starts at -8, then -3 beats it, and nothing beats -3, so -3 is the largest.

Why the other options are wrong:
A) -15 is the smallest value, the result the min algorithm would find.
B) 0 is not in the array; a tracker starting there would wrongly report 0.
C) -8 is only the starting champion; -3 replaces it.

6. **Answer: False**. **Why:** Arrays are fixed size. The length is chosen at creation and never changes; if you need more room, you must create a new array and copy values over.

7. **Output:** Line 1: 20. Line 2: 4. The first loop doubles every element to 2, 4, 6, 8; the for-each loop sums those values to 20, and a[1] holds 4.

8. **Bug and fix:** The loop condition is i <= days.length, which lets i reach 4 for a 4-element array whose last index is 3. The fourth print succeeds, then index 4 stops the program with ArrayIndexOutOfBoundsException. Use i < days.length.

Why: The last valid index is always length - 1, so the classic shape of the loop is for (int i = 0; i < arr.length; i++).

9. **Model answer:**

sort works on the array you give it, rearranging the elements in place, so it has no new array to hand back and its return type is void. To show the result, call Arrays.sort(nums); first, then print nums, most easily with Arrays.toString(nums). Printing the sort call itself does not compile because there is nothing to print.

10. **Model answer:**

```
public class Totaler {

    public static void main(String[] args) {
        int[] values = {4, 9, 1};
        System.out.println(sum(values));
    }

    // Returns the total of every element in arr.
    public static int sum(int[] arr) {
        int total = 0;
        for (int i = 0; i < arr.length; i++) {
            total = total + arr[i];
        }
        return total;
    }
}
```

Mixed Quiz

1. **Answer: B** — 0

Why: Array indexes always start at 0, whatever the element type is.

Why the other options are wrong:
A) -1 is a not-found signal in searches, never a real index.
C) Every array type numbers its boxes from 0.
D) Index 1 is the second element.

2. **Answer: D** — 9

Why: length reports the size chosen at creation, here nine elements.

Why the other options are wrong:
A) 10 would be the length of an eleven-element array.
B) 8 is the last index, not the length.
C) A new array is full of default values; its length is still 9.

3. **Answer: A** — int[] fib = {3, 1, 4};

Why: The initializer list in braces fills the array at creation, and the values are separated by commas.

Why the other options are wrong:
B) new int takes one size in brackets; the initializer form needs braces, not new.
C) fib is declared as a single int, which cannot hold an array.
D) Initializer values must be separated by commas.

4. **Answer: D** — The first one, r

Why: 2D access reads row first, then column: grid[row][column].

Why the other options are wrong:
A) length reports sizes; only indexes select elements.
B) Row and column are different roles; swapping them reads a different cell.
C) The second index picks the column inside that row.

5. **Answer: A** — an array of doubles

Why: The for-each variable's type must match (or widen from) the element type, so d as a double needs double elements.

Why the other options are wrong:
B) The compiler checks the type; an int array would not fit a double variable in this book's style.
C) An int array would need an int loop variable.

D) A 2D array's elements are themselves arrays, not doubles.

6. **Answer: C** — Stores 7 into every element of data

Why: fill overwrites each box with the given value and returns void.

Why the other options are wrong:
A) fill replaces contents; it does not do arithmetic.
B) Arrays are fixed size; nothing resizes them.
D) fill returns void and edits the existing array.

7. **Answer: C** — the target was not found

Why: -1 is the standard not-found code, because no real index can be negative.

Why the other options are wrong:
A) Array size has nothing to do with the -1 signal.
B) The first element is at index 0.
D) A found match returns the actual index, 0 or higher.

8. **Answer: B** — Reading arr[10] of a 3-element array

Why: The compiler does not track sizes, so the bad index only explodes when the line runs: ArrayIndexOutOfBoundsException.

Why the other options are wrong:
A) A missing import is a compile-time cannot find symbol error.
C) A missing semicolon is caught at compile time.
D) length is a field; the parentheses are rejected before the program runs.

9. **Answer: False**. **Why:** length is a field: a stored value attached to every array. It is written without parentheses, and arr.length() does not compile.

10. **Answer: True**. **Why:** sort works in place: it rearranges the original array's elements, and its return type is void.

11. **Answer: True**. **Why:** On a 2D array, grid.length counts rows, and here there are two; each row's length is grid[r].length, which is 5.

12. **Output:** Line 1: 12. Line 2: 34. The outer loop visits row 0 then row 1; the inner loop prints each row's values with no separator, and println ends each row.

13. **Bug and fix:** Arrays.sort returns void, so there is no value to hand to println. Sort first, then print the array: Arrays.sort(nums); followed by System.out.println(Arrays.toString(nums));

Why: sort modifies the array in place and produces no result. Printing the array afterward, with toString or a loop, shows the sorted order.

14. **Model answer:**

Zero is not one of the array's values, so it is a champion from a different contest. For an array like {-8, -3, -15}, no element beats 0, so the tracker ends at 0 even though the true largest value is -3. The safe start is arr[0], the first element, which guarantees the tracker begins at a real value.

15. **Model answer:**

```
public class Finder {

    public static void main(String[] args) {
        int[] data = {8, 15, 23};
        System.out.println(find(data, 15));
        System.out.println(find(data, 99));
    }

    // Returns target's index, or -1 if absent.
    public static int find(int[] arr, int target) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                return i;
            }
        }
        return -1;
    }
}
```

Chapter 8 — Working with Strings

Practice Questions

1. **Answer: A** — 1

Why: The letters a, n appear first at indexes 1 and 2, so the piece "an" starts at index 1; indexOf reports the first match only.

Why the other options are wrong:
B) Index 2 is where "na" starts, a different piece.
C) The piece appears again starting at index 3, but indexOf stops at the first match.
D) -1 is the answer only when the piece is not in the string at all.

2. **Answer: D** — They return new strings and leave the original alone

Why: Strings are immutable: every method builds and returns a brand-new string, and the original keeps its value.

Why the other options are wrong:
A) Nothing is deleted; the original string simply stays as it was.
B) Editing in place is exactly what can never happen to a String.
C) They operate on the whole text, not single characters (charAt's job is single characters).

3. **Answer: B** — s.equals(t)

Why: equals compares the characters of both strings, including upper and lower case differences.

Why the other options are wrong:
A) equalsIgnoreCase deliberately ignores case, so OPEN equals open under it.
C) compareTo signals equality with 0, not with 1.
D) == compares object references and may say false for identical letters.

4. **Answer: C** — Fills the slot with the next argument as text

Why: Each placeholder is filled by the next argument in order; %s shows it as text.

Why the other options are wrong:
A) Starting a new line is the job of %n (or \n).
B) Rounding to two decimals is %.2f's job.
D) % introduces a placeholder, so %s inserts a value instead of printing the letter s.

5. **Answer: D** — append

Why: append tacks text onto the end of the buffer without re-copying what is already there.

Why the other options are wrong:
A) add is not a StringBuilder method.
B) concat is a String method; StringBuilder uses append.
C) insert places text at a chosen position; append is the end-of-text method.

6. **Answer: False**. **Why:** trim returns a new string with the end spaces removed; s itself never changes unless you assign the result: s = s.trim();

7. **Output:** ana on the first line, 1 on the second, BANANA on the third, and banana on the fourth. substring(1, 4) takes indexes 1 to 3, indexOf finds the first a at index 1, and the last line proves word was never changed by toUpperCase.

8. **Bug and fix:** The program prints Different words even though both variables hold cat. == compares object references, and new String built two separate objects. The fix is to compare contents: if (a.equals(b)).

Why: == never looks at the letters inside a string. With literals Java sometimes shares one object and == accidentally agrees, which is exactly why this bug hides until it bites.

9. **Model answer:**

Immutable means the characters of a string can never change after the string is created. Every String method therefore builds and returns a new string instead of editing the old one. To keep the new string you must store it with an assignment, for example s = s.toUpperCase(); otherwise the result is thrown away.

10. **Model answer:**

```
public class Shout {

    public static void main(String[] args) {
        String name = "Maya";
        System.out.println(name.toUpperCase());
        System.out.println(name.length());
        System.out.println(name.charAt(0));
    }
}
```

Mixed Quiz

1. **Answer: C** — 'a'

Why: Indexes start at 0, so index 1 is the second character, a.

Why the other options are wrong:
A) J is at index 0.
B) v is at index 2.
D) charAt returns exactly one char, never two.

2. **Answer: D** — "ephant"

Why: With one argument, substring runs from the given index all the way to the end of the string.

Why the other options are wrong:
A) That is substring(4).
B) That is substring(0, 3).
C) That is substring(2, 5): the two-argument form stops before index 5.

3. **Answer: A** — Removes the spaces at the start and end only

Why: trim builds a copy without leading or trailing spaces: "hi". Spaces in the middle stay.

Why the other options are wrong:
B) Inner spaces are untouched; only the two ends are trimmed.
C) No characters are replaced; trim only removes end spaces.

D) trim never removes letters, only spaces at the two ends.

4. **Answer: B** — true

Why: equalsIgnoreCase compares letters while ignoring upper and lower case, so the two words match.

Why the other options are wrong:
A) -1 belongs to indexOf and compareTo, not to an equals-style test.
C) The method returns a boolean, not a string.
D) equals would say false, but equalsIgnoreCase forgives the case difference.

5. **Answer: B** — When the two strings are equal

Why: Zero is compareTo's signal that the strings hold exactly the same characters.

Why the other options are wrong:
A) Different lengths give a negative or positive result, not zero.
C) Zero is precisely the equal-strings answer.
D) If t comes first, s.compareTo(t) is positive.

6. **Answer: C** — System.out.printf("%.2f%n", price);

Why: %.2f formats a double with two decimal places, rounded normally.

Why the other options are wrong:
A) %2d is for integers and expects an int, not a double.
B) Two specifiers need two arguments, but only one was supplied.
D) %s prints the value but does not fix the number of decimals.

7. **Answer: D** — System.out.printf("[%6s]%n", "cat");

Why: %6s reserves six slots and right-aligns the shorter text inside them, padding with spaces on the left.

Why the other options are wrong:
A) A precision such as %.6s caps the text length; it does not pad to a width.
B) The minus left-aligns, giving [cat].
C) Without the % before 6s, the text [6s] is printed literally.

8. **Answer: A** — hi

Why: toUpperCase returned a new string that was never stored, so s still holds hi.

Why the other options are wrong:
B) HI would print only after s = s.toUpperCase();
C) Java does not capitalize just the first letter in this method call.
D) println runs normally and prints whatever s holds.

9. **Answer: True**. **Why:** Index 1 is included and the end index 2 is excluded, so exactly one character, b, comes out.

10. **Answer: False**. **Why:** printf prints only what the format string contains; you add %n or \n yourself.

11. **Answer: True**. **Why:** -1 is the standard not-found answer, which makes tests like if (s.indexOf("@") == -1) easy to write.

12. **Output:** Items: 3 on the first line, Price: 9.50 on the second (%.2f adds the trailing zero), and [cat] on the third - three spaces of left padding, since %6s right-aligns in width 6.

13. **Bug and fix:** It prints quiet. toUpperCase builds a new string and returns it, but the program never stores the result, so word still holds quiet. The fix is word = word.toUpperCase();

Why: Strings are immutable, so no String method ever changes word in place. Any method result you do not assign is simply lost.

14. **Model answer:**

Because strings are immutable, every + creates a whole new string and copies everything accumulated so far; with thousands of pieces the program spends most of its time copying. StringBuilder keeps one growable buffer and appends each piece without re-copying. Its key method is append, and toString hands back the finished String at the end.

15. **Model answer:**

```
public class LastHalf {

    public static void main(String[] args) {
        String word = "Elephant";
        int half = word.length() / 2;
        System.out.println(word.substring(half));
    }
}
```

Chapter 9 — Files and Exceptions

Practice Questions

1. **Answer: C** — try

Why: The risky statements go inside the try block; the response to a problem lives in catch.

Why the other options are wrong:
A) catch holds the response, which runs only after a problem occurs.
B) finally is for cleanup that normally runs either way.
D) throw creates a problem on purpose; it guards nothing.

2. **Answer: A** — catch (InputMismatchException e)

Why: Scanner's nextInt throws InputMismatchException when the input cannot be read as an int.

Why the other options are wrong:
B) FileNotFoundException is about opening files, not keyboard input.
C) ArithmeticException is for division by zero and friends.
D) IllegalStateException signals a method called at the wrong moment.

3. **Answer: A** — The compiler forces you to handle FileNotFoundException

Why: FileNotFoundException is checked: no try/catch and no throws means the code does not compile. ArithmeticException is unchecked.

Why the other options are wrong:
B) Every exception can be caught, checked or not.
C) It is exactly backwards: file exceptions are the checked ones.
D) The checked/unchecked difference is precisely what sets them apart.

4. **Answer: B** — while (in.hasNextLine())

Why: hasNextLine is true while at least one more line remains, so the loop stops exactly at the end of the file.

Why the other options are wrong:
A) until is not a Java keyword.
C) A fixed count reads the wrong amount for any file that is not exactly three lines.
D) nextLine returns a String, not a boolean, so this does not compile.

5. **Answer: D** — Automatically at the end of the try block

Why: Automatic closing at block end - even when an exception happens - is the feature's whole purpose.

Why the other options are wrong:
A) Much sooner than that; the close happens when the block ends.

B) That is the old manual style; the modern form closes by itself.
C) Disk files do not fill up like glasses of water.

6. **Answer: False**. **Why:** PrintWriter buffers the text; only close (or flush) guarantees it reaches the disk. Try-with-resources closes for you at the end of the block.

7. **Output:** Six lines, in order: A, B, 0, D, E, F. The 0 is nums[1] holding the int-array default; nums[5] then throws, so C never prints; the catch prints D; finally prints E; and the program continues to F.

8. **Bug and fix:** The compiler reports an unhandled exception: FileNotFoundException is checked, and main neither catches it nor declares it. Wrap the reading code in try { ... } catch (FileNotFoundException e) { ... }, or add throws FileNotFoundException to the main line.

Why: Checked exceptions are the compiler's contract: be ready for realistic trouble such as a missing file. The error message names the exception and usually suggests both fixes.

9. **Model answer:**

Unchecked exceptions, such as ArithmeticException, belong to the RuntimeException family: the compiler does not force you to handle them, because they usually come from logic mistakes you should fix. Checked exceptions, such as FileNotFoundException, describe trouble a correct program can still meet, like a missing file. Checked exceptions are the ones that force a try/catch or a throws declaration.

10. **Model answer:**

```
import java.util.InputMismatchException;
import java.util.Scanner;

public class SafeDouble {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter a whole number: ");
        try {
            int n = in.nextInt();
            System.out.println("Twice: " + 2 * n);
        } catch (InputMismatchException e) {
            System.out.println("That was not a "
                    + "whole number");
        }
    }
}
```

Mixed Quiz

1. **Answer: D** — Stopped, with an exception report printed

Why: With no catch anywhere, the program stops at the crashing statement and the red report explains why.

Why the other options are wrong:
A) No pause happens; stopping is immediate.
B) The crashing statement is the end of the road when nobody catches.
C) Nothing keeps running; the program has ended.

2. **Answer: B** — finally

Why: try finishes cleanly, catch is skipped entirely, and finally runs - normally in every case.

Why the other options are wrong:
A) catch is skipped when nothing goes wrong.
C) They run in a fixed order, not all at once.

D) try runs first, not last.

3. **Answer: A** — throw

Why: throw new SomeException(...) builds the object and fires it immediately.

Why the other options are wrong:
B) catch receives an exception; it does not create one.
C) try guards statements; it produces nothing.
D) throws only declares a possibility on a method signature.

4. **Answer: C** — java.io.File

Why: File lives in the java.io package, the home of Java's input/output classes.

Why the other options are wrong:
A) That package holds window and drawing tools, not file classes.
B) File is in java.io, not java.util.
D) Scanner lives in java.util; File does not.

5. **Answer: D** — NumberFormatException

Why: parseInt refuses text that is not a whole number by throwing NumberFormatException.

Why the other options are wrong:
A) That is Scanner's version of bad input; parseInt throws its own type.
B) ArithmeticException is about division by zero, not parsing.
C) IllegalStateException is about wrong timing, not bad text.

6. **Answer: A** — Open in append mode, keeping old content

Why: The boolean append flag keeps existing content and adds new text at the end.

Why the other options are wrong:
B) Appended files can grow without limit.
C) The flag changes behavior, not speed.
D) Overwriting is what happens WITHOUT the true flag.

7. **Answer: C** — in.hasNextInt()

Why: hasNextInt returns true only if the next item can be read as an int, so the loop reads safely.

Why the other options are wrong:
A) nextInt is the reading call itself; calling it unguarded is what crashes.
B) That checks for any line at all, number or not.
D) close disconnects the Scanner; it checks nothing.

8. **Answer: B** — Exactly one: the first whose type matches

Why: Java checks the catch blocks from top to bottom and runs the first matching one, then jumps past the rest.

Why the other options are wrong:
A) A matching catch block always runs - that is its job.
C) Order matters from the top, not from the bottom.
D) One problem gets one handler; the remaining catch blocks are skipped.

9. **Answer: True**. **Why:** It is a checked exception, so the compiler enforces one of those two plans before compiling.

10. **Answer: False**. **Why:** The variable is out of scope after the block - by then the file is closed, so there is nothing left to use it for anyway.

11. **Answer: True**. **Why:** That is why careful programmers say finally NORMALLY runs rather than always: if the JVM itself exits, or the process is killed, the finally block is skipped.

12. **Output:** Three lines: ok, then caught: negative!, then done. check(5) prints ok; check(-1) throws, so the rest of check and the call to check(9) are skipped; the catch prints the message; and done follows.

13. **Bug and fix:** Every run wipes diary.txt before writing, because FileWriter without the append flag starts from the beginning. The intent was clearly to add to the diary, so open it with the append flag: new FileWriter("diary.txt", StandardCharsets.UTF_8, true).

Why: Overwrite is the default; append must be requested with true as the extra argument. One missing boolean is the difference between keeping and losing a file's history.

14. **Model answer:**

PrintWriter collects text in a buffer; until the file is closed (or flushed) that text is not guaranteed to be on the disk, so a program that forgets close can leave an empty or half-written file. Closing also releases the file for other programs. Try-with-resources closes the resource automatically at the end of the block, on every normal path, and its variable cannot be used after the block.

15. **Model answer:**

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class FirstLine {

    public static void main(String[] args) {
        try {
            Scanner in = new Scanner(new File(
                    "welcome.txt"));
            if (in.hasNextLine()) {
                System.out.println(in.nextLine());
            } else {
                System.out.println("The file is "
                        + "empty");
            }
            in.close();
        } catch (FileNotFoundException e) {
            System.out.println("welcome.txt was "
                    + "not found");
        }
    }
}
```

Part 3 — Challenge Solutions

Chapter 1 — Getting to Know Java

Challenge 1.1 — Fix the Broken Program

Step 1. The class is named broken with a lowercase b, so the file name must be broken.java; either rename the file or rename the class to Broken and save it as Broken.java (the cleaner choice).

Step 2. system must be System with a capital S, because Java is case-sensitive.

Step 3. PrintLn with a capital P and L does not exist; the method is println.

Step 4. The first println statement is missing its ending semicolon.

Step 5. The second string literal never closes: a double quote is missing after the word Java, before the closing parenthesis.

```
public class Broken {

    public static void main(String[] args) {
        System.out.println("I am here!");
        System.out.println("Learning Java");
    }
}
```

Challenge 1.2 — Star Staircase

Step 1. Create the class Staircase with the standard main method.

Step 2. Print one star, then two, then three, then four, each with its own println.

Step 3. Double-check the star counts: 1, 2, 3, 4.

Step 4. Compile and run, then compare the output with the target, character by character.

```
public class Staircase {

    public static void main(String[] args) {
        System.out.println("*");
        System.out.println("**");
        System.out.println("***");
        System.out.println("****");
    }
}
```

Challenge 1.3 — Error Detective

Step 1. Message 1: the public class Greet is saved in a file with a different name (perhaps greet.java or Greeting.java). Rename the file to exactly Greet.java.

Step 2. Message 2: a statement is missing its ending semicolon, either on the reported line or the one before it. Add the semicolon.

Step 3. Message 3: the code spells System as system. Java is case-sensitive, so correct it to System.

```
// After all three fixes:
public class Greet {

    public static void main(String[] args) {
        System.out.println("Detective work done");
    }
}
```

Challenge 1.4 — Name Tag

Step 1. Plan all three lines on paper first, counting every dash and space.

Step 2. Print the top border with one println.

Step 3. Print the middle line: a |, two spaces, the name, one space, then a closing |. Adjust spaces so the width matches the borders.

Step 4. Print the bottom border, identical to the top one.

```
public class NameTag {

    public static void main(String[] args) {
        System.out.println("+------------------+");
        System.out.println("|  AMAZING CODER   |");
        System.out.println("+------------------+");
    }
}
```

Challenge 1.5 — Human Compiler

Step 1. The first two print statements write Java and is with trailing spaces on one growing line.

Step 2. println("fun.") finishes that line, so line one reads Java is fun.

Step 3. The last println starts a fresh line and prints Are you ready? there.

Step 4. Expected output: Java is fun. on line one and Are you ready? on line two. Run the program to confirm.

```
// Exact expected output:
// Java is fun.
// Are you ready?
```

Chapter 2 — Variables and Data Types

Challenge 2.1 — Fix the Greeting Machine

Step 1. The Scanner class lives in java.util, so the file needs import java.util.Scanner; above the public class line.

Step 2. The print statement is missing its ending semicolon: it must read System.out.print("Your name: ");

Step 3. string with a lowercase s is not a type; the type is String with a capital S.

Step 4. Compile and run; answering the prompt should print Hello, followed by the name you typed.

```
import java.util.Scanner;

public class GreetFix {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Your name: ");
        String name = in.next();
        System.out.println("Hello, " + name);
    }
}
```

Challenge 2.2 — Swap Machine

Step 1. Declare left = 17 and right = 42, and print both lines exactly as specified.

Step 2. Create int temp = left; to remember the original left value before it is overwritten.

Step 3. Assign left = right; then right = temp; to complete the swap.

Step 4. Print both lines again and check the values traded places.

```
public class SwapMachine {

    public static void main(String[] args) {
        int left = 17;
        int right = 42;
        System.out.println("Left: " + left);
        System.out.println("Right: " + right);

        int temp = left;
        left = right;
        right = temp;

        System.out.println("Left: " + left);
        System.out.println("Right: " + right);
    }
}
```

Challenge 2.3 — Escape Certificate

Step 1. Print line one with "Best Beginner" inside the string, escaping each double quote as \".

Step 2. Print line two with Earned by:, then \t, then the name.

Step 3. Print line three, escaping each backslash as \\ so the path prints with single backslashes.

Step 4. Run the program and compare all three lines with the target, watching the tab gap and the backslashes.

```
public class Certificate {

    public static void main(String[] args) {
        System.out.println("\"Best Beginner\"");
        System.out.println("Earned by:\tLayla");
        System.out.println("Saved to C:\\Certs\\best.txt");
    }
}
```

Challenge 2.4 — Profile Card

Step 1. Import Scanner, create the Scanner, and prompt for the name with print; read it with in.next().

Step 2. Prompt for the age and read it with in.nextInt().

Step 3. Prompt for the shoe size and read it with in.nextDouble().

Step 4. Print the three card lines using concatenation, then run the program and answer all three prompts to test it.

```
import java.util.Scanner;

public class ProfileCard {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Name (one word): ");
        String name = in.next();
        System.out.print("Age: ");
        int age = in.nextInt();
        System.out.print("Shoe size: ");
        double shoeSize = in.nextDouble();

        System.out.println("Name: " + name);
        System.out.println("Age: " + age + " years");
        System.out.println("Shoe size: " + shoeSize);
    }
}
```

Challenge 2.5 — Human Compiler: Variables

Step 1. a starts at 4. Then b = a + 2 computes 6 while a is still 4, so b becomes 6.

Step 2. a += b means a = a + b, so a becomes 4 + 6, which is 10.

Step 3. The first two printlns print a: 10 and b: 6.

Step 4. The last println glues the text a onto the number 10, printing a10; the expected output is a: 10, then b: 6, then a10.

```
public class Trace5 {

    public static void main(String[] args) {
        int a = 4;
        int b = a + 2;   // b is 6 (a was still 4)
        a += b;          // a becomes 4 + 6, so 10

        System.out.println("a: " + a);
        System.out.println("b: " + b);
        System.out.println("a" + a);
    }
}
// Expected output:
// a: 10
// b: 6
// a10
```

Chapter 3 — Operators and Expressions

Challenge 3.1 — Fix the Average Finder

Step 1. The sum a + b + c is 25, an int, because all three operands are ints.

Step 2. Then 25 / 3 is integer division: it truncates to 8, and only then is the 8 stored in the double as 8.0.

Step 3. Fix it by casting the sum: double average = (double) (a + b + c) / 3; which widens 25 to 25.0 before dividing.

Step 4. Run the fixed program; it now prints Average: 8.333333333333334, the double representation of 8.33.

```
public class AverageFix {

    public static void main(String[] args) {
        int a = 7;
        int b = 8;
        int c = 10;
        double average = (double) (a + b + c) / 3;
        System.out.println("Average: " + average);
    }
}
```

Challenge 3.2 — Digit Extractor

Step 1. Print number / 1000 for the thousands digit.

Step 2. Print number / 100 % 10 for the hundreds digit: the division leaves 49, and % 10 keeps the 9.

Step 3. Print number / 10 % 10 for the tens digit.

Step 4. Print number % 10 for the ones digit, then run the program and check that 4, 9, 2, 1 appear in order.

```
public class DigitExtractor {

    public static void main(String[] args) {
        int number = 4921;

        System.out.println(number / 1000);      // 4
        System.out.println(number / 100 % 10);  // 9
        System.out.println(number / 10 % 10);   // 2
        System.out.println(number % 10);        // 1
    }
}
```

Challenge 3.3 — Money Split

Step 1. Dollars are cents / 100; store the leftover with cents % 100 into a rest variable.

Step 2. Quarters are rest / 25; update rest with rest % 25.

Step 3. Dimes are rest / 10; update rest again, then nickels are rest / 5 and pennies are rest % 5.

Step 4. Print each line with its label, then verify the coins add back up to 287 cents (200 + 75 + 10 + 0 + 2).

```
public class MoneySplit {

    public static void main(String[] args) {
        int cents = 287;

        int dollars = cents / 100;   // 2
        int rest = cents % 100;      // 87
        int quarters = rest / 25;    // 3
        rest = rest % 25;            // 12
        int dimes = rest / 10;       // 1
        rest = rest % 10;             // 2
        int nickels = rest / 5;      // 0
        int pennies = rest % 5;       // 2

        System.out.println("Dollars: " + dollars);
        System.out.println("Quarters: " + quarters);
        System.out.println("Dimes: " + dimes);
        System.out.println("Nickels: " + nickels);
        System.out.println("Pennies: " + pennies);
    }
}
```

Challenge 3.4 — Time Machine

Step 1. Hours are totalSeconds / 3600; store the leftover with totalSeconds % 3600.

Step 2. Minutes are rest / 60 and seconds are rest % 60.

Step 3. Print the sentence in the exact requested shape using concatenation.

Step 4. Check by hand: 2 hours (7200 s) plus 3 minutes (180 s) plus 5 s is 7385 s.

```
public class TimeMachine {

    public static void main(String[] args) {
        int totalSeconds = 7385;

        int hours = totalSeconds / 3600;
        int rest = totalSeconds % 3600;
        int minutes = rest / 60;
        int seconds = rest % 60;

        System.out.println(totalSeconds + " seconds is "
                + hours + " h " + minutes + " m "
                + seconds + " s");
    }
}
```

Challenge 3.5 — Digit Sum

Step 1. Write the four digit expressions: number / 1000, number / 100 % 10, number / 10 % 10, and number % 10.

Step 2. Add them into one variable: int sum = ... + ... + ... + ...;

Step 3. Print the labeled result, then check the arithmetic: 4 + 9 + 2 + 1 = 16.

Step 4. Optional stretch: change the number to 8071 and predict the new digit sum (16 again!) before running.

```
public class DigitSum {

    public static void main(String[] args) {
        int number = 4921;

        int sum = number / 1000
                + number / 100 % 10
                + number / 10 % 10
                + number % 10;

        System.out.println("Digit sum: " + sum);
    }
}
```

Chapter 4 — Making Decisions

Challenge 4.1 — Grade Classifier with Input

Step 1. Set up the Scanner, prompt for the score, and read it into an int variable.

Step 2. First branch: if the score is below 0 or above 100 (join the two tests with ||), print Invalid score.

Step 3. Chain else-if branches from 90 down to 60, each printing the matching letter.

Step 4. End with an else that prints Grade: F, which also covers every score below 60.

Step 5. Test with 105 (invalid), -3 (invalid), 95, 84, 71, 60, and 12, checking each answer against the table.

```
import java.util.Scanner;

public class GradeClassifier {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter your score (0-100): ");
        int score = in.nextInt();
        if (score < 0 || score > 100) {
            System.out.println("Invalid score");
        } else if (score >= 90) {
            System.out.println("Grade: A");
        } else if (score >= 80) {
            System.out.println("Grade: B");
        } else if (score >= 70) {
            System.out.println("Grade: C");
        } else if (score >= 60) {
            System.out.println("Grade: D");
        } else {
            System.out.println("Grade: F");
        }
    }
}
```

Challenge 4.2 — Leap Year Checker (Fix and Finish)

Step 1. Change year % 4 = 0 to year % 4 == 0 so the program compiles; a single = is assignment, not comparison.

Step 2. Test with 1900: the current logic calls it a leap year, which is wrong, because 1900 is divisible by 100 but not by 400.

Step 3. Replace the condition with: divisible by 4 AND (not divisible by 100 OR divisible by 400). The parentheses around the OR are required.

Step 4. Merge the two ifs into a single if/else so every year produces exactly one line.

Step 5. Test with 2024, 1900, 2000, and 2023 and check each answer against the rules.

```
import java.util.Scanner;

public class LeapYear {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter a year: ");
        int year = in.nextInt();
        if (year % 4 == 0 && (year % 100 != 0
                || year % 400 == 0)) {
            System.out.println(year + " is a leap year");
        } else {
            System.out.println(year + " is not a leap year");
        }
    }
}
```

Challenge 4.3 — Rock, Paper, Scissors (Two Players)

Step 1. Read both numbers with two prompts and two nextInt calls.

Step 2. First branch: if either number is below 1 or above 3, print Invalid choice.

Step 3. Second branch: if the numbers are equal, print It is a tie!.

Step 4. Third branch: list the three winning pairings for player 1 (rock over scissors, paper over rock, scissors over paper), joined by ||, each wrapped in parentheses.

Step 5. Everything else must be a player 2 win, so a final else prints the message.

```
import java.util.Scanner;

public class RockPaperScissors {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Player 1 (1=rock, 2=paper, 3=scissors): ");
        int p1 = in.nextInt();
        System.out.print("Player 2 (1=rock, 2=paper, 3=scissors): ");
        int p2 = in.nextInt();
        if (p1 < 1 || p1 > 3 || p2 < 1 || p2 > 3) {
            System.out.println("Invalid choice");
        } else if (p1 == p2) {
            System.out.println("It is a tie!");
        } else if ((p1 == 1 && p2 == 3)
                || (p1 == 2 && p2 == 1)
                || (p1 == 3 && p2 == 2)) {
            System.out.println("Player 1 wins!");
        } else {
            System.out.println("Player 2 wins!");
        }
    }
}
```

Challenge 4.4 — Triangle Type Classifier

Step 1. Read three ints a, b, and c, each with its own prompt.

Step 2. First branch: if any side is zero or below (join the three tests with ||), print Invalid sides.

Step 3. Second branch: a == b && b == c means all three match, so print Equilateral.

Step 4. Third branch: a == b || b == c || a == c means exactly two match, so print Isosceles.

Step 5. The final else is Scalene; test with 3 3 3, 4 4 7, 5 6 7, and 0 4 4.

```
import java.util.Scanner;

public class TriangleType {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Side 1: ");
        int a = in.nextInt();
        System.out.print("Side 2: ");
        int b = in.nextInt();
        System.out.print("Side 3: ");
        int c = in.nextInt();
        if (a <= 0 || b <= 0 || c <= 0) {
            System.out.println("Invalid sides");
        } else if (a == b && b == c) {
            System.out.println("Equilateral");
        } else if (a == b || b == c || a == c) {
            System.out.println("Isosceles");
        } else {
            System.out.println("Scalene");
        }
    }
}
```

Challenge 4.5 — BMI Category Calculator

Step 1. Set up the Scanner and read two doubles: weight and height.

Step 2. Compute double bmi = weight / Math.pow(height, 2); and print it once with a label.

Step 3. Chain: below 18.5 prints Underweight; else below 25 prints Normal weight; else below 30 prints Overweight; else prints Obese.

Step 4. Because earlier branches already failed, each else-if needs only its upper bound; first-match branching does the range work for you.

Step 5. Test with 60 kg and 1.7 m (Normal weight) and 95 kg and 1.7 m (Obese).

```
import java.util.Scanner;

public class BmiCategory {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Weight in kg: ");
        double weight = in.nextDouble();
        System.out.print("Height in meters: ");
        double height = in.nextDouble();
        double bmi = weight / Math.pow(height, 2);
        System.out.println("BMI: " + bmi);
        if (bmi < 18.5) {
            System.out.println("Underweight");
        } else if (bmi < 25) {
            System.out.println("Normal weight");
        } else if (bmi < 30) {
            System.out.println("Overweight");
        } else {
            System.out.println("Obese");
        }
    }
}
```

Chapter 5 — Loops

Challenge 5.1 — FizzBuzz Warm-Up

Step 1. Loop i from 1 to 15 with a for loop.

Step 2. First branch: if (i % 15 == 0) print FizzBuzz, because multiples of 15 are exactly the multiples of both 3 and 5.

Step 3. Else if (i % 3 == 0) print Fizz; else if (i % 5 == 0) print Buzz.

Step 4. Else print the number itself. Test the order by trying i = 15 mentally: only the first branch may catch it.

```
public class FizzBuzz {

    public static void main(String[] args) {
        for (int i = 1; i <= 15; i++) {
            if (i % 15 == 0) {
                System.out.println("FizzBuzz");
            } else if (i % 3 == 0) {
                System.out.println("Fizz");
            } else if (i % 5 == 0) {
                System.out.println("Buzz");
            } else {
                System.out.println(i);
            }
        }
    }
}
```

Challenge 5.2 — Digit Sum

Step 1. Store 4729 in a variable n and make an int sum = 0 accumulator.

Step 2. While n > 0: add n % 10 to sum, then set n = n / 10.

Step 3. Trace: 4729 adds 9, n becomes 472; adds 2, n becomes 47; adds 7, n becomes 4; adds 4, n becomes 0.

Step 4. After the loop, print the sum (22).

```
public class DigitSum {

    public static void main(String[] args) {
        int n = 4729;
        int sum = 0;
        while (n > 0) {
            sum += n % 10;
            n = n / 10;
        }
        System.out.println(sum);
    }
}
```

Challenge 5.3 — Countdown on One Line

Step 1. Loop i from 3 down to 1, printing i followed by three dots and a space, using print.

Step 2. Keep the loop body to one print statement; do not end the line inside the loop.

Step 3. After the loop, print GO! with println to finish the line.

Step 4. Compare the output character by character, including the spaces after the dots.

```
public class OneLine {

    public static void main(String[] args) {
        for (int i = 3; i >= 1; i--) {
            System.out.print(i + "... ");
        }
        System.out.println("GO!");
    }
}
```

Challenge 5.4 — Star Triangle

Step 1. Outer for loop: row from 1 to 5.

Step 2. Inner for loop: star from 1 to row, printing * with print (no newline).

Step 3. After the inner loop, one println moves to the next row.

Step 4. Check the pattern: row 3 must print exactly three stars.

```
public class Triangle {

    public static void main(String[] args) {
        for (int row = 1; row <= 5; row++) {
            for (int star = 1; star <= row; star++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

Challenge 5.5 — Collatz Counter

Step 1. Start with n = 27 and an int steps = 0 counter.

Step 2. While n != 1: if n % 2 == 0 then n = n / 2, else n = 3 * n + 1; then add 1 to steps.

Step 3. Trace a few steps to trust the loop: 27 -> 82 -> 41 -> 124 ...

Step 4. When the loop ends, print steps (the expected value is 111).

```
public class Collatz {

    public static void main(String[] args) {
        int n = 27;
        int steps = 0;
        while (n != 1) {
            if (n % 2 == 0) {
                n = n / 2;
            } else {
                n = 3 * n + 1;
            }
            steps++;
        }
        System.out.println(steps);
    }
}
```

Chapter 6 — Methods

Challenge 6.1 — Greeting Machine

Step 1. Create the class and main, then write the greet header exactly as specified: String name first, int times second.

Step 2. Inside greet, add a for loop from 0 up to but not including times.

Step 3. In the loop body, print "Hello, " + name + "!" with println so each greeting lands on its own line.

Step 4. In main, call greet("Layla", 2); and greet("Sam", 3);, then run and count: two Layla lines, then three Sam lines.

```
public class GreetingMachine {

    public static void main(String[] args) {
        greet("Layla", 2);
        greet("Sam", 3);
    }

    // Prints a greeting for name, times times.
    public static void greet(String name, int times) {
        for (int i = 0; i < times; i++) {
            System.out.println("Hello, " + name
                    + "!");
        }
    }
}
```

Challenge 6.2 — Even or Odd Reporter

Step 1. Write isEven so its body is the single line return n % 2 == 0; the comparison already produces a boolean.

Step 2. In main, loop a variable n from 1 to 10 inclusive.

Step 3. Inside the loop, use if (isEven(n)) to choose between printing n + " is even" and n + " is odd".

Step 4. Run it and check the full list from 1 is odd through 10 is even.

```
public class EvenReporter {

    public static void main(String[] args) {
        for (int n = 1; n <= 10; n++) {
            if (isEven(n)) {
                System.out.println(n + " is even");
            } else {
                System.out.println(n + " is odd");
            }
        }
    }

    // Returns true when n divides evenly by 2.
    public static boolean isEven(int n) {
        return n % 2 == 0;
    }
}
```

Challenge 6.3 — Max of Three

Step 1. Declare int best = a; as the current champion inside max3.

Step 2. Add if (b > best) { best = b; } so b can take the crown.

Step 3. Add the same statement for c, then return best.

Step 4. Call max3 three times from main with the requested values; the expected results are 9, -2, and 7.

```
public class MaxOfThree {

    public static void main(String[] args) {
        System.out.println(max3(4, 9, 2));
        System.out.println(max3(-5, -2, -9));
        System.out.println(max3(7, 7, 3));
    }

    // Returns the largest of three values.
    public static int max3(int a, int b, int c) {
        int best = a;
        if (b > best) {
            best = b;
        }
        if (c > best) {
            best = c;
        }
        return best;
    }
}
```

Challenge 6.4 — Temperature Converter

Step 1. Write the method header with double as both return type and parameter type.

Step 2. Return (f - 32.0) * 5.0 / 9.0; the parentheses matter because subtraction must happen first.

Step 3. In main, print three calls: fahrenheitToCelsius(32.0), fahrenheitToCelsius(68.0), and fahrenheitToCelsius(212.0).

Step 4. Expected output: 0.0, 20.0, and 100.0. If you ever pass a value like 98.6, do not be surprised by a long decimal; that is floating-point arithmetic from Chapter 3.

```
public class TempConverter {

    public static void main(String[] args) {
        System.out.println(fahrenheitToCelsius(32.0));
        System.out.println(fahrenheitToCelsius(68.0));
        System.out.println(fahrenheitToCelsius(212.0));
    }

    // Converts Fahrenheit degrees to Celsius.
    public static double fahrenheitToCelsius(double f) {
        return (f - 32.0) * 5.0 / 9.0;
    }
}
```

Challenge 6.5 — Table Trouble

Step 1. The line printTable; names the method but never calls it: a call needs parentheses and arguments, so it must become printTable(2);.

Step 2. The next call printTable(3) is missing its ending semicolon; add it.

Step 3. Add a third call, printTable(4);, so tables for 2, 3, and 4 all appear.

Step 4. Run the program: each table has ten lines like 2 x 1 = 2, and 4 x 10 = 40 ends the output.

```
public class TableTrouble {

    public static void main(String[] args) {
        printTable(2);
        printTable(3);
        printTable(4);
    }

    // Prints the multiplication table for n.
    public static void printTable(int n) {
        for (int i = 1; i <= 10; i++) {
            System.out.println(n + " x " + i
                    + " = " + n * i);
        }
    }
}
```

Chapter 7 — Arrays

Challenge 7.1 — Digit Counter

Step 1. Create int[] counts = new int[10]; so counts[0] counts 0s, counts[1] counts 1s, and so on.

Step 2. Copy the number into a working variable remaining, and loop while remaining > 0.

Step 3. Inside the loop: int digit = remaining % 10; then counts[digit] = counts[digit] + 1; then remaining = remaining / 10;.

Step 4. After the loop, print each digit d from 0 to 9 only when counts[d] > 0, for example 7 appears 3 time(s).

Step 5. Check the totals by hand: 7 appears three times, 2 twice, and 1 and 3 once each.

```
public class DigitCounter {

    public static void main(String[] args) {
        int number = 7721237;
        int[] counts = new int[10];

        int remaining = number;
        while (remaining > 0) {
            int digit = remaining % 10;
            counts[digit] = counts[digit] + 1;
            remaining = remaining / 10;
        }

        for (int d = 0; d < counts.length; d++) {
            if (counts[d] > 0) {
                System.out.println(d + " appears "
                        + counts[d] + " time(s)");
            }
        }
    }
}
```

Challenge 7.2 — Reverse the List

Step 1. Import java.util.Scanner, create the Scanner, and make int[] nums = new int[5];.

Step 2. Loop i from 0 to 4 and store each entered value with nums[i] = in.nextInt();.

Step 3. Print the heading line, then loop i from 4 down to 0, printing nums[i] each pass.

Step 4. Test with the values 3, 8, 1, 9, 4; the output should read 4, 9, 1, 8, 3 from top to bottom.

```
import java.util.Scanner;

public class ReverseInput {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int[] nums = new int[5];

        System.out.println("Enter 5 whole numbers:");
        for (int i = 0; i < nums.length; i++) {
            nums[i] = in.nextInt();
        }

        System.out.println("In reverse order:");
        for (int i = nums.length - 1; i >= 0; i--) {
            System.out.println(nums[i]);
        }
    }
}
```

Challenge 7.3 — Second Largest

Step 1. Initialize both trackers to the first element: int best = values[0]; int second = values[0];.

Step 2. Loop from index 1 onward. If values[i] > best, set second = best; then best = values[i];.

Step 3. Else if values[i] > second && values[i] != best, set second = values[i];.

Step 4. After the loop, print both trackers and confirm the output is 47 and then 30.

```
public class SecondLargest {

    public static void main(String[] args) {
        int[] values = {12, 47, 3, 47, 30};

        int best = values[0];
        int second = values[0];
        for (int i = 1; i < values.length; i++) {
            if (values[i] > best) {
                second = best;
                best = values[i];
            } else if (values[i] > second
                    && values[i] != best) {
                second = values[i];
            }
        }
        System.out.println("Largest: " + best);
        System.out.println("Second largest: "
                + second);
    }
}
```

Challenge 7.4 — Grade Buckets

Step 1. Create int[] buckets = new int[5]; for A, B, C, D, and F in that order.

Step 2. Loop over the scores, and for each one use if/else-if from s >= 90 down to s >= 60, with a final else for F.

Step 3. Inside each branch, add 1 to the matching counter, for example buckets[0] = buckets[0] + 1; for A.

Step 4. Print the five lines; the expected counts are A: 2, B: 2, C: 1, D: 1, F: 2.

```
public class GradeBuckets {

    public static void main(String[] args) {
        int[] scores = {95, 72, 88, 61, 55, 100,
                83, 47};
        int[] buckets = new int[5];

        for (int i = 0; i < scores.length; i++) {
            int s = scores[i];
            if (s >= 90) {
                buckets[0] = buckets[0] + 1;
            } else if (s >= 80) {
                buckets[1] = buckets[1] + 1;
            } else if (s >= 70) {
                buckets[2] = buckets[2] + 1;
            } else if (s >= 60) {
                buckets[3] = buckets[3] + 1;
            } else {
                buckets[4] = buckets[4] + 1;
            }
        }

        System.out.println("A: " + buckets[0]);
        System.out.println("B: " + buckets[1]);
        System.out.println("C: " + buckets[2]);
        System.out.println("D: " + buckets[3]);
        System.out.println("F: " + buckets[4]);
    }
}
```

Challenge 7.5 — Magic Square Checker

Step 1. Create the grid with the initializer, and declare boolean magic = true;.

Step 2. Loop r from 0 to 2, computing each row sum and setting magic to false when it is not 15.

Step 3. Loop c from 0 to 2, computing each column sum the same way.

Step 4. Compute the two diagonals: square[0][0] + square[1][1] + square[2][2] and square[0][2] + square[1][1] + square[2][0].

Step 5. Print the verdict; the given grid is the classic Lo Shu square, so every line sums to 15 and the result is true.

```
public class MagicSquare {

    public static void main(String[] args) {
        int[][] square = {
            {8, 1, 6},
            {3, 5, 7},
            {4, 9, 2}
        };
        boolean magic = true;

        for (int r = 0; r < 3; r++) {
            int rowSum = square[r][0]
                    + square[r][1] + square[r][2];
            if (rowSum != 15) {
                magic = false;
            }
        }
        for (int c = 0; c < 3; c++) {
            int colSum = square[0][c]
                    + square[1][c] + square[2][c];
            if (colSum != 15) {
                magic = false;
            }
        }
        int diag1 = square[0][0] + square[1][1]
                + square[2][2];
        int diag2 = square[0][2] + square[1][1]
                + square[2][0];
        if (diag1 != 15 || diag2 != 15) {
            magic = false;
        }

        System.out.println("Magic square: "
                + magic);
    }
}
```

Chapter 8 — Working with Strings

Challenge 8.1 — Vowel Counter

Step 1. Store the sentence, then immediately convert it: String lower = sentence.toLowerCase(); - now you only need to look for lowercase vowels.

Step 2. Loop from 0 to lower.length() - 1, pulling each character out with charAt(i) into a char variable.

Step 3. Test the character against 'a', 'e', 'i', 'o', 'u' with one if joined by || and add 1 to a counter when any of them matches.

Step 4. After the loop, print the counter with a friendly label, and check the total by hand for a short sentence.

```
public class VowelCount {

    public static void main(String[] args) {
        String sentence = "Java Is Fun To Learn";
        String lower = sentence.toLowerCase();
        int vowels = 0;
        for (int i = 0; i < lower.length(); i++) {
            char c = lower.charAt(i);
            if (c == 'a' || c == 'e' || c == 'i'
                    || c == 'o' || c == 'u') {
                vowels++;
            }
        }
        System.out.println("Vowels: " + vowels);
    }
}
```

Challenge 8.2 — Initials Extractor

Step 1. Create the three String variables with any names you like, in lower or upper case.

Step 2. Extract each initial with charAt(0) into a char variable: the first letter is always at index 0.

Step 3. Print the initials with + and "." between them; remember that mixing a char with Strings performs text concatenation.

Step 4. Change the names and run again to make sure the program works for any three words.

```
public class Initials {

    public static void main(String[] args) {
        String first = "Maya";
        String middle = "Clarice";
        String last = "Reyes";
        char f = first.charAt(0);
        char m = middle.charAt(0);
        char l = last.charAt(0);
        System.out.println(f + "." + m + "."
                + l + ".");
    }
}
```

Challenge 8.3 — Palindrome Checker

Step 1. The bug is the == comparison: word and backwards are two different String objects with the same letters, so == reports false every time.

Step 2. Replace == with word.equals(backwards) so the test compares contents.

Step 3. Add a Scanner, print a prompt, and read the user's word with nextLine().

Step 4. Build the backwards version with StringBuilder: create it from the word, call reverse(), then toString() to get a String you can compare.

Step 5. Report the verdict with println, including the word itself in the message.

```
import java.util.Scanner;

public class Palindrome {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter a word: ");
        String word = in.nextLine();
        StringBuilder sb = new StringBuilder(word);
        sb.reverse();
        String backwards = sb.toString();
        if (word.equals(backwards)) {
            System.out.println(word
                    + " is a palindrome!");
        } else {
            System.out.println(word
                    + " is not a palindrome");
        }
    }
}
```

Challenge 8.4 — Word Counter

Step 1. Store the sentence and create a space counter set to 0.

Step 2. Loop over every index and use charAt(i) == ' ' to detect spaces, adding 1 to the counter for each one.

Step 3. After the loop, decide the word count: if the sentence length is 0, the count is 0; otherwise it is spaces + 1.

Step 4. Print the result, then test with an empty string and with a one-word sentence to verify both edges.

```
public class WordCount {

    public static void main(String[] args) {
        String sentence = "the quick brown fox jumps";
        int spaces = 0;
        for (int i = 0; i < sentence.length(); i++) {
            if (sentence.charAt(i) == ' ') {
                spaces++;
            }
        }
        int words;
        if (sentence.length() == 0) {
            words = 0;
        } else {
            words = spaces + 1;
        }
        System.out.println("Words: " + words);
    }
}
```

Challenge 8.5 — Acronym Builder

Step 1. Create the String array with the words of your phrase.

Step 2. Create an empty StringBuilder before the loop.

Step 3. Use a for-each loop over the array and append w.charAt(0) for each word - the first letter is always at index 0.

Step 4. After the loop, convert with toString(), uppercase with toUpperCase(), and print the acronym.

```
public class Acronym {

    public static void main(String[] args) {
        String[] words = {"personal", "digital",
                "assistant"};
        StringBuilder sb = new StringBuilder();
        for (String w : words) {
            sb.append(w.charAt(0));
        }
        System.out.println(sb.toString().toUpperCase());
    }
}
```

Chapter 9 — Files and Exceptions

Challenge 9.1 — Line Number Copier

Step 1. Open both files inside one try block: a Scanner on notes.txt for reading and a PrintWriter on a FileWriter for numbered.txt.

Step 2. Keep an int counter starting at 1; loop while in.hasNextLine(), and write counter + ": " + the line to the writer, adding 1 to the counter each pass.

Step 3. Close both the reader and the writer after the loop, then print a confirmation message.

Step 4. Catch IOException once below the try and print a clear message with e.getMessage(); test by renaming notes.txt away and running again.

```
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.charset.StandardCharsets;
import java.util.Scanner;

public class LineCopier {

    public static void main(String[] args) {
        try {
            Scanner in = new Scanner(new File(
                    "notes.txt"));
            PrintWriter out = new PrintWriter(
                    new FileWriter("numbered.txt",
                            StandardCharsets.UTF_8));
            int n = 1;
            while (in.hasNextLine()) {
                out.println(n + ": "
                        + in.nextLine());
                n++;
            }
            in.close();
            out.close();
            System.out.println("Copied with "
                    + "line numbers");
        } catch (IOException e) {
            System.out.println("File trouble: "
                    + e.getMessage());
        }
    }
}
```

Challenge 9.2 — Diary Appender

Step 1. Create a Scanner on System.in and print instructions: type the entry, finish with a line containing END.

Step 2. Inside try-with-resources, open a PrintWriter on new FileWriter("diary.txt", StandardCharsets.UTF_8, true) - the true is what protects old entries.

Step 3. Loop forever: read a line with nextLine(), break when it equals "END", otherwise out.println(line).

Step 4. After the loop, print a confirmation, and keep a catch (IOException e) below the try-with-resources.

Step 5. Run the program twice and open diary.txt to confirm both entries survived.

```
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.charset.StandardCharsets;
import java.util.Scanner;

public class DiaryApp {

    public static void main(String[] args) {
        Scanner keyboard = new Scanner(System.in);
        System.out.println("Type your entry.");
        System.out.println("Finish with END on "
                + "its own line");
        try (PrintWriter out = new PrintWriter(
                new FileWriter("diary.txt",
                        StandardCharsets.UTF_8,
                        true))) {
            while (true) {
                String line = keyboard.nextLine();
                if (line.equals("END")) {
                    break;
                }
                out.println(line);
            }
            System.out.println("Entry saved");
        } catch (IOException e) {
            System.out.println("Could not "
                    + "write: "
                    + e.getMessage());
        }
    }
}
```

Challenge 9.3 — Number-Sum Reader

Step 1. Create an int total starting at 0.

Step 2. Inside try/catch for FileNotFoundException, open a Scanner on numbers.txt.

Step 3. Loop while in.hasNextInt(), adding in.nextInt() to the total each pass, then close the Scanner.

Step 4. Print the total after the loop, and print the friendly missing-file message in the catch block.

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class SumFile {

    public static void main(String[] args) {
        int total = 0;
        try {
            Scanner in = new Scanner(new File(
                    "numbers.txt"));
            while (in.hasNextInt()) {
                total = total + in.nextInt();
            }
            in.close();
            System.out.println("Sum: " + total);
        } catch (FileNotFoundException e) {
            System.out.println("numbers.txt "
                    + "was not found");
        }
    }
}
```

Challenge 9.4 — Echo Machine

Step 1. Stage one: inside try-with-resources, open a PrintWriter on echo.txt and println three lines, then let the block close the file for you; keep a catch (IOException e) and return from main inside it if writing failed.

Step 2. Stage two: in a fresh try/catch for FileNotFoundException, open a Scanner on echo.txt.

Step 3. Loop with in.hasNextLine() and println each line read, then close the Scanner.

Step 4. Run the program and confirm the three lines come back; then open echo.txt in your editor to see that the screen output matches the file.

```
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.charset.StandardCharsets;
import java.util.Scanner;

public class EchoMachine {

    public static void main(String[] args) {
        try (PrintWriter out = new PrintWriter(
                new FileWriter("echo.txt",
                        StandardCharsets.UTF_8))) {
            out.println("Hello from the file!");
            out.println("Second line");
            out.println("Goodbye");
        } catch (IOException e) {
            System.out.println("Could not "
                    + "write: "
                    + e.getMessage());
            return;
        }
        try {
            Scanner in = new Scanner(new File(
                    "echo.txt"));
            while (in.hasNextLine()) {
                System.out.println(in.nextLine());
            }
            in.close();
        } catch (FileNotFoundException e) {
            System.out.println("Reading "
                    + "failed: "
                    + e.getMessage());
        }
    }
}
```

Challenge 9.5 — Safe Divider

Step 1. Create a Scanner, prompt for the first number, and read it as a word with in.next(); do the same for the second.

Step 2. Inside one try block, convert both words with Integer.parseInt - a text like abc throws NumberFormatException here.

Step 3. Still inside the try, test the second number with an if: if it is 0, print Cannot divide by zero; otherwise print the result of a / b.

Step 4. After the try, add catch (NumberFormatException e) that prints Please type whole numbers only, and test all three paths: normal numbers, a zero divisor, and letters.

```
import java.util.Scanner;

public class SafeDivider {

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("First number: ");
        String first = in.next();
        System.out.print("Second number: ");
        String second = in.next();
        try {
            int a = Integer.parseInt(first);
            int b = Integer.parseInt(second);
            if (b == 0) {
                System.out.println("Cannot "
                        + "divide by zero");
            } else {
                System.out.println("Answer: "
                        + (a / b));
            }
        } catch (NumberFormatException e) {
            System.out.println("Please type "
                    + "whole numbers only");
        }
    }
}
```

REFERENCE

Index

Topics, keywords, and methods with the chapter and lesson where they are taught. Format: **Ch chapter · L lesson**.

Topic / Keyword / Method	Where it is taught
+=, -=, *=, /=, %=	Ch 2 L2
addStudent (pattern)	Final Project Step 2
++ and --	Ch 5 L2
append mode (files)	Ch 9 L4
argument vs parameter	Ch 6 L2
arithmetic operators	Ch 3 L1
ArrayIndexOutOfBoundsException	Ch 7 L1
arrays	Ch 7 L1
arrays, 2D	Ch 7 L4
arrays, default values	Ch 7 L1
arrays, parallel	Final Project Step 1
assignment =	Ch 2 L1
avg pattern (accumulator)	Ch 5 L2
boolean type	Ch 2 L4
break (loops)	Ch 5 L4
break (switch)	Ch 4 L4
bytecode (.class)	Ch 1 L2
case (switch)	Ch 4 L4
cast, truncating	Ch 3 L3
charAt	Ch 8 L1
checked exceptions	Ch 9 L3
class declaration	Ch 1 L3
classes, naming files	Ch 1 L2
collatz pattern	Ch 5 Challenge 5
comments /* */	Ch 1 L4
comments //	Ch 1 L4
compareTo	Ch 8 L2
compile-time errors	Ch 1 L4

Topic / Keyword / Method	Where it is taught
compiler javac	Ch 1 L2
concatenation +	Ch 2 L3
conditional logic	Ch 4 L1
conditions (true/false)	Ch 4 L1
continue	Ch 5 L4
current working directory	Ch 9 L3
default (switch)	Ch 4 L4
default values, arrays	Ch 7 L1
digit counters (arrays)	Ch 7 Challenge 1
/ (division)	Ch 3 L1
do-while loop	Ch 5 L3
(double) cast	Ch 3 L3
else	Ch 4 L2
else-if chains, order matters	Ch 4 L2
empty string	Ch 8 L2
encoding UTF-8	Ch 9 L3
equals (strings)	Ch 8 L2
equalsIgnoreCase	Ch 8 L2
escape sequences	Ch 2 L3
exceptions	Ch 9 L1
expression	Ch 3 L1
fall-through (switch)	Ch 4 L4
File (class)	Ch 9 L3
FileNotFoundException	Ch 9 L3
FileWriter	Ch 9 L4
finally	Ch 9 L2
FizzBuzz pattern	Ch 5 Challenge 1
flags (boolean)	Ch 4 L3
floating-point numbers	Ch 2 L2
for loop	Ch 5 L2
for-each loop	Ch 7 L2
format strings, printf	Ch 8 L3
hasNextLine	Ch 9 L3
identifiers (names)	Ch 2 L1

Topic / Keyword / Method	Where it is taught
if statement	Ch 4 L1
immutable strings	Ch 8 L2
import	Ch 2 L4
indexOf	Ch 8 L1
infinite loops	Ch 5 L1
int type	Ch 2 L2
(int) cast	Ch 3 L3
integer division	Ch 3 L2
/ (integer division)	Ch 3 L2
IOException	Ch 9 L4
Javadoc comments	Ch 6 L4
JDK	Ch 1 L2
JVM	Ch 1 L2
keywords	Ch 1 L3
length (arrays, field)	Ch 7 L1
length() (strings, method)	Ch 8 L1
linear search	Ch 7 L3
logic errors	Ch 1 L4
&& (logical AND)	Ch 3 L4
! (logical NOT)	Ch 3 L4
logical operators	Ch 3 L4
|| (logical OR)	Ch 3 L4
main method	Ch 1 L3
Math.abs / min / max	Ch 3 L3
Math.pow / sqrt	Ch 3 L3
Math.random	Mini-Project 2 Step 1
Math.round	Ch 3 L3
max/min in arrays	Ch 7 L3
methods	Ch 6 L1
modulo %	Ch 3 L2
nested if	Ch 4 L3
nested loops	Ch 5 L4
nextInt	Ch 2 L4
nextLine (pitfall after nextInt)	Ch 2 L4

Topic / Keyword / Method	Where it is taught
null (String arrays)	Ch 7 L1
off-by-one errors	Ch 5 Mistakes
+ operator (numbers)	Ch 3 L1
+ operator (strings)	Ch 2 L3
operator precedence	Ch 3 L3
operators, arithmetic	Ch 3 L1
overloading	Ch 6 L4
parameters	Ch 6 L2
pass-by-value (primitives)	Ch 6 L2
predefined Math methods	Ch 3 L3
predefined methods	Ch 1 L3
primitive types	Ch 2 L2
print vs println	Ch 1 L3
printf	Ch 8 L3
println	Ch 1 L3
PrintWriter	Ch 9 L4
range checks with &&	Ch 4 L3
remainder %	Ch 3 L2
% (remainder)	Ch 3 L2
return statement	Ch 6 L3
return values	Ch 6 L3
reverse a string	Ch 8 L4
runtime errors	Ch 1 L4
Scanner (files)	Ch 9 L3
Scanner (keyboard)	Ch 2 L4
scope of variables	Ch 6 L4
sentinel values	Ch 5 L4
short-circuit evaluation	Ch 3 L4
sort (arrays), Arrays.sort	Ch 7 L4
Statements, semicolons	Ch 1 L3
String formatting	Ch 8 L3
String type	Ch 2 L3
StringBuilder	Ch 8 L4
strings, comparing (==)	Ch 8 L2

Topic / Keyword / Method	Where it is taught
substring	Ch 8 L1
switch statement	Ch 4 L4
throw	Ch 9 L2
throws (declaration)	Ch 9 L4
toString (Arrays)	Ch 7 L4
toUpperCase/toLowerCase	Ch 8 L1
trim	Ch 8 L1
truth tables (AND/OR/NOT)	Ch 3 L4
try-with-resources	Ch 9 L4
try/catch	Ch 9 L1
type casting	Ch 3 L3
Type conversion, widening	Ch 3 L3
variables	Ch 2 L1
while loop	Ch 5 L1
writing files	Ch 9 L4

FIRST EDITION

Java from Zero

A Beginner's Guide to Programming in Java

INSIDE THIS BOOK

01 **36 small lessons** across 9 chapters, each with runnable code you can type and run

02 **144 Lesson Check questions** that verify understanding immediately

03 **90 practice problems** per-chapter sets ordered from easy to hard

04 **135 mixed quiz questions** that test every idea from the chapter

05 **45 challenges** with hints and fully worked, step-by-step solutions

06 **3 mini-projects**: a cafe calculator, a guessing game, and a journal app

07 **1 final project**: a complete grade book combining every concept

08 **Detailed answer key** explaining why each answer is right — and why the others are wrong

09 **Glossary of 60+ terms** and a full index pointing to every lesson

> *"Programs must be written for people to read, and only incidentally for machines to execute."*
>
> — HAROLD ABELSON

DIRECTED BY

mohamed ahmed fayad

BEGINNER LEVEL · SCHOOL STUDENTS & SELF-LEARNERS